Technical Reference Manual for Myshkin

Revised November 4th, 2026

Contents

1 System Configuration **7**

2 Packaging and Pins Configuration **9**

3 Address Space **12**

4 Startup and Reset Behavior **13**

5 Interrupts **14**
5.1 Interrupt Vector Table . . . 14
5.2 Recursive Interrupt Handling . . . 14
5.3 Sleep Mode and Interrupt Wake-Up . . . 15

6 CPU Instructions and Assembly **16**
6.1 CPU Registers . . . 16
6.2 Load/Store Memory Instructions . . . 17
6.3 Shift Instructions . . . 17
6.4 Arithmetic Instructions . . . 18
6.5 Bitwise Instructions . . . 18
6.6 Comparison Instructions . . . 19
6.7 Branch Instructions . . . 19
6.8 Jump and Link Instructions . . . 20
6.9 Atomic Memory Operations (A Extension) . . . 20
6.10 Bit Manipulation Instructions (Zb* Extensions) . . . 21
6.10.1 Address Generation (Zba) . . . 21
6.10.2 Basic Bit Manipulation (Zbb) . . . 21
6.10.3 Carryless Multiplication (Zbc) . . . 22
6.10.4 Single-Bit Manipulation (Zbs) . . . 22
6.11 Pseudo-Instructions . . . 23

7 Forth Interpreter **25**
7.1 Memory Access Functions . . . 25
7.2 Print and Input Functions . . . 26
7.3 Arithmetic Functions . . . 27
7.4 Bitwise Logic Functions . . . 29
7.5 Comparison Functions . . . 31
7.6 Boolean Functions . . . 31
7.7 Stack Manipulation Functions . . . 32
7.8 SPI Flash R/W and Programming Functions . . . 34
7.9 Miscellaneous Functions . . . 34
7.10 Creating New Forth Functions . . . 36
7.11 Conditionals . . . 36
7.12 Loops . . . 36

8 Software Development and Build Process **37**
8.1 The RISC-V Toolchain and its Extensions . . . 37
8.2 Getting the RISC-V Toolchain on Linux . . . 37
8.2.1 Option 1: Prebuilt xPack Toolchain (Recommended) . . . 38
8.2.2 Option 2: Building from Source . . . 38
8.3 Getting the RISC-V Toolchain on Windows . . . 39
8.4 Compilation . . . 39
8.5 Linking . . . 41
8.6 Intel Hex File Generation . . . 41
8.7 A Note on Using C++ . . . 42

9 Uploading Program to SPI Flash **43**
9.1 SPI Flash Boot Process . . . 43
9.2 Upload Process . . . 43

10 `GPIOx` Peripheral **45**
10.1 Primary/GPIO Mode Functionality . . . 45
10.2 Register Access Convenience Features . . . 45
10.3 Secondary/Peripheral Mode Functionality . . . 46
10.4 Pin Interrupts . . . 46
10.5 Register Description . . . 47
10.5.1 `PIN` Register . . . 47
10.5.2 `POUT` Register . . . 47
10.5.3 `POUTS` Register . . . 47
10.5.4 `POUTC` Register . . . 47
10.5.5 `POUTT` Register . . . 48
10.5.6 `PDIR` Register . . . 48
10.5.7 `PIF` Register . . . 48
10.5.8 `PIES` Register . . . 48
10.5.9 `PIE` Register . . . 49
10.5.10 `PSEL` Register . . . 49
10.5.11 `PREN` Register . . . 49

11 `SPIx` Peripheral **50**
11.1 SPI Pins . . . 50
11.2 Generalized SPI Transfer Process . . . 51
11.3 Bit and Byte Ordering . . . 52
11.4 SPI Baud Clock . . . 53
11.5 Transmit Buffer Register Queue . . . 53
11.6 Flags and Interrupts . . . 53
11.7 Master Mode Transfer Procedure . . . 53
11.8 Slave Mode Transfer Procedure . . . 55
11.9 External SPI Flash Extended Memory Access . . . 55
11.9.1 Flash Address Translation . . . 55
11.9.2 Flash Initialization Procedure . . . 56
11.10 `SPI0` Boot Behavior . . . 56
11.11 Register Description . . . 57
11.11.1 `SPICR` Register . . . 57

11.11.2 SPISR Register . . . 58
11.11.3 SPITX Register . . . 59
11.11.4 SPIRX Register . . . 59
11.11.5 SPIFOS Register . . . 60

12 UARTx Peripheral **62**
12.1 UART Pins . . . 62
12.2 Generalized UART Transmission Process . . . 63
12.3 UART Baud Generation . . . 63
12.4 Parity Bit . . . 63
12.5 Transmit Buffer Register Queue . . . 64
12.6 Flags and Interrupts . . . 64
12.7 Transmit Procedure . . . 65
12.8 Receive Procedure . . . 65
12.9 UART0 Boot Behavior . . . 66
12.10 Register Description . . . 67
12.10.1 UARTCR Register . . . 67
12.10.2 UARTSR Register . . . 67
12.10.3 UARTBR Register . . . 68
12.10.4 UARTRX Register . . . 69
12.10.5 UARTTX Register . . . 69

13 TIMERx Peripheral **70**
13.1 Timer Pins . . . 70
13.2 Clock Sources and Divider . . . 70
13.3 Starting, Halting, Setting, and Reading the Timer . . . 71
13.4 Overflow and Rollover Behavior . . . 71
13.5 Output Compare Units and Pulse Width Modulation . . . 71
13.6 Input Capture Units . . . 72
13.7 Flags and Interrupts . . . 72
13.8 Register Description . . . 74
13.8.1 TIMCR Register . . . 74
13.8.2 TIMSR Register . . . 76
13.8.3 TIMVAL Register . . . 77
13.8.4 TIMCMP0 Register . . . 77
13.8.5 TIMCMP1 Register . . . 78
13.8.6 TIMCMP2 Register . . . 78
13.8.7 TIMCAP0 Register . . . 79
13.8.8 TIMCAP1 Register . . . 80

14 SYSTEM Peripheral **81**
14.1 System Clocks and Clock Management . . . 81
14.2 Power Saving Features . . . 82
14.3 CPU Sleep Mode . . . 82
14.4 CRC Module . . . 83
14.5 Interrupt Control . . . 83
14.6 Watchdog Timer . . . 84
14.7 Register Description . . . 85

14.7.1 SYSCLKCR Register . . . 85
14.7.2 CLKDIVCR Register . . . 86
14.7.3 BLOCKPWR Register . . . 86
14.7.4 CRCDATA Register . . . 87
14.7.5 CRCSTATE Register . . . 87
14.7.6 IRQENL Register . . . 88
14.7.7 IRQENM Register . . . 88
14.7.8 IRQENU Register . . . 89
14.7.9 IRQPRIL Register . . . 90
14.7.10 IRQPRIM Register . . . 90
14.7.11 IRQPRIU Register . . . 91
14.7.12 IRQCR Register . . . 91
14.7.13 WDTCR Register . . . 92
14.7.14 WDTSR Register . . . 93
14.7.15 WDTPASS Register . . . 93
14.7.16 WDTVAL Register . . . 94
14.7.17 DCO0BIAS Register . . . 95
14.7.18 DCO1BIAS Register . . . 95

15 NPU Peripheral **96**
15.1 Register Description . . . 97
15.1.1 NPUCR Register . . . 97
15.1.2 NPUIVSAR Register . . . 97
15.1.3 NPUWVSAR Register . . . 98
15.1.4 NPUOVSAR Register . . . 99

16 SARADC Peripheral **100**
16.1 Register Description . . . 102
16.1.1 SARADC_CR Register . . . 102
16.1.2 SARADC_CDIV Register . . . 102
16.1.3 SARADC_SR Register . . . 103
16.1.4 SARADC_DATA Register . . . 103
16.1.5 SARADC_TPR Register . . . 104

17 AFE Peripheral **105**
17.1 Register Description . . . 111
17.1.1 AFE_CR Register . . . 111
17.1.2 AFE_TPR Register . . . 112
17.1.3 AFE_SR Register . . . 113
17.1.4 AFE_ADC_VAL Register . . . 113
17.1.5 BIAS_CR Register . . . 114
17.1.6 BIAS_ADJ Register . . . 114
17.1.7 BIAS_DBP Register . . . 114
17.1.8 BIAS_DBPC Register . . . 115
17.1.9 BIAS_DBNC Register . . . 115
17.1.10 BIAS_DBN Register . . . 116
17.1.11 BIAS_TC_POT Register . . . 116
17.1.12 BIAS_LC_POT Register . . . 116

17.1.13 BIAS_TIA_G_POT Register . . . 117
17.1.14 BIAS_DSADC_VCM Register . . . 117
17.1.15 BIAS_REV_POT Register . . . 118
17.1.16 BIAS_TC_DSADC Register . . . 118
17.1.17 BIAS_LC_DSADC Register . . . 118
17.1.18 BIAS_RIN_DSADC Register . . . 119
17.1.19 BIAS_RFB_DSADC Register . . . 119

18 I2Cx Peripheral 120
18.1 Register Description . . . 121
18.1.1 I2CCR Register . . . 121
18.1.2 I2CFCR Register . . . 123
18.1.3 I2CSR Register . . . 124
18.1.4 I2CMTX Register . . . 127
18.1.5 I2CMRX Register . . . 127
18.1.6 I2CSTX Register . . . 127
18.1.7 I2CSRX Register . . . 127
18.1.8 I2CAR Register . . . 128
18.1.9 I2CAMR Register . . . 128

19 Register and Peripheral Memory Map 129
19.1 GPIO0 Peripheral . . . 129
19.2 GPIO1 Peripheral . . . 129
19.3 SPI0 Peripheral . . . 129
19.4 SPI1 Peripheral . . . 129
19.5 UART0 Peripheral . . . 130
19.6 UART1 Peripheral . . . 130
19.7 TIMER0 Peripheral . . . 130
19.8 TIMER1 Peripheral . . . 130
19.9 GPIO2 Peripheral . . . 131
19.10 SYSTEM Peripheral . . . 131
19.11 NPU Peripheral . . . 132
19.12 SARADC Peripheral . . . 132
19.13 AFE Peripheral . . . 132
19.14 GPIO3 Peripheral . . . 133
19.15 I2C0 Peripheral . . . 133
19.16 I2C1 Peripheral . . . 133

20 Errata 134

1 System Configuration

The microcontroller (MCU) described in this document utilizes the VestaRV CPU core, a custom 32-bit RISC-V processor. The microcontroller is composed of the VestaRV CPU, a ROM that contains boot software and a Forth interpreter, RAM memory cell(s), and several peripherals. The modular architecture enables customization of the peripheral set, memory configuration, and system features to match specific application requirements. Figure 1 shows the layout of the fabricated MCU chip.

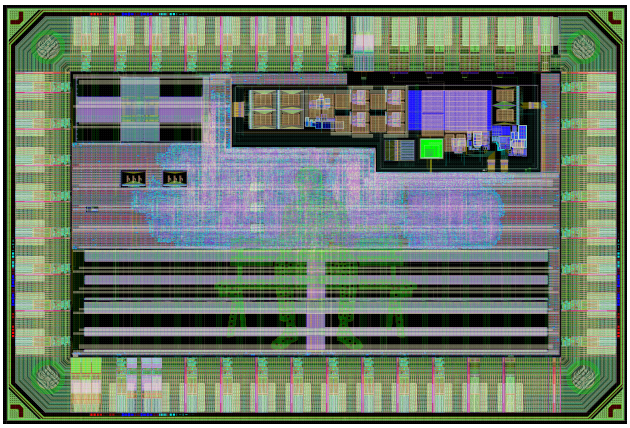


Figure 1: Myshkin MCU Layout Image (1.5 mm × 1.0 mm)

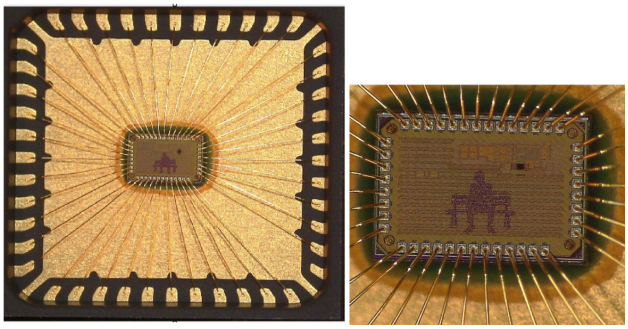


Figure 2: Myshkin MCU Bonding Image (1.5 mm × 1.0 mm)

The following list details the MCU configuration:

- Chip name: Myshkin
- Compatible with RISC-V compiler RV32I[M][C]
- 32-bit memory bus
- Contains 32 general purpose dual-port CPU registers
- Supports the RISC-V compressed instruction set and allows the use of the [C] compiler extension. Allowing the compiler to use the [C] extension reduces executable code size, but also takes the processor longer to fetch 32-bit long instructions that are aligned on a 2 but not 4 byte boundary from memory.

- Includes a multi-stage bit shifter in the ALU to save power and chip area for bit shift operations at the expense of speed
- Includes a fast 32-bit signed integer hardware multiplier in the ALU and allows the use of the [M] compiler extension
- Includes a medium speed 32-bit signed integer hardware divider and remainder calculator in the ALU
- Supports maskable hardware interrupts generated by peripheral signals
- 32× GPIO pins with edge-triggered interrupts and multiplexed peripheral functions
- 2× SPI interfaces (one with flash memory access and one without flash memory access)
- 2× UART interfaces
- 2× 32-bit timers with pulse-width modulation outputs and input capture units
- 2× I^2C interfaces (both master and slave mode)
- A neural processing unit (NPU) co-processor for hardware acceleration of machine learning tasks
- A CRC calculation engine
- 2× internal digitally controllable oscillator
- 2× external clock pins for clock generation and accurate timing
- A 10-bit, SAR ADC
- A 12-bit, dual-slope DAC

2 Packaging and Pins Configuration

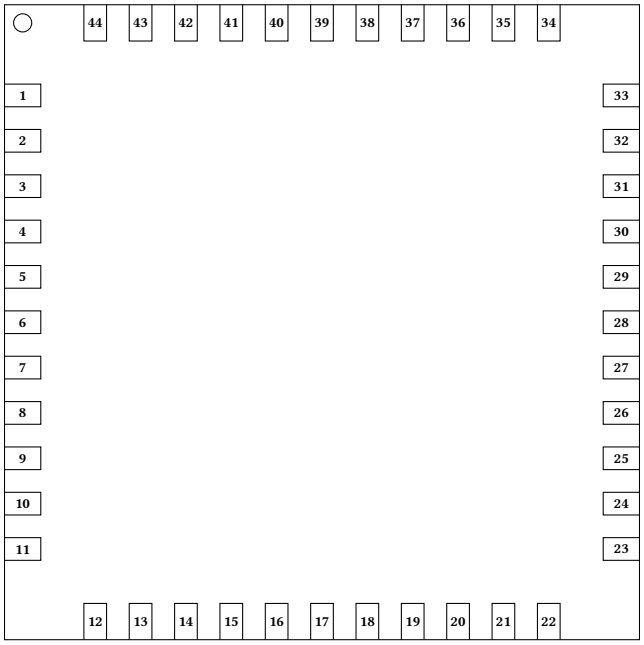


Figure 3: QFN-44 (7 × 7 mm, 0.5 mm pitch)

The chip package is a QFN-44 with dimensions 7 × 7 mm and pitch 0.5 mm. The package footprint (as seen from the top down) is shown in Figure 3. The pins on the package are listed in Table 1.

Table 1: Package Pins

#	Pin	I/O	Power Domain	Power Pins
1	P3.0	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
2	P2.7	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
3	P2.6	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
4	P2.5	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
5	P2.4	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
6	P2.3	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
7	P2.2	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
8	P2.1	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
9	P2.0	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
10	VDD	Power Input	Digital Core (1.0 V)	VDD/VSS
11	RESETN	Input	Digital I/O (3.3 V)	VDDPST/VSSPST
12	VDDPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
13	P1.7	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
14	P1.6	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
15	P1.5	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
16	P1.4	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
17	P1.3	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
18	P1.2	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST

Table 1 continued on next page

Table 1 continued from previous page

#	Pin	I/O	Power Domain	Power Pins
19	P1.1	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
20	P1.0	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
21	VSSPST	Power Input	Digital I/O (3.3 V)	VDDPST/VSSPST
22	VSS	Power Input	Digital Core (1.0 V)	VDD/VSS
23	*NC*	–	–	–
24	P0.7	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
25	P0.6	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
26	P0.5	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
27	P0.4	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
28	P0.3	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
29	P0.2	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
30	P0.1	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
31	P0.0	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
32	AVSS	Power Input	Analog (3.3 V)	AVDD/AVSS
33	RE	Input/Output	Analog (3.3 V)	AVDD/AVSS
34	CE	Input/Output	Analog (3.3 V)	AVDD/AVSS
35	ATP-IN	Input	Analog (3.3 V)	AVDD/AVSS
36	ATP-OUT	Output	Analog (3.3 V)	AVDD/AVSS
37	AVDD	Power Input	Analog (3.3 V)	AVDD/AVSS
38	P3.7	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
39	P3.6	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
40	P3.5	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
41	P3.4	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
42	P3.3	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
43	P3.2	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST
44	P3.1	Input/Output	Digital I/O (3.3 V)	VDDPST/VSSPST

Many of the digital I/O pins are GPIO pins. These pins can be in either primary/GPIO mode, or secondary/peripheral mode (see Section 10 on the GPIO peripherals for more details). A list of all the GPIO pins, along with their primary and secondary functions and reset values, is shown in Table 2.

Table 2: GPIO Pins

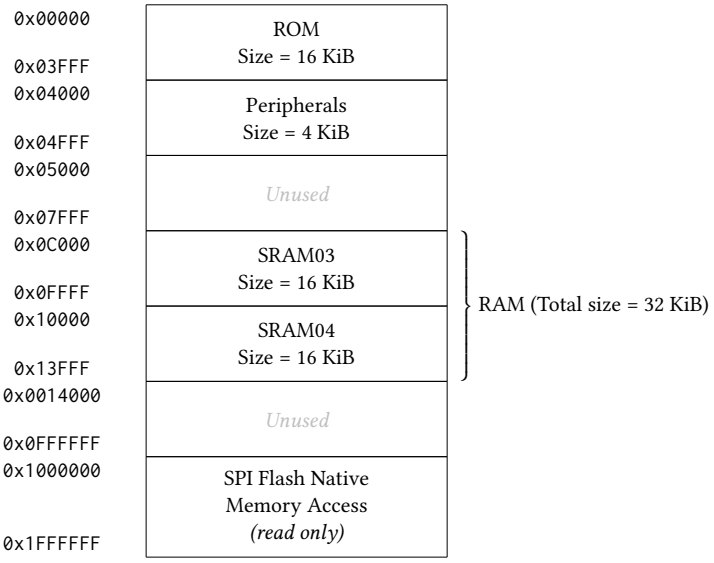
	Primary	Secondary	*Reset Values*			
Pin	Function	Function	SEL	OUT	DIR	REN
P0.0	`GPIO0`	`CS_FLASH`	0 *(Primary)*	1 *(High)*	1 *(Output)*	0 *(Disabled)*
P0.1	`GPIO1`	`MISO0`	1 *(Secondary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P0.2	`GPIO2`	`MOSI0`	1 *(Secondary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P0.3	`GPIO3`	`SCK0`	1 *(Secondary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P0.4	`GPIO4`	`LFXT`	1 *(Secondary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P0.5	`GPIO5`	`HFXT`	1 *(Secondary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P0.6	`GPIO6`	`TRAP`	1 *(Secondary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P0.7	`BOOT`	–	0 *(Primary)*	0 *(Low)*	0 *(Input)*	1 *(Enabled)*
P1.0	`GPIO8`	`CS1`	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P1.1	`GPIO9`	`MISO1`	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*

Table 2 continued on next page

Table 2 continued from previous page

Pin	Primary Function	Secondary Function	*Reset Values* SEL	OUT	DIR	REN
P1.2	GPIO10	MOSI1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P1.3	GPIO11	SCK1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P1.4	GPIO12	TX0	1 *(Secondary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P1.5	GPIO13	RX0	1 *(Secondary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P1.6	GPIO14	TX1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P1.7	GPIO15	RX1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.0	GPIO16	T0CMP0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.1	GPIO17	T0CMP1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.2	GPIO18	T0CAP0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.3	GPIO19	T0CAP1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.4	GPIO20	T1CMP0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.5	GPIO21	T1CMP1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.6	GPIO22	T1CAP0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P2.7	GPIO23	T1CAP1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.0	GPIO24	SDA0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.1	GPIO25	SCL0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.2	GPIO26	SDA1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.3	GPIO27	SCL1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.4	GPIO28	DTP0	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.5	GPIO29	DTP1	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.6	GPIO30	DTP2	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*
P3.7	GPIO31	DTP3	0 *(Primary)*	0 *(Low)*	0 *(Input)*	0 *(Disabled)*

3 Address Space



The 32-bit memory bus on the MCU is in the von Neumann architecture. All addressable memory elements (i.e. the ROM, RAM, and memory mapped registers) are all located on a single bus. A pointer can access any of these locations in memory.

The ROM cell contains boot code, which is read-only. Each SRAM cell can be read or written to, and may contain both program and variable information. The chip does not contain programmable non-volatile memory (e.g. flash memory), so the program must be entirely stored in RAM along with all runtime memory. Each SRAM and ROM cell can be individually powered off or on using the `SYSTEM` peripheral. When powered off, the cells use less power and cannot be read or written to. SRAM cells that are powered off do not retain their previous data, and the data becomes undefined.

The interrupt vector table (IVT) begins at address `0x8000` and contains jump instructions to the interrupt service routines (ISRs). When an interrupt occurs, the CPU directly vectors to the corresponding entry in the IVT, which then jumps to the appropriate ISR. Each IVT entry is 4 bytes and typically contains a RISC-V jump instruction (e.g., `j handler`). The IVT can be initialized and configured in software, allowing flexible interrupt routing and handler assignment at runtime.

4 Startup and Reset Behavior

To place the MCU in reset mode, drive the `resetn` pin LOW (asserted). To take the MCU out of reset mode, let the `resetn` pin go high (deasserted). The `resetn` pin has an internal 47 kΩ pullup resistor which constantly attempts to pull the `resetn` pin HIGH. If the user does not wish to use the `resetn` pin, it is safe to leave it unconnected.

When the MCU is first powered up, a power-on reset (POR) circuit resets the MCU. As the core voltage, beginning at 0 V, begins to climb, the POR circuit does not assert the internal reset. Once the core voltage reaches a particular value (which is temperature and process dependent, but typically around 0.7 V), the POR circuit asserts the internal reset signal for about 20 ns and then deasserts it. If either the `resetn` pin or the POR reset signal is asserted, the MCU will be in reset mode. In reset, the memory mapped registers and CPU state variables are all reset to their powerup values.

When the MCU comes out of reset, it immediately begins to execute the bootloader stored in the ROM, setting the program counter to address `0x0000`. If the `BOOT` pin is LOW, the bootloader launches a Forth interpreter which communicates over `UART0` at 115,200 baud (see Section 7 for details concerning the Forth interpreter). If the `BOOT` pin is HIGH, the bootloader reads and executes the program stored on an external SPI flash on `SPI0` (see Section 9 for details on how to program the MCU). When the bootloader reads the program stored in the SPI flash, it dumps the contents of the SPI flash directly into RAM, completely filling the RAM from beginning to end using an address-to-address copy. After the bootloader has copied the program stored in the SPI flash into RAM, it jumps (moves the program counter) to address `0x814C` and begins executing the code.

5 Interrupts

Table 3: Interrupt Vectors

Priority	Interrupt Source
0	CPU Internal Timer
1	EBREAK, ECALL, or Illegal Instruction
2	Bus Error (unaligned memory access)
0	SYSTEM
1	GPIO0
9	SPI0
11	SPI1
13	UART0
16	TIMER0
22	TIMER1
28	GPIO1
36	GPIO2
44	GPIO3
52	UART1
55	AFE
56	SARADC
57	I2C0
70	I2C1

The VestaRV CPU implements a priority-based interrupt system supporting 83 distinct interrupt sources as enumerated in the table above. Each interrupt can be individually enabled or disabled through the SYS_IRQ_EN register in the SYSTEM peripheral, where setting a bit to 1 enables the corresponding interrupt and clearing it to 0 disables the interrupt. All interrupts are disabled by default upon MCU reset and must be explicitly enabled in software before use. Additionally, peripherals generating interrupts must be configured to enable interrupt generation at the peripheral level.

5.1 Interrupt Vector Table

The interrupt vector table (IVT) is located at the base of RAM, beginning at address `0x8000`. Unlike traditional pointer-based interrupt vector tables, the VestaRV IVT contains direct jump instructions to the interrupt service routines (ISRs). Each entry in the IVT is a 4-byte RISC-V jump instruction (e.g., `j handler`) that vectors execution immediately to the corresponding ISR when an interrupt occurs. This architecture eliminates the indirection overhead of loading function pointers and provides deterministic, low-latency interrupt response.

While the VestaRV hardware does not mandate a specific IVT structure, it is strongly recommended to populate the IVT with jump instructions during system initialization. The automatic code generation tools produce linker scripts and initialization code that configure the IVT at the beginning of RAM. The memory map header files provide macros to facilitate proper ISR placement and IVT entry configuration.

5.2 Recursive Interrupt Handling

The VestaRV CPU supports recursive interrupt handling through hardware-assisted context preservation. When an interrupt occurs, the CPU automatically saves the complete processor context—including all

32 general-purpose registers and the program counter—onto the system stack. This automatic context switching enables nested interrupts, allowing higher-priority interrupts to preempt lower-priority interrupt handlers without software intervention for register preservation.

Upon entering an ISR, the stack pointer references the saved context, and the ISR may execute normally without concern for register conflicts with the interrupted code. When the ISR completes and executes the `retirq` instruction, the CPU automatically restores the saved context from the stack and resumes execution at the interrupted location. If a higher-priority interrupt occurs during ISR execution, the current ISR context is saved and the new ISR begins execution, creating a stack of nested interrupt contexts. This mechanism allows for efficient, deterministic handling of complex interrupt scenarios while maintaining system responsiveness.

The depth of interrupt nesting is limited only by available stack space. Programmers must ensure sufficient stack allocation to accommodate the maximum anticipated nesting level, accounting for both interrupt contexts (120 bytes per level) and local variables used within ISRs.

5.3 Sleep Mode and Interrupt Wake-Up

If an interrupt occurs while the CPU is in sleep mode, the processor will automatically wake from sleep and service the interrupt. Upon execution of the `retirq` instruction, the CPU will return to sleep mode unless the ISR or woken code explicitly reconfigures the clock and power settings. During sleep mode, the CPU clock is gated to conserve power, but all register contents and the program counter are retained. If the clock source for MCLK is disabled during sleep, any enabled interrupt will automatically power up and enable the source clock for the duration of interrupt servicing.

Clock and power management, including sleep mode configuration, are controlled through the `SYSTEM` peripheral. Note that if SMCLK is disabled, peripherals may be unable to assert interrupt signals, potentially preventing wake from sleep.

6 CPU Instructions and Assembly

The Myshkin MCU integrates the VestaRV CPU core, a custom 32-bit RISC-V processor implementing the RV32IMAC instruction set with bit manipulation extensions (Zba, Zbb, Zbc, Zbs). This configuration provides integer arithmetic (I), hardware multiplication and division (M), atomic operations (A), compressed 16-bit instructions (C), and enhanced bit manipulation capabilities. The RISC-V instruction set architecture (ISA) defines the binary encoding of machine instructions, their operational semantics, and the register architecture used for computation and control flow.

6.1 CPU Registers

There are 32 CPU registers defined in the RISC-V ISA. Though the user can technically use any register for any purpose (except for `x0`/`zero`, which is read-only and is always equal to 0x00000000), it is generally advisable to use each register for its intended purpose and follow the convention that the RISC-V ISA describes. The convention was created to create a standard use for each register so that assembly functions can be written that will be compatible with any RISC-V assembly program. The RISC-V compiler always follows this convention. Therefore, if the user also follows the convention, then the user's assembly code will be drop-in compatible with other compiled code (such as C code). If the user does not follow the convention, then it may cause many runtime errors.

Each register has a specific purpose and a designated "saver" that is allowed to write to it. The saver is either the caller (the code that calls a function) or the callee (the function that was called). This allows for communication between caller and callee such that arguments can be passed from caller to callee and returned data can be sent back from callee to caller. For example, the registers `a0-a7` are intended for function arguments and are written to by the caller. Registers `s0-s11` are preserved across function calls and are written to by the callee. Furthermore, registers `t0-t6` are temporary register that can be used for any purpose and are written to by the caller.

In assembly, registers can be referred to interchangibly by their register number or name (e.g. you can either type `x1` or `ra` to refer to the same register). Generally, one should only use the register names since they give hints as to the purpose of the register.

A CPU instruction can have up to two source register `rs1` and `rs2`, and up to one destination register `rd`. `rs1`, `rs2`, and `rd` can be any of the 32 CPU registers (e.g. `sp`, `t6`, `zero`, etc.). The source registers are used as "arguments" to the instruction and are not modified by the instruction, and the destination register is modified by the instruction as its output.

A list of all 32 CPU registers is provided in Table 4.

Table 4: CPU Registers

Register	Name	Description	Saver
`x0`	`zero`	Hard-wired zero (read-only)	N/A
`x1`	`ra`	Return Address	Caller
`x2`	`sp`	Stack Pointer	Callee
`x3`	`gp`	Global Pointer	–
`x4`	`tp`	Thread Pointer	–
`x5`	`t0`	Temporary/Alternate Link Register	Caller
`x6-x7`	`t1-t2`	Temporary Registers	Caller
`x8`	`s0/fp`	Saved Register/Frame Pointer	Callee
`x9`	`s1`	Saved Register	Callee
`x10-x11`	`a0-a1`	Function Arguments/Return Values	Caller

Table 4: CPU Registers

Register	Name	Description	Saver
x12-x17	a2-a7	Function Arguments	Caller
x18-x27	s2-s11	Saved Registers	Callee
x28-x31	t3-t6	Temporary Registers	Caller

6.2 Load/Store Memory Instructions

These instructions interface between the memory bus (containing RAM, ROM, and memory-mapped registers) and the CPU registers. They are used to load data from memory into a CPU register or store data from a CPU register into memory. The immediate parameter (`i`) refers to a hard-coded 12-bit constant that is stored inside the CPU instruction itself and is added to the value of one of the registers. All load/store instructions take 2 MCLK cycles to execute.

- Load Signed Byte: `lb rd, I(rs1)` : loads the signed byte (8 bits) stored at memory location pointed to by (`rs1` + `I`) into the destination CPU register `rd`
- Load Unsigned Byte: `lbu rd, I(rs1)` : loads the unsigned byte (8 bits) stored at memory location pointed to by (`rs1` + `I`) into the destination CPU register `rd`
- Load Signed Half Word: `lh rd, I(rs1)` : loads the signed half word (16 bits) stored at memory location pointed to by (`rs1` + `I`) into the destination CPU register `rd`
- Load Unsigned Half Word: `lhu rd, I(rs1)` : loads the unsigned half word (16 bits) stored at memory location pointed to by (`rs1` + `I`) into the destination CPU register `rd`
- Load Word: `lw rd, I(rs1)` : loads the signed or unsigned word (32 bits) stored at memory location pointed to by (`rs1` + `I`) into the destination CPU register `rd`
- Store byte: `sb rs1, I(rs2)` : stores the signed or unsigned byte (8 bits) from source CPU regsiter 1 `rs1` into the memory location pointed to by (`rs2` + `I`)
- Store half word: `sh rs1, I(rs2)` : stores the signed or unsigned half word (16 bits) from source CPU regsiter 1 `rs1` into the memory location pointed to by (`rs2` + `I`)
- Store word: `sh rs1, I(rs2)` : stores the signed or unsigned word (32 bits) from source CPU regsiter 1 `rs1` into the memory location pointed to by (`rs2` + `I`)

6.3 Shift Instructions

These instructions perform bit shift operations (e.g. <<). The immediate parameter (`I`) refers to a hard-coded 12-bit constant that is stored inside the CPU instruction itself and is used as the number of bits to shift. Each shift instruction takes 1 MCLK cycle to execute.

- Shift Left: `sll rd, rs1, rs2` : Logical shifts the value in `rs1` left by `rs2` bits and stores the result in `rd`.
- Shift Left Immediate: `slli rd, rs1, I` : Logical shifts the value in `rs1` left by `I` bits and stores the result in `rd`.

- Shift Right: `sll rd, rs1, rs2`: Logical shifts the value in `rs1` right by `rs2` bits and stores the result in `rd`.
- Shift Right Immediate: `slli rd, rs1, I`: Logical shifts the value in `rs1` right by `I` bits and stores the result in `rd`.
- Shift Right Arithmetic: `sll rd, rs1, rs2`: Arithmetic shifts the value in `rs1` right by `rs2` bits and stores the result in `rd`.
- Shift Right Arithmetic Immediate: `slli rd, rs1, I`: Arithmetic shifts the value in `rs1` right by `I` bits and stores the result in `rd`.

6.4 Arithmetic Instructions

These instructions perform arithmetic operations. The immediate parameter (`I`) refers to a hard-coded 12-bit signed integer that is stored inside the CPU instruction itself and is added to the operation (except for in the `auipc` instruction). These instructions take 1 MCLK cycle to execute.

- Addition: `add rd, rs1, rs2`: Adds the signed/unsigned integer values in `rs1` and `rs2` together and stores the sum in `rd`
- Addition Immediate: `addi rd, rs1, I`: Adds the signed/unsigned integer value in `rs1` with the immediate 12-bit signed integer `I` together and stores the sum in `rd`
- Subtraction: `sub rd, rs1, rs2`: Subtracts the signed/unsigned integer values in `rs1` from `rs2` and stores the result in `rd` (i.e. `rd` $\leftarrow$ `rs1` $-$ `rs2`)
- Load Upper Immediate: `lui rd, I`: Loads the 20-bit immediate value `I` into the upper 20 bits of the destination CPU register `rd` (the lower 12 bits are all cleared to 0s)
- Add Upper Immediate to PC: `auipc rd, I`: Adds the 12-bit signed integer immediate `I` to the program counter to jump the program's execution to a location `I` away from the current program counter. Note that `I` can be negative.

6.5 Bitwise Instructions

These instructions perform bitwise operations. The immediate parameter (`I`) refers to a hard-coded 12-bit signed integer that is stored inside the CPU instruction itself and is used as one of the operands of the bitwise operations. These instructions take 1 MCLK cycle to execute.

- Bitwise exclusive OR: `xor rd, rs1, rs2`: Performs the bitwise XOR of `rs1` and `rs2` and stores the result in `rd`.
- Bitwise exclusive OR Immediate: `xori rd, rs1, I`: Performs the bitwise XOR of `rs1` and the immediate `I` and stores the result in `rd`.
- Bitwise OR: `or rd, rs1, rs2`: Performs the bitwise OR of `rs1` and `rs2` and stores the result in `rd`.
- Bitwise OR Immediate: `ori rd, rs1, I`: Performs the bitwise OR of `rs1` and the immediate `I` and stores the result in `rd`.

- Bitwise AND: `and rd, rs1, rs2` : Performs the bitwise AND of `rs1` and `rs2` and stores the result in `rd`.
- Bitwise AND Immediate: `andi rd, rs1, I` : Performs the bitwise AND of `rs1` and the immediate `I` and stores the result in `rd`.

6.6 Comparison Instructions

These instructions perform comparison operations. The results of these instructions are either true (0x00000001) or false (0x00000000). The immediate parameter (`I`) refers to a hard-coded 12-bit signed or unsigned integer that is stored inside the CPU instruction itself and is used as one of the operands of the bitwise operations. These instructions take 1 MCLK cycle to execute.

- Set when Less Than: `slt rd, rs1, rs2` : If `rs1 < rs2` (using signed integers), the destination register `rd` is set to 0x00000001, otherwise it is set to 0x00000000.
- Set when Less Than Immediate: `slti rd, rs1, I` : If `rs1 < I` (using signed integers), the destination register `rd` is set to 0x00000001, otherwise it is set to 0x00000000.
- Set when Less Than Unsigned: `sltu rd, rs1, rs2` : If `rs1 < rs2` (using unsigned integers), the destination register `rd` is set to 0x00000001, otherwise it is set to 0x00000000.
- Set when Less Than Immediate Unsigned: `sltiu rd, rs1, I` : If `rs1 < I` (using unsigned integers), the destination register `rd` is set to 0x00000001, otherwise it is set to 0x00000000.

6.7 Branch Instructions

These instructions are conditional instructions that branch (jump) the program execution to a new location if the condition is met, or proceed on to the next instruction like normal if the condition is not met. The immediate parameter (`I`) refers to a hard-coded 12-bit signed integer that is stored inside the CPU instruction itself and is added to the program counter if the condition is met. Usually, in assembly, the program writer can simply provide the *label* to the desired branch location/function in the program (e.g. `labelToFunction:`) rather than its actual memory location, which is usually not known before being built. These instructions take 1 MCLK cycle to execute regardless of whether the branch is taken or not.

- Branch if Equal: `beq rs1, rs2, I` : Branches to PC + `I` if `rs1` = `rs2`, proceeds to the next instruction like normal otherwise.
- Branch if Not Equal: `bne rs1, rs2, I` : Branches to PC + `I` if `rs1` ≠ `rs2`, proceeds to the next instruction like normal otherwise.
- Branch if Less Than: `blt rs1, rs2, I` : Branches to PC + `I` if `rs1` < `rs2` (signed integers), proceeds to the next instruction like normal otherwise.
- Branch if Greater Than or Equal: `bge rs1, rs2, I` : Branches to PC + `I` if `rs1` ≥ `rs2` (signed integers), proceeds to the next instruction like normal otherwise.
- Branch if Less Than Unsigned: `bltu rs1, rs2, I` : Branches to PC + `I` if `rs1` < `rs2` (unsigned integers), proceeds to the next instruction like normal otherwise.
- Branch if Greater Than or Equal Unsigned: `bgeu rs1, rs2, I` : Branches to PC + `I` if `rs1` ≥ `rs2` (unsigned integers), proceeds to the next instruction like normal otherwise.

6.8 Jump and Link Instructions

The jump and link instructions write the address of the next consecutive instruction in the program to the destination register, but then jump the program counter to another location in memory. These instructions are used to call new functions while preserving the "return address", or the memory location of the instruction to go back to after the function returns. The immediate parameter (`I`) refers to a hard-coded 20-bit signed integer that is stored inside the CPU instruction itself and is used to modify the program counter. Usually, in assembly, the program writer can simply provide the *label* to the desired location/function in the program (e.g. `labelToFunction:`) rather than its actual memory location, which is usually not known before being built. All jump and link instructions take 1 MCLK cycle to execute.

- Jump and Link: `jal rd, I` Stores the return address to `rd` (which is usually the return address register `ra`) and then adds `I` to the program counter, effectively jumping to PC + `I`.
- Jump and Link Register: `jalr rd, rs1, I` Stores the return address to `rd` (which is usually the return address register `ra`) and then sets the program counter to `rs1` + `I`, effectively jumping to `rd1` + `I`.

6.9 Atomic Memory Operations (A Extension)

The VestaRV core implements the RISC-V Atomic (A) extension, providing atomic read-modify-write operations for thread synchronization. Since the Myshkin MCU is a single-core system, these instructions serve primarily as proof-of-concept implementations and do not provide practical multiprocessor synchronization benefits. However, they remain useful for ensuring atomic operations in interrupt-driven code or for software portability. All atomic memory operations operate on 32-bit words and take 5 MCLK cycles to execute.

- Load Reserved: `lr.w rd, (rs1)` : Loads a 32-bit word from memory address in `rs1` into `rd` and reserves the memory location for atomic access.
- Store Conditional: `sc.w rd, rs2, (rs1)` : Conditionally stores the 32-bit word in `rs2` to the memory address in `rs1`. Returns 0 in `rd` on success, nonzero on failure.
- Atomic Swap: `amoswap.w rd, rs2, (rs1)` : Atomically swaps the 32-bit word at memory address in `rs1` with the value in `rs2`, storing the original memory value in `rd`.
- Atomic Add: `amoadd.w rd, rs2, (rs1)` : Atomically adds `rs2` to the 32-bit word at memory address in `rs1`, storing the original memory value in `rd`.
- Atomic XOR: `amoxor.w rd, rs2, (rs1)` : Atomically performs bitwise XOR of `rs2` with the 32-bit word at memory address in `rs1`, storing the original memory value in `rd`.
- Atomic AND: `amoand.w rd, rs2, (rs1)` : Atomically performs bitwise AND of `rs2` with the 32-bit word at memory address in `rs1`, storing the original memory value in `rd`.
- Atomic OR: `amoor.w rd, rs2, (rs1)` : Atomically performs bitwise OR of `rs2` with the 32-bit word at memory address in `rs1`, storing the original memory value in `rd`.
- Atomic Minimum: `amomin.w rd, rs2, (rs1)` : Atomically compares `rs2` with the 32-bit signed word at memory address in `rs1`, storing the minimum value to memory and the original memory value in `rd`.

- Atomic Maximum: `amomax.w rd, rs2, (rs1)` : Atomically compares `rs2` with the 32-bit signed word at memory address in `rs1`, storing the maximum value to memory and the original memory value in `rd`.
- Atomic Minimum Unsigned: `amominu.w rd, rs2, (rs1)` : Atomically compares `rs2` with the 32-bit unsigned word at memory address in `rs1`, storing the minimum value to memory and the original memory value in `rd`.
- Atomic Maximum Unsigned: `amomaxu.w rd, rs2, (rs1)` : Atomically compares `rs2` with the 32-bit unsigned word at memory address in `rs1`, storing the maximum value to memory and the original memory value in `rd`.

6.10 Bit Manipulation Instructions (Zb* Extensions)

The VestaRV core implements several bit manipulation extensions (Zba, Zbb, Zbc, Zbs) that provide efficient operations for bit-level data manipulation, cryptographic primitives, and address generation. All bit manipulation instructions execute in 1 MCLK cycle.

6.10.1 Address Generation (Zba)

- Shift Left Add: `sh1add rd, rs1, rs2` : Shifts `rs1` left by 1 bit, adds `rs2`, and stores result in `rd`. Equivalent to `rd = (rs1 « 1) + rs2`.
- Shift Left Add 2: `sh2add rd, rs1, rs2` : Shifts `rs1` left by 2 bits, adds `rs2`, and stores result in `rd`. Equivalent to `rd = (rs1 « 2) + rs2`.
- Shift Left Add 3: `sh3add rd, rs1, rs2` : Shifts `rs1` left by 3 bits, adds `rs2`, and stores result in `rd`. Equivalent to `rd = (rs1 « 3) + rs2`.

6.10.2 Basic Bit Manipulation (Zbb)

- AND with Inverted: `andn rd, rs1, rs2` : Performs bitwise AND of `rs1` with the bitwise complement of `rs2`, storing result in `rd`.
- OR with Inverted: `orn rd, rs1, rs2` : Performs bitwise OR of `rs1` with the bitwise complement of `rs2`, storing result in `rd`.
- XOR with Inverted: `xnor rd, rs1, rs2` : Performs bitwise XOR of `rs1` with the bitwise complement of `rs2`, storing result in `rd`.
- Count Leading Zeros: `clz rd, rs1` : Counts the number of consecutive zero bits starting from the most significant bit in `rs1`, storing the count in `rd`.
- Count Trailing Zeros: `ctz rd, rs1` : Counts the number of consecutive zero bits starting from the least significant bit in `rs1`, storing the count in `rd`.
- Count Population: `cpop rd, rs1` : Counts the number of set bits (1s) in `rs1`, storing the count in `rd`.
- Maximum: `max rd, rs1, rs2` : Stores the maximum of `rs1` and `rs2` (signed comparison) in `rd`.
- Maximum Unsigned: `maxu rd, rs1, rs2` : Stores the maximum of `rs1` and `rs2` (unsigned comparison) in `rd`.

- Minimum: `min rd, rs1, rs2` : Stores the minimum of `rs1` and `rs2` (signed comparison) in `rd`.
- Minimum Unsigned: `minu rd, rs1, rs2` : Stores the minimum of `rs1` and `rs2` (unsigned comparison) in `rd`.
- Sign Extend Byte: `sext.b rd, rs1` : Sign-extends the least significant byte in `rs1` to 32 bits, storing result in `rd`.
- Sign Extend Halfword: `sext.h rd, rs1` : Sign-extends the least significant halfword (16 bits) in `rs1` to 32 bits, storing result in `rd`.
- Zero Extend Halfword: `zext.h rd, rs1` : Zero-extends the least significant halfword in `rs1` to 32 bits, storing result in `rd`.
- Rotate Left: `rol rd, rs1, rs2` : Rotates `rs1` left by the number of bits specified in the lower 5 bits of `rs2`, storing result in `rd`.
- Rotate Right: `ror rd, rs1, rs2` : Rotates `rs1` right by the number of bits specified in the lower 5 bits of `rs2`, storing result in `rd`.
- Rotate Right Immediate: `rori rd, rs1, shamt` : Rotates `rs1` right by `shamt` bits, storing result in `rd`.
- OR Combine: `orc.b rd, rs1` : Sets each byte in `rd` to 0xFF if the corresponding byte in `rs1` is nonzero, otherwise sets to 0x00.
- Byte Reverse: `rev8 rd, rs1` : Reverses the byte order in `rs1`, storing result in `rd` (converts between big-endian and little-endian).

6.10.3 Carryless Multiplication (Zbc)

- Carryless Multiply Low: `clmul rd, rs1, rs2` : Performs carryless multiplication of `rs1` and `rs2`, storing the lower 32 bits of the result in `rd`. Useful for CRC calculations and cryptographic operations.
- Carryless Multiply High: `clmulh rd, rs1, rs2` : Performs carryless multiplication of `rs1` and `rs2`, storing the upper 32 bits of the result in `rd`.
- Carryless Multiply Reversed: `clmulr rd, rs1, rs2` : Performs carryless multiplication with a bit-reversed result, storing bits [62:31] of the 64-bit carryless product in `rd`.

6.10.4 Single-Bit Manipulation (Zbs)

- Bit Clear: `bclr rd, rs1, rs2` : Clears the bit in `rs1` at the position specified by the lower 5 bits of `rs2`, storing result in `rd`.
- Bit Clear Immediate: `bclri rd, rs1, shamt` : Clears the bit in `rs1` at position `shamt`, storing result in `rd`.
- Bit Extract: `bext rd, rs1, rs2` : Extracts the bit in `rs1` at the position specified by the lower 5 bits of `rs2`, storing the single bit (zero-extended) in `rd`.
- Bit Extract Immediate: `bexti rd, rs1, shamt` : Extracts the bit in `rs1` at position `shamt`, storing the single bit (zero-extended) in `rd`.

- Bit Invert: `binv rd, rs1, rs2`: Inverts the bit in `rs1` at the position specified by the lower 5 bits of `rs2`, storing result in `rd`.
- Bit Invert Immediate: `binvi rd, rs1, shamt`: Inverts the bit in `rs1` at position `shamt`, storing result in `rd`.
- Bit Set: `bset rd, rs1, rs2`: Sets the bit in `rs1` at the position specified by the lower 5 bits of `rs2`, storing result in `rd`.
- Bit Set Immediate: `bseti rd, rs1, shamt`: Sets the bit in `rs1` at position `shamt`, storing result in `rd`.

6.11 Pseudo-Instructions

The RISC-V assembler also includes several pseudo-instructions for convenience. These pseudo-instructions are actually special cases of regular instructions, and can all be expressed as regular instructions.

- No operation: `nop` → `addi zero, zero, 0`
- Move: `mv rd, rs1` → `addi rd, rs1, 0`
- Bitwise Complement/Inversion: `not rd, rs1` → `xor rd, rs1, -1`
- Set if Equal to Zero: `seqz rd, rs1` → `sltiu rd, rs1, 1`
- Set if Not Equal to Zero: `snez rd, rs1` → `sltu rd, zero, rs1`
- Set if Less Than Zero: `sltz rd, rs1` → `slt rd, rs1, zero`
- Set if Greater Than Zero: `sgtz rd, rs1` → `slt rd, zero, rs1`
- Jump (without linking): `j I` → `jal zero, I`
- Return: `ret` → `jalr zero, ra, 0`
- Call function: `call I` → `auipc t1 I; jalr ra t1 I`
- Load immediate: `li rd, I` *(this could compile to several instructions)*
- Branch if Greater Than: `bgt rs1, rs2, I` → `blt rs2, rs1, I`
- Branch if Less Than or Equal To: `ble rs1, rs2, I` → `bge rs2, rs1, I`
- Branch if Greater Than Unsigned: `bgtu rs1, rs2, I` → `bltu rs2, rs1, I`
- Branch if Less Than or Equal To Unsigned: `bleu rs1, rs2, I` → `bgeu rs2, rs1, I`
- Branch if Equal to Zero: `beqz rs1, I` → `beq rs1, zero, I`
- Branch if Not Equal to Zero: `bnez rs1, I` → `bne rs1, zero, I`
- Branch if Less Than or Equal To Zero: `blez rs1, I` → `bge zero, rs1, I`

- Branch if Greater Than or Equal To Zero: `bgez, rs1, I` → `bge rs1, zero, I`
- Branch if Less Than Zero: `bltz rs1, I` → `blt rs1, zero, I`
- Branch if Greater Than Zero: `bgtz rs1, I` → `blt zero, rs1, I`

7 Forth Interpreter

When the MCU comes out of reset and the BOOT pin is LOW, the MCU boots to an interactive Forth interpreter based on the Forth language. Forth is a stack-based language where variables are pushed to a stack in LIFO (last in, first out) order. When a variable is set, it is pushed to the top of the stack (TOS). When a variable is used, it is popped (or consumed) from the TOS. Functions that take arguments pop them from the TOS. Functions that have return values push them to the TOS (after first popping its arguments). Variables are not assigned names, and the only way of distinguishing them from one another is by their order on the stack. All variables are treated as 32-bit signed integers. Variables may be pushed to the stack through the command line by simply typing the variable value into the interpreter as a plain text integer value or as a plain text hexadecimal value. Hexadecimal values must be preceded by the string `0x` without spaces in between (e.g. `0xA4`), but the value may be upper or lower case and does not need leading zeros. Variables are not pushed or popped and functions are not executed until a newline (or line feed) `\n` character is received, at which point the entire received line will be executed in order. New functions may be declared, and may use the stack and any already defined functions to perform their procedures. If the Forth interpreter does not recognize an input as a variable or a function, it prints the `?` char over the UART.

The Forth interpreter contained in the ROM of the MCU communicates over UART0 at 2,048 baud. The baud may be changed by reconfiguring SMCLK and the UART0 peripheral. When the Forth interpreter first starts up, it prints the string `rv4th!\n>`, and is then ready to receive characters over the UART. The Forth interpreter does not perform error handling on stack underflows. In the case of a stack underflow, it simply uses the value 0 as the TOS. By default, the echo is enabled, which repeates all characters received by the Forth interpreter back to the host. This is useful for typing into a terminal emulator manually, but may be switched off with the command `0 echo`

Several useful built-in Forth functions are listed below. The syntax is read as follows: The Forth function keyword is surrounded by quotations (though you do not actually type the quotes into the Forth interpreter), and is followed by the list of arguments and return values in parentheses. Left of the double dash is the list of arguments in the order that they would be typed into the Forth interpreter (reverse stack order), and right of the double dash is the list of return values in reverse stack order. Reverse stack order means that the rightmost item is at the TOS, and items to the left of it are below it on the stack. Arguments are always popped from the stack before the function executes, and return values are always pushed to the stack after the function finishes.

7.1 Memory Access Functions

- `@ ( addr -- )`

  Read directly from the memory address and push the value to the TOS

  Arguments: (in order from TOS down, reverse text order):

  1. addr: (TOS) The address to read from (must be aligned to a 4-byte (32-bit) boundary)

  Return values: none

- `! ( x addr -- )`

  Write directly to memory address

  Arguments: (in order from TOS down, reverse text order):

  1. addr: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)

2. `x`: The value to write to the memory address

Return values: none

- `+!` `( x addr -- )`

Add to value, direct memory access (*addr += x)

Arguments: (in order from TOS down, reverse text order):

1. `addr`: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
2. `x`: The value to add to the value at the memory address

Return values: none

- `sbi` `( bitnum addr -- )`

Set bit, direct memory access (*addr |= (1 « bitnum))

Arguments: (in order from TOS down, reverse text order):

1. `addr`: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
2. `bitnum`: The bit number

Return values: none

- `cbi` `( bitnum addr -- )`

Clear bit, direct memory access (*addr &= ~(1 « bitnum))

Arguments: (in order from TOS down, reverse text order):

1. `addr`: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
2. `bitnum`: The bit number

Return values: none

- `mask` `( bitmask x addr -- )`

Change only masked bits, direct memory access (*addr = (*addr & ~bitmask) | (x & bitmask))

Arguments: (in order from TOS down, reverse text order):

1. `addr`: (TOS) The address to write to (must be aligned to a 4-byte (32-bit) boundary)
2. `x`: The value to write (only masked bits will be written)
3. `bitmask`: The bit mask. Only bits with 1s can be changed

Return values: none

7.2 Print and Input Functions

- `.` `( x -- )`

Prints the TOS as an integer

Arguments: (in order from TOS down, reverse text order):

1. `x`: (TOS) The value to print

Return values: none

- `h.` `( x -- )`

  Prints the TOS as a hex string

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) The value to print

  Return values: none

- `echo` `( bool -- )`

  Change the echo state, which controls whether the Forth interpreter repeats all of its received characters

  Arguments: (in order from TOS down, reverse text order):

  1. `bool`: (TOS) If 0, does not echo. Otherwise, it does

  Return values: none

- `emit` `( x -- )`

  Prints the char at the TOS (integers are mapped to their ASCII counterparts)

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) The ASCII value of the char to print

  Return values: none

- `key` `( -- char )`

  Waits for a char over UART and pushes the ASCII value of the char to the TOS

  Arguments: none

  Return values: (in order from TOS down, reverse text order):

  1. `char`: (TOS) The char that was received over UART

- `list` `( -- )`

  Prints all currently available functions

  Arguments: none

  Return values: none

7.3 Arithmetic Functions

- `+` `( y x -- ret )`

  Adds two numbers (y + x)

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

Return values: (in order from TOS down, reverse text order):

1. `ret`: (TOS) Return value

- `-` `( y x -- ret )`
  Subtracts two numbers (y - x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `*` `( y x -- retprodhi retprodlo )`
  Multiplies two numbers (y * x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `retprodlo`: (TOS) Lower 32 bits of the return product
  2. `retprodhi`: Upper 32 bits of the return product
- `/%` `( y x -- retdiv retrem )`
  Divides two numbers with remainder (y / x and y % x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `retrem`: (TOS) The remainder (or modulo) result
  2. `retdiv`: The integer division result
- `neg` `( x -- ret )`
  Returns the negative (twos complement) of x
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value

- `abs ( x -- ret )`
  Absolute value (|x|)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value

7.4 Bitwise Logic Functions

- `~ ( x -- ret )`
  Bitwise complement/inversion (~x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `| ( y x -- ret )`
  Bitwise OR (y | x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `& ( y x -- ret )`
  Bitwise AND (y & x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `^ ( y x -- ret )`
  Bitwise XOR (y ^ x)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

Return values: (in order from TOS down, reverse text order):

1. `ret`: (TOS) Return value

- `*2` `( x -- ret )`
  Shift left 1 bit (x « 1)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `/2` `( x -- ret )`
  Shift right 1 bit (x » 1)
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `sll` `( x bits -- ret )`
  Shift left logical (x « bits)
  Arguments: (in order from TOS down, reverse text order):
  1. `bits`: (TOS) Signed integer
  2. `x`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `srl` `( x bits -- ret )`
  Shift right logical (x » bits), no sign extension
  Arguments: (in order from TOS down, reverse text order):
  1. `bits`: (TOS) Signed integer
  2. `x`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value

7.5 Comparison Functions

- `>` `( y x -- ret )`

  Greater than. Returns 1 if y > x, returns 0 otherwise.

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) 1 if true, 0 if false

- `<` `( y x -- ret )`

  Less than. Returns 1 if y < x, returns 0 otherwise.

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) 1 if true, 0 if false

- `==` `( y x -- ret )`

  Equals. Returns 1 if y == x, returns 0 otherwise.

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) 1 if true, 0 if false

7.6 Boolean Functions

- `not` `( x -- ret )`

  Logical not (!x). Returns 1 if x is equal to 0, returns 0 otherwise

  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) Return value

- `and` `( y x -- ret )`

  Logical and (y && x). Returns 1 if y and x are both true (nonzero), returns 0 otherwise

  Arguments: (in order from TOS down, reverse text order):

1. `x`: (TOS) Signed integer
2. `y`: Signed integer

Return values: (in order from TOS down, reverse text order):

1. `ret`: (TOS) Return value

- `or ( y x -- ret )`
  Logical or (y || x). Returns 1 if y or x are true (nonzero), returns 0 otherwise
  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):

  1. `ret`: (TOS) Return value

7.7 Stack Manipulation Functions

- `dup ( x -- x x )`
  Duplicates the TOS (pops the TOS and then pushes the value to the TOS twice)
  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Value to duplicate

  Return values: (in order from TOS down, reverse text order):

  1. `x`: (TOS) The duplicated value
  2. `x`: The original TOS

- `drop ( x -- )`
  Pops the TOS and sets it as the internal Forth `i` value. Usually used to delete the TOS.
  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) The value that is dropped

  Return values: none

- `swap ( y x -- x y )`
  Swaps the TOS with the value below the TOS
  Arguments: (in order from TOS down, reverse text order):

  1. `x`: (TOS) Old TOS
  2. `y`: Old value below the TOS

  Return values: (in order from TOS down, reverse text order):

  1. `y`: (TOS) Old value below the TOS
  2. `x`: Old TOS

- `over` `( y x -- y x y )`
  Copies the value below the TOS and pushes it to the TOS
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed Integer
  2. `y`: Signed Integer

  Return values: (in order from TOS down, reverse text order):
  1. `y`: (TOS) Signed Integer
  2. `x`: Signed Integer
  3. `y`: Signed Integer
- `tuck` `( y x -- x y x )`
  Copies the TOS and inserts it two after the TOS
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed Integer
  2. `y`: Signed Integer

  Return values: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed Integer
  2. `y`: Signed Integer
  3. `x`: Signed Integer
- `roll` `( n -- )`
  Removes value at index n of the stack and pushes it to the TOS (zero indexed)
  Arguments: (in order from TOS down, reverse text order):
  1. `n`: (TOS) Stack index (zero indexed)

  Return values: none
- `pick` `( n -- )`
  Copies value at index n of the stack and pushes it to the TOS (zero indexed)
  Arguments: (in order from TOS down, reverse text order):
  1. `n`: (TOS) Stack index (zero indexed)

  Return values: none
- `depth` `( -- ret )`
  Gets size of stack and pushes it to the top of the stack (making the new stack size the value of TOS + 1)
  Arguments: none
  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value

7.8 SPI Flash R/W and Programming Functions

- `fr ( bin length addr -- )`

  Reads data from the SPI flash.

  Arguments: (in order from TOS down, reverse text order):

  1. `addr`: (TOS) The address in the SPI flash to begin reading from
  2. `length`: The number of bytes to read
  3. `bin`: If true (nonzero), transmits binary data, otherwise transmits ASCII hex string

  Return values: none

- `fe ( conf addr -- eraseSuccessful )`

  Erases a 256-byte page from the SPI flash memory. The start address must be 256-byte aligned.

  Arguments: (in order from TOS down, reverse text order):

  1. `addr`: (TOS) The start address of the page to erase in the SPI flash. Must be 256-byte aligned.
  2. `conf`: If equal to 123, erases the page and returns 1, otherwise does not erase and returns 0

  Return values: (in order from TOS down, reverse text order):

  1. `eraseSuccessful`: (TOS) Nonzero if successful, zero if failed

- `fw ( bin addr -- )`

  Writes data to the SPI flash. Must write exactly 256 bytes at a time. The start address must be 256-byte aligned.

  Arguments: (in order from TOS down, reverse text order):

  1. `addr`: (TOS) The address in the SPI flash to begin writing to. Must be 256-byte aligned.
  2. `bin`: If true (nonzero), expects to receive binary data, otherwise expects to receive an ASCII hex string

  Return values: none

7.9 Miscellaneous Functions

- `rst ( -- )`

  Performs a soft reset (turns on all memory cells and jumps to address 0 in the ROM, does not reset clocks or hardware)

  Arguments: none

  Return values: none

- `clk ( meastime clksel -- freq )`

  Measures selected clock frequency

  Arguments: (in order from TOS down, reverse text order):

  1. `clksel`: (TOS) Clock select. 0 = SMCLK; 1 = MCLK; 2 = LFXT; 3 = HFXT

2. `meastime`: Measurement time. 0 = 1 s; 1 = 0.5 s; 2 = 0.25 s; 3 = 0.125 s

Return values: (in order from TOS down, reverse text order):

1. `freq`: (TOS) Selected clock frequency in Hz

- `min` `( y x -- ret )`
  Returns minimum value of y and x
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `max` `( y x -- ret )`
  Returns maximum value of y and x
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer
  2. `y`: Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `swpb` `( x -- ret )`
  Swap bits 15 downto 8 with bits 7 downto 0 of TOS, also causes bits 31 downto 16 to all be 0
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value
- `swphw` `( x -- ret )`
  Swap upper 16 bits with lower 16 bits of TOS
  Arguments: (in order from TOS down, reverse text order):
  1. `x`: (TOS) Signed integer

  Return values: (in order from TOS down, reverse text order):
  1. `ret`: (TOS) Return value

7.10 Creating New Forth Functions

To create a new Forth function on-the-fly, simply use the syntax `: funcname definition ;` where `funcname` is the name of the new function and `definition` is the body of the function. The new function may be called at any time after the semicolon (even before a new line, though it will not actually be executed until a new line) by typing the function name, just like all other Forth functions. New Forth functions can only add or remove variables from the stack and call other defined Forth functions. The arguments to custom Forth functions are the initial values on the stack that the custom function pops off the TOS. Likewise, the return values are any new values the custom function pushes to the TOS.

Listed below is an example program that increments the TOS by one when it is called:

```
: inc 1 + ;
```

7.11 Conditionals

The `if then` functions implement a conditional statement which may only be used inside a user defined function. The conditional is used with the syntax:

```
if procedureIfTrue then
```

inside a function (do not forget the space and semicolon which must come after forthinlinethen). The `if` function pops the TOS and then executes whatever is defined within `procedureIfTrue` if and only if the TOS is true (nonzero). The `then` function makrks the end of the conditional procedure. For example, a function to print the text `cool` if the TOS is equal to 5 might look like this:

```
: cool? 5 == if 108 111 111 99 emit emit emit emit then ;
```

The `else` keyword may be inserted after `procedureIfTrue` if you want to execute something if the conditional is false. The syntax then becomes:

```
if procedureIfTrue else procedureIfFalse then
```

7.12 Loops

Forth has the equivalent of C-like `while` loops (or, more accurately, `do while` loops) with its `begin until` loops, which may only be executed inside a user defined function. The syntax is:

```
begin loopProcedure until
```

which first executes the loop procedure. When it gets to the `until` statement, it then checks if the TOS is true (nonzero) or false (zero). If the TOS is false, it loops back to the beginning of the loop. If true, it exits the loop and continues on.

Forth also has the equivalent of C-like `for` loops with its `do loop` functions. Again, this may only be used inside a user defined function. It uses the syntax:

```
stop start do loopProcedure loop
```

where `stop` is the index the loop stops on (it does NOT execute the loop procedure when the index is equal to `stop`), `start` is the first loop index, and `loopProcedure` is any functions or variables that you want to execute inside the loop. There is a special function `i` that may only be called from within a loop which pushes the loop index to the TOS. The functions `j` and `k` behave similarly, except they return the indices of the second and third nested loops respectively when using nested loops. The `i` function always returns the index of the deepest (or most frequent) loop.

Below is an example Forth program that prints the numbers from 0 to 9 using a `do loop` procedure:

```
: looptest 10 0 do i . loop ;
```

8 Software Development and Build Process

8.1 The RISC-V Toolchain and its Extensions

To compile software for the VestaRV CPU core, you will need the RISC-V GNU toolchain, an open source compiler suite supporting multiple programming languages. The base instruction set, RV32I (Integer), provides fundamental integer arithmetic operations, excluding multiplication, division, and modulo. All RV32I instructions are 32 bits in length without exception.

The Myshkin MCU's VestaRV core implements several standard and extended instruction sets beyond the base RV32I:

- **M Extension (Integer Multiplication and Division):** Adds hardware multiply (`mul`), divide (`div`), and modulo/remainder (`rem`) instructions with both signed and unsigned variants. Without this extension, the compiler must emulate these operations in software, significantly impacting performance.
- **A Extension (Atomic Instructions):** Provides atomic memory operations including load-reserved/store-conditional (`lr.w`/`sc.w`) and atomic read-modify-write operations (`amoadd.w`, `amoswap.w`, etc.). While primarily designed for multiprocessor synchronization, these instructions ensure atomic operations in interrupt-driven single-core systems.
- **C Extension (Compressed Instructions):** Introduces 16-bit compressed encodings for frequently used instructions, reducing program size by approximately 20-30% according to RISC-V documentation. Uncompressed instructions on non-4-byte-aligned addresses may incur additional execution cycles. This extension significantly improves code density without sacrificing functionality.
- **Zba Extension (Address Generation):** Provides shift-and-add instructions (`sh1add`, `sh2add`, `sh3add`) optimized for efficient array indexing and pointer arithmetic, particularly beneficial for accessing elements in arrays with 2, 4, or 8-byte strides.
- **Zbb Extension (Basic Bit Manipulation):** Includes instructions for bit counting (`clz`, `ctz`, `cpop`), logical operations with inverted operands (`andn`, `orn`, `xnor`), min/max operations, sign/zero extension, rotations, and byte manipulation. These instructions accelerate bit-level algorithms, cryptographic primitives, and data packing/unpacking.
- **Zbc Extension (Carryless Multiplication):** Implements carryless multiplication instructions (`clmul`, `clmulh`, `clmulr`) essential for efficient CRC calculations, error detection codes, and certain cryptographic operations such as Galois field arithmetic.
- **Zbs Extension (Single-Bit Manipulation):** Offers single-bit set, clear, invert, and extract operations (`bset`, `bclr`, `binv`, `bext`) with both register and immediate variants, facilitating efficient manipulation of bit fields and flags.

When compiling for the Myshkin MCU, use `-march=rv32im_zba_zbb_zbc_zbs -mno-div`. The core also implements the A and C extensions, but this build avoids the core errata (Section 20). Individual extensions may be selectively disabled by omitting them from the `-march` flag if desired for compatibility testing or code size optimization.

8.2 Getting the RISC-V Toolchain on Linux

There are two primary methods for obtaining the RISC-V toolchain on Linux: installing a prebuilt binary distribution or building from source. For most users, the prebuilt xPack distribution is recommended due to its simplicity and time savings.

8.2.1 Option 1: Prebuilt xPack Toolchain (Recommended)

The xPack project provides precompiled RISC-V GCC toolchains with support for all standard extensions including RV32IMAC and bit manipulation (Zb*). This is the fastest and simplest installation method.

1. Download the latest xPack RISC-V Embedded GCC release from:

   https://github.com/xpack-dev-tools/riscv-none-elf-gcc-xpack/releases

   Select the appropriate package for your system (typically `xpack-riscv-none-elf-gcc-*-linux-x64.tar.gz` for 64-bit Linux systems).

2. Create a directory for the toolchain and extract the archive:

   ```
   mkdir -p ~/riscv-toolchain
   ```

   ```
   tar -xf xpack-riscv-none-elf-gcc-*.tar.gz -C ~/riscv-toolchain/
   ```

3. Set the toolchain path as an environment variable. Add the following to your `~/.bashrc` or `~/.profile` file:

   ```
   export RISCV_TOOLCHAIN_DIR=~/riscv-toolchain/xpack-riscv-none-elf-gcc-13.2.0-2
   ```

   ```
   export PATH=$RISCV_TOOLCHAIN_DIR/bin:$PATH
   ```

   Replace `13.2.0-2` with your downloaded version number.

4. Reload your shell configuration or open a new terminal:

   ```
   source ~/.bashrc
   ```

5. Verify the installation:

   ```
   riscv-none-elf-gcc --version
   ```

   You should see the GCC version information confirming successful installation.

8.2.2 Option 2: Building from Source

Building the toolchain from source provides maximum flexibility and ensures the latest features, but requires significantly more time and disk space. You can find the source code for the RISC-V toolchain at https://github.com/riscv/riscv-gnu-toolchain, which also contains instructions for building the toolchain. The following procedure builds a toolchain with full support for RV32IMAC and bit manipulation extensions.

First, install the required build dependencies on Ubuntu or Linux Mint:

```
sudo apt install autoconf automake autotools-dev curl libmpc-dev
   libmpfr-dev libgmp-dev gawk build-essential bison flex texinfo gperf
    libtool patchutils bc zlib1g-dev libexpat-dev
```

The instructions are as follows:

1. Switch to your home directory with

   ```
   cd ~
   ```

2. Make a new directory with

   ```
   mkdir riscv-download
   ```

   and go into it with

   ```
   cd riscv-download
   ```

3. Download the RV32IMC toolchain with

   `git clone --recursive https://github.com/riscv/riscv-gnu-toolchain riscv-gnu-toolchain -rv32imc`

4. Enter into the new directory with

   `cd riscv-gnu-toolchain-rv32imc`

5. Create a build directory with

   `mkdir build`

   Enter into the new build directory with `cd build`

6. Configure the build with

   `../configure --with-arch=rv32imc --prefix=/opt/riscv/rv32imc`

7. Begin the build process with

   `sudo make`

   This will take quite a while. If you wish to speed up the process, you can allocate more CPU cores to the build process by using `make -jn` instead of `make` where n can be replaced by the number of CPU cores you want to use to build the toolchain.). Once the build process has finished successfully, the toolchan will be located at `/opt/riscv/rv32imc`

8. Add a permanent environment variable to your system that defines the location of the `riscv` directory (perhaps in your `.bashrc` script) with something like

   `export RISCV=/opt/riscv`

   You may also wish to add the `/opt/riscv/rv32imc/bin` directory to your PATH, though this is optional.

9. You may now safely delete the `riscv-download` directory with

   `sudo rm -rf ~/riscv-download`

8.3 Getting the RISC-V Toolchain on Windows

The RISC-V toolchain has no official support for Windows, but several options exist to get it working. For Windows 10 users, it is recommended to install the Windows Subsystem for Linux (WSL) in Ubuntu mode, which can be found in the Microsoft Store app. Then, in WSL, follow the installation instructions in Section 8.2. You must then use WSL to compile all software for the RISC-V MCU. If you have an older version of Windows, it is suggested that you use an Ubuntu or Linux Mint virtual machine to build the RISC-V toolchain and compile your MCU software.

8.4 Compilation

The RISC-V toolchain includes a GCC compiler. Thus, the compilation process is similar to other toolchains that use GCC. To compile a source file into an object file, use:
`riscv32-unknown-elf-gcc -c $CFLAGS srcFile -o objectFile.o`
or for the xPack toolchain:
`riscv-none-elf-gcc -c $CFLAGS srcFile -o objectFile.o`

where `$CFLAGS` should be replaced by any compilation flags, `srcFile` is the source code file (C, C++, assembly, or other GCC-supported languages), and `objectFile.o` is the output object file. The compiler determines which launguage is used in a source file by the file extension (`.c` is for C, `.cpp` is for C++, `.S` is for assembly code which must be preprocessed, and `.s` is for assembly code which is not preprocessed).

There are several compilation flags that are suggested:

- `-march=rv32im_zba_zbb_zbc_zbs -mno-div` Architecture flags. These specify which ISA extensions are enabled during compilation: the base integer instructions (I), multiplication (M), and all bit manipulation extensions (Zba, Zbb, Zbc, Zbs). Compressed instructions (C) are left out and `-mno-div` replaces hardware division with library calls, because of the core errata (Section 20). Other extensions may be disabled by omitting them from the string (e.g., `-march=rv32im` to exclude bit manipulation).
- `-mabi=ilp32` Application Binary Interface (ABI) flag. Specifies the calling convention and data model. Use `ilp32` for integer-only ABI without floating point support, which is appropriate for the VestaRV core.
- `-Os` Optimize for program size flag. Attempts to minimize the compiled program size, which is particularly important for memory-constrained embedded systems.
- `-O<number>` Optimization level flag. Controls compiler optimization aggressiveness. Common values are `-O0` (no optimization, useful for debugging), `-O1` (basic optimizations), `-O2` (recommended for production), `-O3` (aggressive optimizations, may increase code size), and `-Og` (optimize for debugging experience).
- `-I <include_directory>` Include directory flag. Adds the specified directory to the list of directories searched for header files. Any files in the include directory may be included with the `#include <filename.h>` preprocessor directive.
- `-g` GDB debugger enable flag. Includes debugging symbols in the compiled output. While the VestaRV core does not support hardware debugging, adding the `-g` flag to both the compiler and linker commands will allow disassembly listings (`.lst` files) to show the correspondence between source code and generated assembly instructions.
- `-Wall -Wextra` Warning flags. Enable comprehensive compiler warnings to catch potential issues during development. The `-Wall` flag enables common warnings, while `-Wextra` adds additional checks.

Example compilation command for Myshkin MCU:

```
riscv-none-elf-gcc -c \
  -march=rv32im_zba_zbb_zbc_zbs -mno-div \
  -mabi=ilp32 -O2 -Wall -Wextra \
  -I./include -g \
  main.c -o main.o
```

You must compile every source file that you wish to use in your project. If your project is in C or C++, it is suggested that you use the provided `start.S` assembly file to call your main function. This assembly file needs to have its first instruction at address `0x8200`, so it must be the first file to be given to the linker. You must have one and only one `int main()` function throughout all of your source files in a single project. For C++ projects, you need to start the main function with `extern "C"int main()` to prevent the C++ compiler from name-mangling the main function.

8.5 Linking

Once all source files have been compiled, they must be linked to produce the output program binary file, the `.elf` file. The linker assigns memory addresses to every variable and function. The MCU contains no writable non-volatile memory, so the program and the variables all share the RAM (with a notable exception for the bootloader and Forth interpreter stored on the ROM). The linker must be aware of the RAM and ROM start address and size, the interrupt vector table (IVT) start address and size (along with the address of each ISR pointer within the IVT), the first program instruction address that the bootloader jumps to after loading the program into RAM, and the address of the master interrupt handler.

Defining the aforementioned memory segments and addresses is done using a linker script. It is highly suggested to use the provided linker script `MCU.ld` and its include files and tweak it to suit your need.

To execute the linker, use:

```
riscv32-unknown-elf-gcc $LFLAGS $OBJECTS -o program.elf
```

where `$LFLAGS` is a list of linker flags, `$OBJECTS` is a list of all the compiled object files needed for the project, and `program.elf` is the output ELF binary containing the program.

There are several linker flags that are suggested:

- `-march=rv32im_zba_zbb_zbc_zbs` Architecture flag. This specifies which extensions are enabled in the linking process. The VestaRV core supports the RV32IMAC instruction set plus the Zba, Zbb, Zbc, and Zbs bit manipulation extensions. This flag must match the architecture flags used during compilation.
- `-flto` Link-time optimization flag. This instructs the linker to perform optimizations to the final binary.
- `-nostdlib` Disables the use of the standard library for linking, decreasing software bloat. However, using the flag precludes using standard library functions such as software emulation of integer multiplication and division, floating point numbers, initialization routines, memory allocation, etc. It is suggested that you only use this flag if you have a specific need to reduce the software size.
- `-T linker_script.ld` Path to the linker script
- `-Map mapfile.map` Path to the output map file, which contains the symbol table for your project with their addresses
- `-L linkerfiles_dir` Path to the linker scripts directory that contains any files that are included in the main linker script
- `-g` GDB debugger enable flag. While the VestaRV core does not support hardware debugging, adding the `-g` flag to both the compiler and linker commands will allow the `.lst` file to show the assembly commands your code compiles into.

8.6 Intel Hex File Generation

Once the final ELF file has been created from the linker, an Intel hex file can be created, which converts the binary program data into an easy-to-read ASCII file. Furthermore, the Python package `IntelHex` can read the file, allowing you to use Python to upload a new program to the SPI flash. A Python program that utilizes `IntelHex` to upload new programs, called `forthprog.py`, is provided.

8.7 A Note on Using C++

C++ generates some extra data sections that C and assembly do not use. In particular, it generates the `.eh_frame` section, which is used for exception handling. You may not need exception handling, in which case you can pass in the compiler flags:

```
-fno-exceptions -fno-unwind-tables -fno-asynchronous-unwind-tables
```

which will significantly reduce the size of `.eh_frame`. If you wish to eliminate it entirely, you can create a phony section in the linker script at an unused memory address and point the `.eh_frame` section to it.

9 Uploading Program to SPI Flash

9.1 SPI Flash Boot Process

In order to boot in SPI flash mode, there must first be a vaild program stored in the SPI flash memory in the proper location. In SPI flash mode, the bootloader reads the data in the SPI flash memory starting at address `0x8200` and copies every 32-bit word into RAM, also starting at address `0x8000`. This particular address is the beginning of the RAM on the MCU. The bootloader stops copying from the SPI flash once completely fills the RAM.

Note the distinction between copying 32-bit words as opposed to 8-bit bytes. Though bytes and words are both MSB-first, they are also transmitted starting with the lowest address to the greatest address from the SPI flash. If the bootloader copied every 8-bit byte, the transmission sequence it would expect would be:
`byte0, byte1, byte2, byte3, byte4, byte5, byte6, byte7 ...`
However, because the bootloader expects MSB-first 32-bit words, the sequence must be:
`byte3, byte2, byte1, byte0, byte7, byte6, byte5, byte4 ...`
Therefore, the program data on the SPI flash must follow the 32-bit word sequence.

9.2 Upload Process

The Forth boot mode has a mechanism for writing data to the SPI flash, which may be used to upload a new program for use in the SPI flash boot mode. When in Forth boot mode, the flash write `fw` function writes a page of data (which is exactly 256 bytes long and begins on an address that is a multiple of 256) to the SPI flash. The Forth function is used with the syntax:

```
bin addr fw
```

If `bin` is true (nonzero), the command expects 256 bytes of binary data to be transmitted over the UART. If false, it expects a 512-character long ASCII hex string representing the data to be transmitted over the UART. The `addr` argument must be the address of the beginning of the 256-byte page, and must be aligned to a 256-byte boundary.

To execute the command, a newline (or line feed) character must be transmitted to the MCU over the UART. Once the command begins executing, it initiates some handshaking to ensure a proper transaction. The MCU prints a `$` character to indicate it is ready for the page data to be transmitted. After that, you must transmit the page data in the format specified by the `bin` argument. Remember to transmit the data in the proper order (byte3, byte2, byte1, byte0, byte7, byte6, byte5, byte4 ...). Once the page data has been transmitted, the MCU will compute a CRC value upon the data using the CRC16_CDMA2000 algorithm (polynomial = 0xC857, initial CRC value = 0xFFFF, no final XOR, no data reflection, no output reflection) and then transmit the CRC value as a 4-character long hex string. If this value matches your own CRC computation (or if you do not care and simply want it to write the data), transmit a single `Y` character without anything else to tell the MCU to write the data to the SPI flash. Any other character will abort the process, leaving the SPI flash memory unchanged. You are technically supposed to wait an average of 10 ms (25 ms at the greatest) before writing more data to the SPI flash. However, the substantial handshaking and transmission times likely fulfill this requirement, and it is likely that no delay is needed.

If the entire page is blank, you may instead use the flash erase `fe` function to erase the page, making every byte in the page equal to 0xFF. This is much quicker than writing data to the SPI flash, and can speed up programming times substantially for smaller programs. The syntax to use it is:

```
conf addr fe
```

If the `conf` parameter is equal to 123, the function will erase the 256-byte page at address `addr` (which must be 256-byte aligned) and then return 1. If not equal to 123, it will not erase the page and will return 0. You are technically supposed to wait an average of 6 ms (25 ms at the greatest) before another erase

or write operation. Because the erase function is much quicker than the write function, it may become necessary to delay between erases at higher bauds.

You may read back the data stored on the SPI flash with the `fr` function. See Section 7.8 for details.

In order to write an entire program to the SPI flash, you must write/erase a total of 128 pages (each with a length of 256 bytes) beginning with page address 0x8000. After the data is written, you may execute the newly uploaded program by putting the MCU in reset, setting the BOOT pin to SPI flash mode, and taking the chip out of reset. Or, you can simply set the BOOT pin and move the program counter to address 0x0000, which may be done with the `rst` Forth function.

It is highly suggested to use the provided Python program `forthprog.py` to upload new programs to the SPI flash, which takes an Intel hex file containing the new program and sends it to the MCU and the SPI flash using the process described above.

10 GPIOx Peripheral

The MCU has several GPIOx peripherals, each of which controls a set of general purpose input/output (GPIO) pins. The VestaRV implementation supports GPIO ports with 8, 16, or 32 pins. Each GPIO peripheral (typically called a port) constitutes a parallel port, allowing all pins in the port to be updated simultaneously. Each GPIO pin may also be configured and updated individually, allowing for design flexibility. Each GPIO pin is referred to as Px.y, where x refers to the port number and y refers to the corresponding bit in the GPIO registers that controls the pin. For example, P2.5 refers to port 2, bit 5.

Every GPIO pin on the MCU can be in one of two modes: primary/GPIO mode, or secondary/peripheral mode. Section 2 details the secondary function of each GPIO pin.

10.1 Primary/GPIO Mode Functionality

In primary/GPIO mode, the GPIO pin state is controlled by the GPIO registers PxDIR, PxOUT, and PxREN. The PxDIR register determines if the pin is in input or output mode, the PxOUT register determines if the pin outputs a logic high or logic low level when the pin is in output mode, and the PxREN register enables or disables the internal pullup/pulldown resistor. Table 5 shows the various states available to every GPIO pin.

PxDIR[y]	PxOUT[y]	PxREN[y]	Px.y State
0	-	0	High impedance input, resistor disabled
0	-	1	Pullup/pulldown resistor enabled
1	0	-	Output, strongly driven low
1	1	-	Output, strongly driven high

Table 5: GPIO Configurations

The PxIN register always reads the digital state of the pin, regardless of whether it is in input or output mode, and regardless of whether it is in primary/GPIO mode or secondary/peripheral mode. The pin Px.y is at logic low level if PxIN[y] is 0, and is at logic high level if it is 1. Note that PxIN is latched on the falling edge of the memory enable signal to ensure stable readings during memory access.

10.2 Register Access Convenience Features

The GPIO peripheral provides convenient set, clear, and toggle registers for efficient bit manipulation without read-modify-write operations:

- PxOUTS: Writing 1 to a bit sets the corresponding output pin high. Writing 0 has no effect. Reading returns the current PxOUT value.
- PxOUTC: Writing 1 to a bit clears the corresponding output pin low. Writing 0 has no effect. Reading returns the inverted PxOUT value.
- PxOUTT: Writing 1 to a bit toggles the corresponding output pin. Writing 0 has no effect. Reading returns the current PxOUT value.

These registers eliminate race conditions in interrupt-driven or multi-threaded code where multiple code paths may modify GPIO outputs.

10.3 Secondary/Peripheral Mode Functionality

Most GPIO pins have a secondary/peripheral function which is controlled by one of the peripherals in the MCU. The mode of the GPIO is determined by the PxSEL register. If PxSEL[y] is 0, Px.y is in primary/GPIO mode, and if it is 1, Px.y is in secondary/peripheral mode.

When in peripheral mode, the peripheral seizes control over the pin's DIR, OUT, and REN controls, overriding PxDIR[y], PxOUT[y], and PxREN[y]. For example, when the SCK0 pin is in peripheral mode, the SPI0 peripheral controls the state of the pin, and the corresponding PxDIR[y], PxOUT[y], and PxREN[y] registers have no effect until PxSEL[y] returns to 0.

Some pins with secondary/peripheral functions may only be used as inputs by the peripheral. For example, timer capture pins like T0CAP0 must be manually configured as inputs in PxDIR before use, even when PxSEL[y] is set to enable the secondary function.

10.4 Pin Interrupts

The VestaRV GPIO implementation provides individual interrupt outputs for each GPIO pin. Each pin can generate an interrupt independently, allowing for flexible interrupt handling through the system's interrupt vector table.

To enable interrupts on a pin:

1. Unmask the corresponding GPIO pin interrupt in the system interrupt controller
2. Set PxIE[y] to 1 for the desired pin Px.y
3. Configure the interrupt edge using PxIES[y]: 0 for low-to-high (rising edge), 1 for high-to-low (falling edge)

When the pin Px.y experiences the edge selected by PxIES[y] while PxIE[y] is enabled, the interrupt flag PxIF[y] is immediately set to 1 and the corresponding pin-specific ISR is executed. The interrupt flag must be cleared by writing 1 to PxIF[y] before the ISR returns, otherwise the interrupt will re-trigger.

Important: Modifying PxIES[y] while PxIE[y] is enabled may immediately generate a spurious interrupt. Always disable the pin interrupt (PxIE[y] = 0) before changing PxIES[y].

The PxIF register is also latched on the falling edge of the memory enable signal to ensure stable readings during interrupt service routines.

10.5 Register Description

10.5.1 PIN Register

Register memory slot: 0, *offset:* 0 bytes, *size:* 1 byte
GPIO read pin register. Each bit corresponds to the input logic state of the GPIO pin of the same number. The register is latched on the falling edge of the memory enable signal. Reading a 0 in a bit indicates a logic low pin state; reading a 1 indicates a logic high state. This register always reflects the pin state regardless of pin direction or peripheral select settings.

7	6	5	4	3	2	1	0
IN[7]	IN[6]	IN[5]	IN[4]	IN[3]	IN[2]	IN[1]	IN[0]
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	r-(0)

10.5.2 POUT Register

Register memory slot: 1, *offset:* 4 bytes, *size:* 1 byte
GPIO output drive register. Each bit corresponds to the output logic state of the GPIO pin of the same number. Only has an effect if the pin is configured as an output in PxDIR and is set to GPIO (primary) mode in PxSEL. Write a 0 to the desired bit to make the corresponding pin output a logic low value; write a 1 to output a logic high value.

7	6	5	4	3	2	1	0
OUT[7]	OUT[6]	OUT[5]	OUT[4]	OUT[3]	OUT[2]	OUT[1]	OUT[0]
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

10.5.3 POUTS Register

Register memory slot: 2, *offset:* 8 bytes, *size:* 1 byte
GPIO output drive set register. Each bit corresponds to the output logic state of the GPIO pin of the same number. Only has an effect if the pin is configured as an output in PxDIR and is in GPIO (primary) mode in PxSEL. Write a 1 to the desired bit to set the corresponding pin (make the pin output a logic high value). Writing a 0 has no effect. Reading this register is equivalent to reading the output drive register PxOUT.

7	6	5	4	3	2	1	0
OUTS[7]	OUTS[6]	OUTS[5]	OUTS[4]	OUTS[3]	OUTS[2]	OUTS[1]	OUTS[0]
rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)

10.5.4 POUTC Register

Register memory slot: 3, *offset:* 12 bytes, *size:* 1 byte
GPIO output drive clear register. Each bit corresponds to the output logic state of the GPIO pin of the same number. Only has an effect if the pin is configured as an output in PxDIR and is in GPIO (primary) mode in PxSEL. Write a 1 to the desired bit to clear the corresponding pin (make the pin output a logic low value). Writing a 0 has no effect. Reading this register yields the inversion of the output drive register PxOUT.

7	6	5	4	3	2	1	0
OUTC[7]	OUTC[6]	OUTC[5]	OUTC[4]	OUTC[3]	OUTC[2]	OUTC[1]	OUTC[0]
rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)

10.5.5 POUTT Register

Register memory slot: 4, *offset:* 16 bytes, *size:* 1 byte
GPIO output drive toggle register. Each bit corresponds to the output logic state of the GPIO pin of the same number. Only has an effect if the pin is configured as an output in PxDIR and is in GPIO (primary) mode in PxSEL. Write a 1 to the desired bit to toggle the corresponding pin state. Writing a 0 has no effect. Reading this register is equivalent to reading the output drive register PxOUT.

7	6	5	4	3	2	1	0
OUTT[7]	OUTT[6]	OUTT[5]	OUTT[4]	OUTT[3]	OUTT[2]	OUTT[1]	OUTT[0]
rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)

10.5.6 PDIR Register

Register memory slot: 5, *offset:* 20 bytes, *size:* 1 byte
GPIO pin direction register. Each bit corresponds to the GPIO pin of the same number. Only has an effect if the pin is configured in GPIO (primary) mode in PxSEL. Write a 0 to the desired bit to set the corresponding pin to input mode; write a 1 to set to output mode.

7	6	5	4	3	2	1	0
DIR[7]	DIR[6]	DIR[5]	DIR[4]	DIR[3]	DIR[2]	DIR[1]	DIR[0]
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

10.5.7 PIF Register

Register memory slot: 6, *offset:* 24 bytes, *size:* 1 byte
GPIO interrupt flag register. Each bit corresponds to the GPIO pin of the same number. The register is latched on the falling edge of the memory enable signal. Reading a 0 in a bit indicates there is no pending interrupt for the corresponding pin; reading a 1 indicates there is a new interrupt pending for the corresponding pin. Write a 1 to each bit for which you wish to clear the interrupt flag. Writing 0 has no effect.

7	6	5	4	3	2	1	0
IF[7]	IF[6]	IF[5]	IF[4]	IF[3]	IF[2]	IF[1]	IF[0]
rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)

10.5.8 PIES Register

Register memory slot: 7, *offset:* 28 bytes, *size:* 1 byte
GPIO interrupt edge select register. Each bit corresponds to the GPIO pin of the same number. Write a 0 to the desired bit to set the corresponding pin interrupt to trigger on low-to-high (rising) edge; write a 1 to set to high-to-low (falling) edge triggering.

7	6	5	4	3	2	1	0
IES[7]	IES[6]	IES[5]	IES[4]	IES[3]	IES[2]	IES[1]	IES[0]
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

10.5.9 PIE Register

Register memory slot: 8, *offset:* 32 bytes, *size:* 1 byte
GPIO interrupt enable register. Each bit corresponds to the GPIO pin of the same number. Write a 0 to the desired bit to disable the pin interrupt; write a 1 to enable the pin interrupt. Each pin has an individual interrupt output that connects to the system interrupt vector table. Interrupts function in both GPIO (primary) and secondary function (peripheral) modes.

7	6	5	4	3	2	1	0
IE[7]	IE[6]	IE[5]	IE[4]	IE[3]	IE[2]	IE[1]	IE[0]
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

10.5.10 PSEL Register

Register memory slot: 9, *offset:* 36 bytes, *size:* 1 byte
GPIO peripheral select register. Each bit corresponds to the GPIO pin of the same number. Write a 0 to the desired bit to set the corresponding pin to GPIO (primary) mode; write a 1 to set the pin to secondary function (peripheral) mode. When a pin is in secondary function (peripheral) mode, the governing peripheral takes control of the pin output, direction, and resistor enable states, and the PxOUT, PxDIR, and PxREN registers have no effect on the pin. Pin interrupts remain available when in secondary function (peripheral) mode in addition to any interrupts the secondary function/peripheral may generate. If a pin has no secondary function defined, setting PxSEL to 1 will configure the pin as a high-impedance input.

7	6	5	4	3	2	1	0
SEL[7]	SEL[6]	SEL[5]	SEL[4]	SEL[3]	SEL[2]	SEL[1]	SEL[0]
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

10.5.11 PREN Register

Register memory slot: 10, *offset:* 40 bytes, *size:* 1 byte
GPIO resistor enable register. Each bit corresponds to the GPIO pin of the same number. Only has an effect if the pin is configured in GPIO (primary) mode in PxSEL. Write a 0 to the desired bit to disable the pin pullup/pulldown resistor; write a 1 to enable the pin pullup/pulldown resistor.

7	6	5	4	3	2	1	0
REN[7]	REN[6]	REN[5]	REN[4]	REN[3]	REN[2]	REN[1]	REN[0]
rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

11 SPIx Peripheral

The Serial Peripheral Interface (SPI) peripheral is a high speed, full-duplex, synchronous, multipoint serial interface. It transfers one bit at a time. The SPI bus requires three wires (SCK, MOSI, and MISO), along with one CS wire for every slave. The main features of the SPI peripheral are listed below:

- Master or slave mode (`SPI0` is master-only, `SPI1` supports both master and slave modes)
- LSB-first or MSB-first transfers
- 8-, 16-, or 32-bit transfer lengths
- Master mode supports SPI modes 0-3
- Slave mode supports SPI modes 1 and 3 (CPHA = 1 only)
- Optional byte swap for multi-byte transfers with systems of different endiannesses
- Programmable transfer rate with configurable baud rate divider
- Continuous transfer operation to reduce dead time between frames
- Interrupts for transfer complete and TX register empty
- Flash extended memory feature on `SPI0` for memory-mapped read access to external SPI flash

11.1 SPI Pins

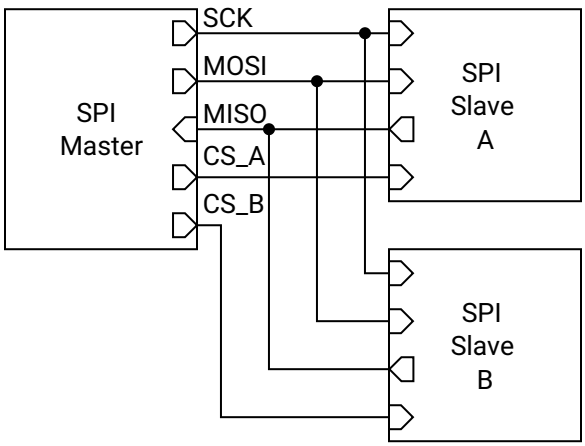


Figure 4: SPI Connection Diagram

The SPI bus has four pins: `SCKx`, `MOSIx`, `MISOx`, and `CSx` (not present on `SPI0`). A connection diagram for the SPI bus is shown in Figure 4.

`SCKx` is the serial clock pin, which governs when each data bit is both latched and updated. It must be attached to the SCK pin on every other device that shares the SPI bus. When the corresponding `PxSEL[y]` bit is a 1, the `SPIx` peripheral seizes the `DIR`, `OUT`, and `REN` controls of `SCKx`.

`MOSIx` is the master out slave in pin, which transfers each data bit from the master to the slave. It is an output when the SPI port is enabled and in master mode, otherwise it is a high impedance input. It

must be attached to the MOSI pin on every other device that shares the SPI bus. When the corresponding PxSEL[y] bit is a 1, the SPIx peripheral seizes the DIR, OUT, and REN controls of MOSIx.

MISOx is the master in slave out pin, which transfers each data bit from the slave to the master. It is an output when the SPI port is enabled, in slave mode, and the CSx pin is asserted (low), otherwise it is a high impedance input. It must be attached to the MISO pin on every other device that shares the SPI bus. When the corresponding PxSEL[y] bit is a 1, the SPIx peripheral seizes the DIR, OUT, and REN controls of MISOx.

CSx is the chip select pin for the SPIx peripheral on this chip. It is a high impedance input that waits to be asserted (pulled low) by an external master if this SPIx peripheral is in slave mode, which begins the SPI transfer. One CS wire needs to be connected between a GPIO pin of the master and the CS pin of the slave for every slave on the bus. Each slave CS wire must be attached to a dedicated GPIO pin on the master in a one-to-one fashion. The SPIx peripheral never seizes control of the CSx pin (regardless of the state of the corresponding PxSEL[y] bit), and as such it must be configured as a high impedance input manually.

11.2 Generalized SPI Transfer Process

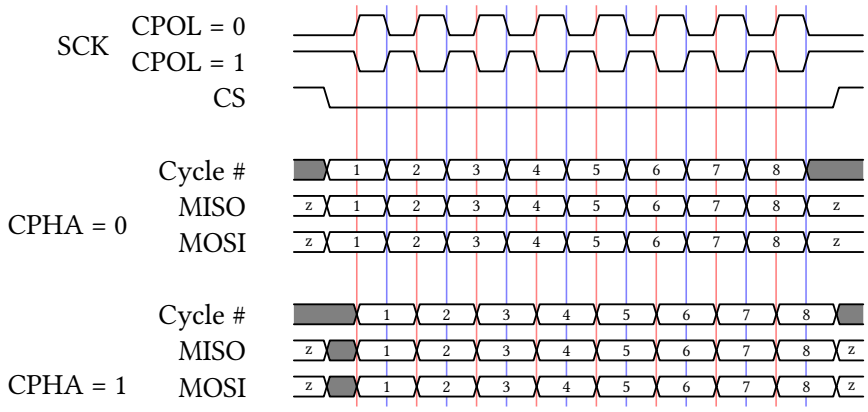


Figure 5: SPI Timing Diagram

When no transfer is occurring, the SPI bus is idle. The transfer begins when the master asserts (pulls low) the CS wire attached to the desired slave. Immediately, the master seizes control of the MOSI wire by making its MOSI pin an output, and the slave does the same for the MISO wire. Note that the master always has control of the SCK and CS wires, which are configured as outputs. Then, the master issues SCK clock pulses in increments of 8, 16, or 32 (one frame of data). On every transition of SCK, both the master and slave alternate between updating and latching the values of MOSI and MISO, the master updating MOSI or latching MISO, and the slave updating MISO or latching MOSI. When the wire is updated, the next bit of data is pushed out of the TX shift register and written to the wire. When the wire is latched, that bit is read from the wire and pushed into the RX shift register. As the SPI port is full-duplex, the master and slave both send data to the other simultaneously. After each frame of data has been transferred, both the master and slave RX shift registers contain their respective received words (the 8-, 16-, or 32-bit data transmitted in the frame). At that point, the master may either continue the process by transferring another frame, or cease the transfer by deasserting (pulling high) the CS wire. The frames that the master issues while the slave's CS wire is low is called a packet, and the master may issue as many frames in a packet as it wants before deasserting CS.

The order of the update/latch process, as well as the polarity of the SCK wire, are determined by two 1-bit parameters: clock polarity (CPOL) and clock phase (CPHA). CPOL determines the idle state and

transition direction of SCK, while CPHA determines when the data in MOSI and MISO is updated and latched. These parameters together are referred to as the SPI mode, or SPIMODE, shown in Table 6. Note that this SPI peripheral supports all four SPI modes while in master mode, but only supports SPI modes 1 and 3 while in slave mode (CPHA = 1). Figure 5 shows a SPI timing diagram for all four SPI modes, with a single-frame packet with 8-bit frames.

SPIMODE	CPOL	CPHA	SCK Idle	Data Update Edge	Data Latch Edge
0	0	0	Low	Trailing/Falling	Leading/Rising
1	0	1	Low	Leading/Rising	Trailing/Falling
2	1	0	High	Trailing/Rising	Leading/Falling
3	1	1	High	Leading/Falling	Trailing/Rising

Table 6: SPIMODE Key

Note that during the very first frame of a transfer, the slave is required to send data to the master on the MISO wire, regardless of if it has any valid data to send. If it has no data to send, a slave will issue a "dummy" word, which the master must ignore.

11.3 Bit and Byte Ordering

The SPI peripheral has the ability to transfer the most significant bit (MSB) first or the least significant bit (LSB) first using `SPIMSB`. When `SPIMSB` is 0, the LSB is transferred first, and when `SPIMSB` is 1, the MSB is transferred first.

For 16- and 32-bit transfers, the byte ordering may optionally be swapped using `SPITXSB` and `SPIRXSB`, which swap the bytes being transmitted and received, respectively. For example, during a 16-bit transfer, if `SPITXSB` is enabled, then byte 1 of `SPITX` will be transmitted followed by byte 0. If `SPIRXSB` is enabled, then the first byte received will be placed in byte 1 of `SPIRX`, and the next 8 bits received will be placed in byte 0. In a 32-bit transfer, the byte order would thus be 3, 2, 1, 0. Figure 6 illustrates this ordering.

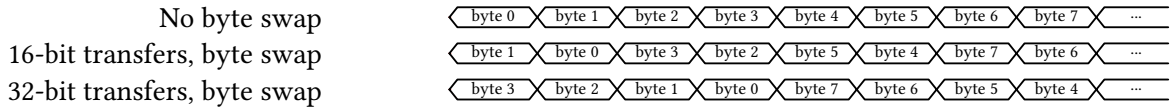


Figure 6: SPI Byte Ordering

Note that the bits in the frame are ordered MSB-first or LSB-first *before* the byte swap operation takes place. As such, a 16-bit, MSB-first transfer with byte swapping activated would transmit the bits 7 down to 0, and then 15 down to 8. Figure 7 shows the bit ordering for a 16-bit transfer.

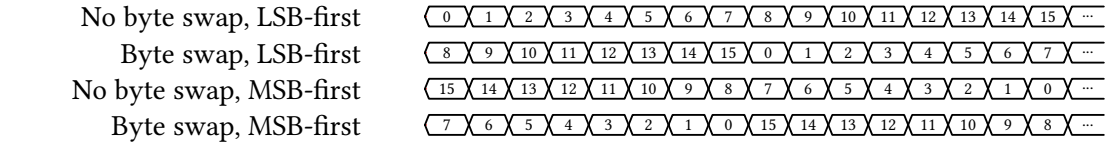


Figure 7: SPI Bit Ordering (16-bit transfer)

11.4 SPI Baud Clock

When in master mode, the frequency of the SCKx clock pin is determined by the frequency of SMCLK and the SPI baud control SPIBR, given by the expression:

$$f_{\text{SCK}} = \frac{f_{\text{SMCLK}}}{2 \times (1 + \text{SPIBR})}$$

Note that the maximum frequency of SCKx is half that of SMCLK.

11.5 Transmit Buffer Register Queue

Writing to the SPI transmit buffer register SPIxTX queues up the word written to it as the next frame to transmit. If in master mode and a transfer is not currently ongoing (i.e., SCKx is not clocking), the SPI peripheral immediately begins transmitting the word written to SPIxTX, and SPITEIF is set immediately. If a transfer is currently ongoing, the SPI peripheral waits for the current frame to finish transferring. As soon as it finishes, the SPI peripheral begins transmitting the new frame written to SPIxTX and sets SPITEIF. If in slave mode, the SPI peripheral latches and begins transferring the last word written to SPIxTX and sets SPITEIF whenever the master starts the first clock cycle of SCKx.

The TX queue allows the SPI peripheral to constantly transfer data without any dead time in between frames, ensuring maximum transfer speed. However, the user must take care not to overflow the queue, which happens if the SPIxTX is written two or more times during a single frame transfer. Queue overflows can be prevented by waiting for the SPITEIF flag to be 1 or SPIBUSY to be 0 before writing to SPIxTX.

11.6 Flags and Interrupts

The SPI peripheral has two interrupt flags (SPITCIF and SPITEIF) and one status indicator (SPIBUSY). If the corresponding interrupts are enabled in the SPIxCR register and the SPIx ISR is not masked, the interrupt flags will generate interrupts when set to 1. Both interrupt flags must be cleared before the ISR returns to prevent spurious re-execution.

The SPITCIF interrupt flag is set when a SPI transfer completes. It can be cleared by writing 1 to it or by reading the SPIxRX register.

The SPITEIF interrupt flag is set when the SPIxTX transmit buffer becomes empty and ready to accept new data. In master mode without an ongoing transfer, it sets immediately when writing to SPIxTX. During an active transfer, it sets when the current frame completes and the queued data begins transmission. In slave mode, it sets when the master begins the first SCK clock cycle. This flag must be cleared by writing 1 to it.

The SPIBUSY indicator reflects the peripheral's transfer state. In master mode, it is 1 during active frame transmission (SCK actively clocking) and 0 when idle. Note that SPIBUSY can be 0 between frames if no new data is queued before the previous frame completes. In slave mode, SPIBUSY is 1 when the CSx pin is asserted (low) and 0 when deasserted (high).

11.7 Master Mode Transfer Procedure

To use the SPI peripheral in master mode, the user must first initialize the SPIx peripheral by configuring the SPIxCR register. Of note, SPIEN must be 1 and SPISM must be 0 to enable the SPI peripheral in master mode. Then, the GPIO pins multiplexed with SCKx, MISOx, and MOSIx must be set to secondary/peripheral mode with their respective bits in the PxSEL registers. Finally, the GPIO pins attached to each slave's CS pin need to be set to output a logic high voltage (note that these CS pins are *not* the same as this SPIx peripheral's own CSx pin, which is used when this device is in slave mode).

To initiate a SPI transfer, the master should first clear the SPI status register SPIxSR by writing 0 to it, and then must assert (pull low) the slave's CS pin. Then, the master may begin transferring as many frames as it wants in the packet. To send a frame, the master must first check if the SPIxTX register is empty (i.e. ready to accept a new word to queue for transfer) by ensuring that either SPITEIF is 1 or SPIBUSY is 0. Then, the master may write the next word to transmit to SPIxTX. Next, if SPITCIF is 1, the master should read the value of the SPIxRX register to read the value the slave transmitted on the prior frame. Then, the master must clear the status register and may either start another frame or end the packet by deasserting (pulling high) the CS wire.

A simplified procedure for producing master mode SPI transfers is shown in Listing 1 that does not use the TX queue for continuous transfers.

```c
/** Includes **/
#include <MemoryMap.h>

/** Function Declarations **/
uint32_t spi_transfer(uint32_t tx_data)

/** Main Function **/
int main()
{
    // Initialize the SPI peripheral
    SPIxCR = SPIEN;

    // Configure the SCKx, MOSIx, and MISOx pins
    SCKx_PxSEL |= SCKx_BIT;
    MOSIx_PxSEL |= MOSIx_BIT;
    MISOx_PxSEL |= MISOx_BIT;

    // Configure the GPIO pin attached to the slave's CS pin as an output high
    PxSEL &= ~(BITy);
    PxOUT |= BITy;
    PxDIR |= BITy;

    // Assert the CS pin
    PxOUT &= ~(BITy);

    // Do a single SPI transfer
    uint32_t tx_data = 57;
    uint32_t rx_data;

    rx_data = spi_transfer(tx_data);

    // Deassert the CS pin
    PxOUT |= BITy;

    return 0;
}

/** Function Definitions **/
uint32_t spi_transfer(uint32_t tx_data)
{
    // Begin a SPI frame by writing the transmit data to SPIxTX
    SPIxTX = tx_data;

    // Wait for the frame to finish transmitting
    while (SPIxSR & SPIBUSY) {}

    // Return the received data from the slave
    return SPIxRX;
}
```

Listing 1: Example Program for SPI Master Transfers

11.8 Slave Mode Transfer Procedure

To use the SPI peripheral in slave mode, the user must first initialize the SPIx peripheral by configuring the SPIxCR register. Of note, SPIEN must be 1 and SPISM must be 1 to enable the SPI peripheral in slave mode. Then, the GPIO pins multiplexed with SCKx, MISOx, and MOSIx must be set to secondary/peripheral mode with their respective bits in the PxSEL registers. Finally, the CSx pin must be set as a high impedance input in GPIO mode to allow the master to assert this SPIx peripheral.

Immediately after initialization, a word should be written to the SPIxTX register, which will be transmitted to the master when the master asserts the CSx pin and begins sending SCK pulses. Whatever data is contained in SPIxTX will always be transmitted to the master when the master begins each frame regardless of if the slave has updated the value of SPIxTX, so care must be taken to always put valid data in the SPIxTX register before the master begins the next frame. Once the master begins sending a frame, the SPITEIF flag will become 1, indicating that the next word to transmit to the master should be written to SPIxTX. At the end of the frame, the SPITCIF flag will become 1, and the data in SPIxRX will contain the data received from the master. SPIxRX should then be read, and the status register SPIxSR must be cleared. Note that the SPIxRX register must be read before the end of the next frame, else the data will be overwritten and unrecoverable.

There is no dedicated interrupt indicating when the CSx pin is asserted or deasserted, but this functionalty can be achieved using the ordinary GPIO pin interrupts.

11.9 External SPI Flash Extended Memory Access

The SPI0 peripheral includes a flash extended memory feature that enables memory-mapped read access to external SPI flash memory. When enabled via the SPIFEN control bit, this feature allows the CPU to read from the SPI flash using standard load instructions, as if the flash were directly addressable ROM on the memory bus. This capability is designed to support programs larger than the internal RAM capacity, up to the full size of the SPI flash (typically 8 MB or more).

Important: Using flash extended memory significantly reduces execution speed compared to RAM-based execution, as each 32-bit read requires multiple SCK clock cycles to retrieve data from the external flash. This feature is best suited for read-only data, lookup tables, or code sections that are not performance-critical.

The flash extended memory feature supports standard SPI flash memories with 256-byte pages. When SPIFEN is enabled, memory accesses in the address range 0x01000000 to 0x01FFFFFF are automatically translated to SPI flash read operations. The SPI0 peripheral manages the flash read protocol transparently, including chip select control, command sequencing, and data retrieval.

11.9.1 Flash Address Translation

The SPI0 peripheral translates CPU memory addresses to SPI flash addresses using the SPI0FOS (SPI Flash Offset) register. The actual flash address accessed is computed as:

$$\text{Flash Address} = (\text{CPU Address} + \texttt{SPI0FOS}) \mod 2^{24}$$

This offset mechanism allows flexible mapping of flash contents to different virtual addresses in the CPU's address space. The 24-bit address wraps at 0xFFFFFF, discarding any overflow bits. By modifying SPI0FOS, software can remap flash regions without physically moving data in the flash memory.

11.9.2 Flash Initialization Procedure

Before enabling the flash extended memory feature, the SPI flash must be properly initialized. The initialization sequence depends on the specific flash chip, but typically includes:

1. Configure SPI0 in master mode with appropriate SPI mode (typically SPIMODE3), 8-bit transfers initially, and SCK frequency within flash specifications (often 50-85 MHz maximum)
2. Set SCK0, MOSI0, and MISO0 pins to secondary/peripheral mode via PxSEL
3. Configure the flash chip select pin (CS_FLASH) as GPIO output, initially high (deasserted)
4. Wake the flash from deep power-down mode if necessary (command varies by flash chip, often 0xAB)
5. Configure flash-specific features such as page size or addressing mode using vendor-specific commands
6. Transfer control of CS_FLASH to the SPI peripheral by setting it to secondary/peripheral mode
7. Enable flash extended memory by setting SPIFEN = 1 in SPI0CR

Once initialized, the flash extended memory feature operates transparently. CPU read operations in the designated address range automatically trigger SPI flash transactions without software intervention.

11.10 SPI0 Boot Behavior

The SPI0 peripheral is used during the boot process to load programs from external SPI flash memory. When the MCU powers on or resets, the BOOT pin determines whether the system fetches the program from SPI flash or launches the interactive Forth interpreter. If programmed to boot from flash, the bootloader in ROM uses SPI0 to read the program from the attached SPI flash and copy it to RAM before jumping to the program start address.

Important: During boot, the SPI0 peripheral actively drives the MISO0 line. If other slave devices share the SPI0 bus, they must not attempt to drive MISO0 during the boot sequence. This conflict can occur if a slave's chip select (CS) pin is controlled by a GPIO pin that defaults to high-impedance input at reset, potentially allowing the slave's CS to float low (asserted state).

To prevent bus contention during boot:

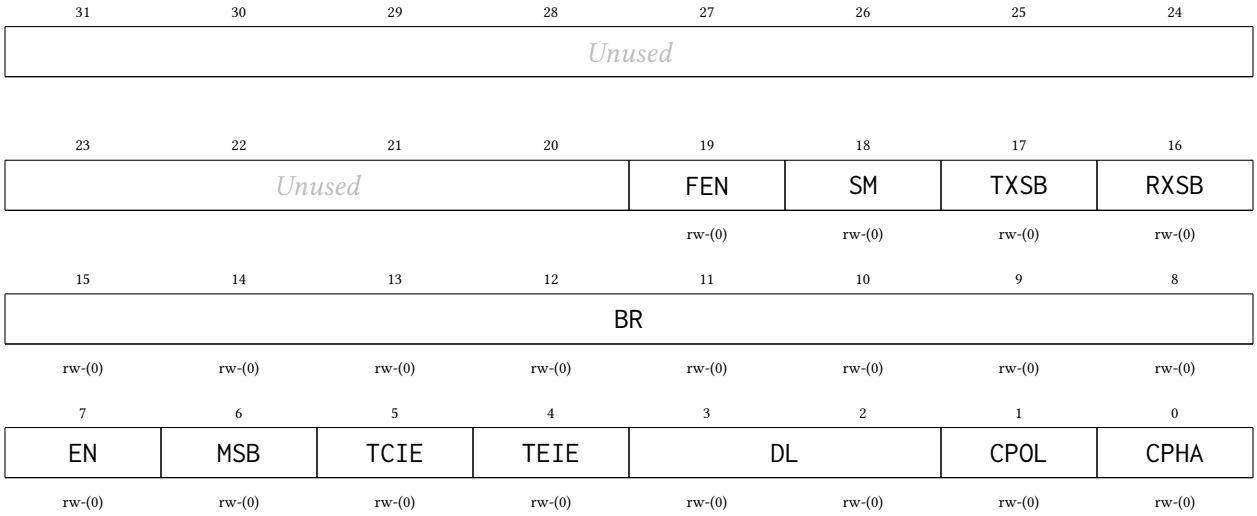
- Use GPIO pins with pull-up resistors or start-high reset behavior to control slave CS pins
- Ensure all non-flash slaves on the SPI0 bus have their CS pins driven high during MCU reset
- Consider dedicating SPI0 exclusively to the boot flash, using SPI1 for other SPI devices
- Add external pull-up resistors to slave CS lines if GPIO reset behavior is insufficient

After the boot sequence completes and the program begins executing, SPI0 becomes available for normal SPI operations, including the flash extended memory feature if enabled by software.

11.11 Register Description

11.11.1 SPICR Register

Register memory slot: 0, *offset:* 0 bytes, *size:* 4 bytes
SPI control register



Bitfield	Range	Description
Unused	Bits 31-20	
FEN	Bit 19	SPI Flash extended memory enable. When enabled, allows native read-only memory access to the SPI flash memory via the system memory bus. The SPI peripheral automatically handles flash read commands and provides transparent memory-mapped access. Available only on SPI0; reads as 0 on SPI1. 0 Flash extended memory disabled 1 Flash extended memory enabled
SM	Bit 18	SPI slave mode select. Configures the SPI peripheral for master or slave operation. SPI0 is master-only; this bit has no effect on SPI0. SPI1 supports both master and slave modes. 0 Master mode 1 Slave mode
TXSB	Bit 17	SPI TX swap bytes. Swaps the byte order in 16- and 32-bit transmissions. In 32-bit transmissions, bytes 3 and 0 are swapped and bytes 2 and 1 are swapped. In 16-bit transmissions, bytes 1 and 0 are swapped. Does not affect 8-bit transmissions. 0 Bytes not swapped 1 Bytes swapped
RXSB	Bit 16	SPI RX swap bytes. Swaps the byte order in 16- and 32-bit receptions. In 32-bit receptions, bytes 3 and 0 are swapped and bytes 2 and 1 are swapped. In 16-bit receptions, bytes 1 and 0 are swapped. Does not affect 8-bit receptions. 0 Bytes not swapped

		1 Bytes swapped
BR	Bits 15-8	SPI clock (SCK) baud rate control for master mode. Baud rate = SMCLK / (2 * (1 + SPIBR)). For example, with SMCLK at 24 MHz: SPIBR=0 gives 12 MHz, SPIBR=1 gives 8 MHz, SPIBR=2 gives 6 MHz, SPIBR=5 gives 4 MHz, SPIBR=11 gives 2 MHz, SPIBR=23 gives 1 MHz.
EN	Bit 7	SPI enable. When disabled, all SPI operations cease and the peripheral is held in reset state. 0 Disabled 1 Enabled
MSB	Bit 6	Bit endianness select. Determines whether data is transmitted and received MSB-first or LSB-first. 0 LSB-first 1 MSB-first
TCIE	Bit 5	SPI transmit complete interrupt enable. Interrupt triggers when a full SPI transfer (transmit and receive) completes. 0 Interrupt disabled 1 Interrupt enabled
TEIE	Bit 4	SPI transmit register empty interrupt enable. Interrupt triggers when SPIxTX register is empty and ready to accept new data for the next transfer. 0 Interrupt disabled 1 Interrupt enabled
DL	Bits 3-2	SPI transmission data length select. Determines the number of bits transferred per SPI transaction. 00 SPIDL_8: 8-bit transfers 01 SPIDL_16: 16-bit transfers 10 SPIDL_32: 32-bit transfers 11 SPIDL_RES: Reserved (do not use)
CPOL	Bit 1	SPI clock (SCK) polarity. Determines the idle state of the SCK line. 0 SCK idles low (SPIMODE0 or SPIMODE1) 1 SCK idles high (SPIMODE2 or SPIMODE3)
CPHA	Bit 0	SPI clock (SCK) phase. Determines when data is sampled relative to the SCK edge. In slave mode, only SPICPHA=1 is supported. 0 Data sampled on leading edge, shifted on trailing edge (SPIMODE0 or SPIMODE2) 1 Data shifted on leading edge, sampled on trailing edge (SPIMODE1 or SPIMODE3)

11.11.2 SPISR Register

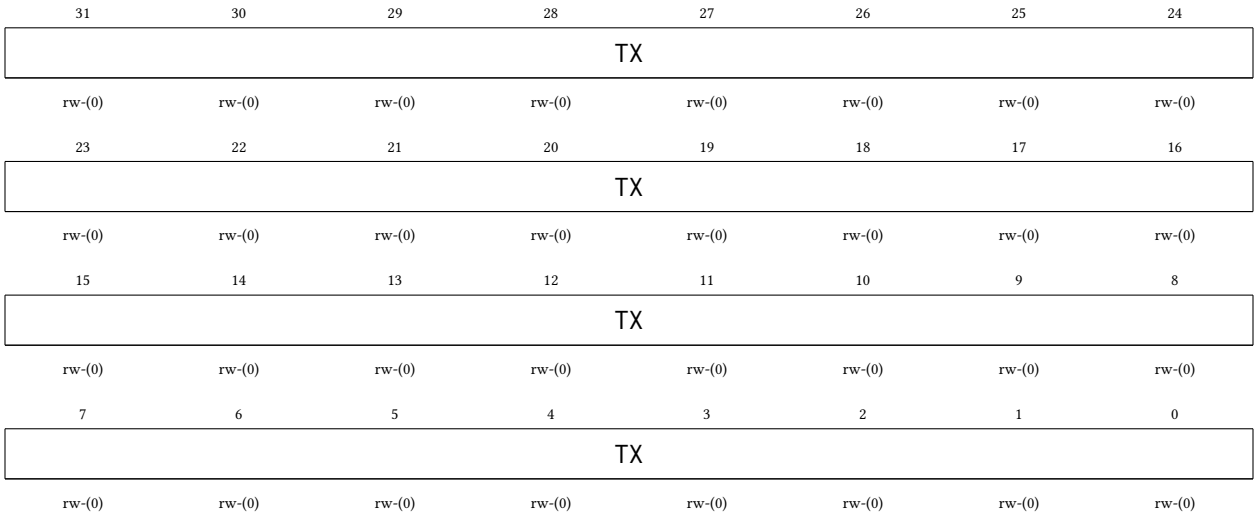
Register memory slot: 1, *offset:* 4 bytes, *size:* 1 byte
SPI status register. Provides real-time status of SPI transfer operations and interrupt flags.

7	6	5	4	3	2	1	0
Unused					BUSY	TCIF	TEIF
					r-(0)	rw1-(0)	rw1-(0)

Bitfield	Range	Description
Unused	Bits 7-3	
BUSY	Bit 2	Indicates whether a SPI transfer is currently in progress. In master mode, set when a transfer starts and cleared when complete. In slave mode, set when chip select is asserted (driven low) and cleared when deasserted. 0 SPI is idle 1 SPI transfer in progress
TCIF	Bit 1	SPI transfer complete interrupt flag. Set when a SPI transfer completes. Must be cleared by writing 1 to this bit or by reading SPIxRX register. 0 No transfer completed 1 Transfer completed
TEIF	Bit 0	SPI transmit register empty interrupt flag. Set when SPIxTX register is empty and ready to accept new data. The data will not be transmitted until any current transfer completes. Must be cleared by writing 1 to this bit. 0 SPIxTX not empty 1 SPIxTX empty and ready

11.11.3 SPITX Register

Register memory slot: 2, *offset:* 8 bytes, *size:* 4 bytes
SPI transmit buffer register. In master mode, writing to this register initiates a new SPI transfer. In slave mode, writing to this register queues data to be transmitted during the next master-initiated transfer. Actual number of bits transmitted is determined by SPIDL setting.

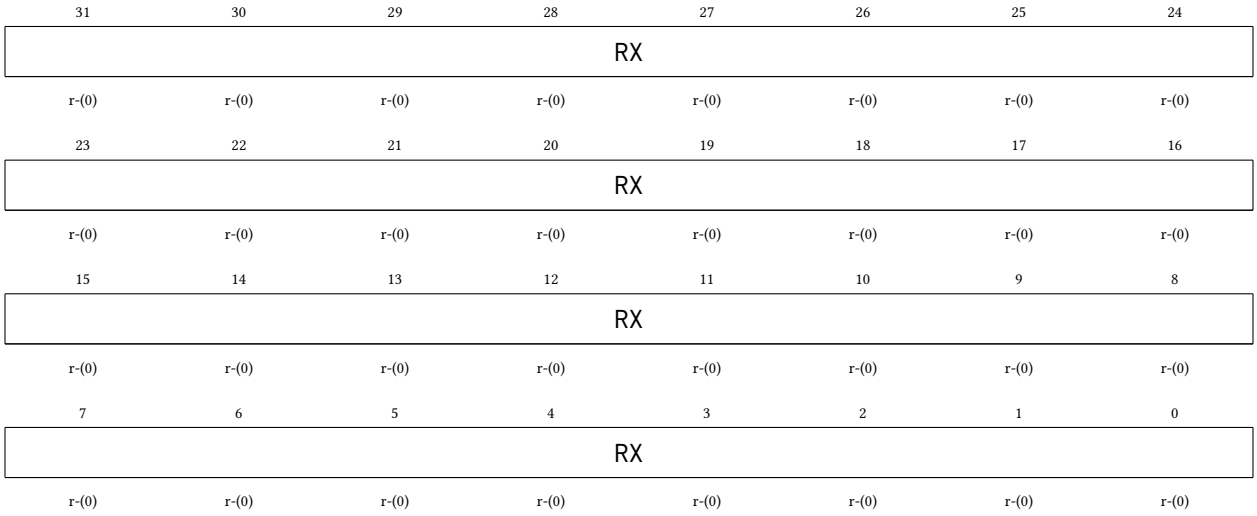


Bitfield	Range	Description
TX	Bits 31-0	

11.11.4 SPIRX Register

Register memory slot: 3, *offset:* 12 bytes, *size:* 4 bytes

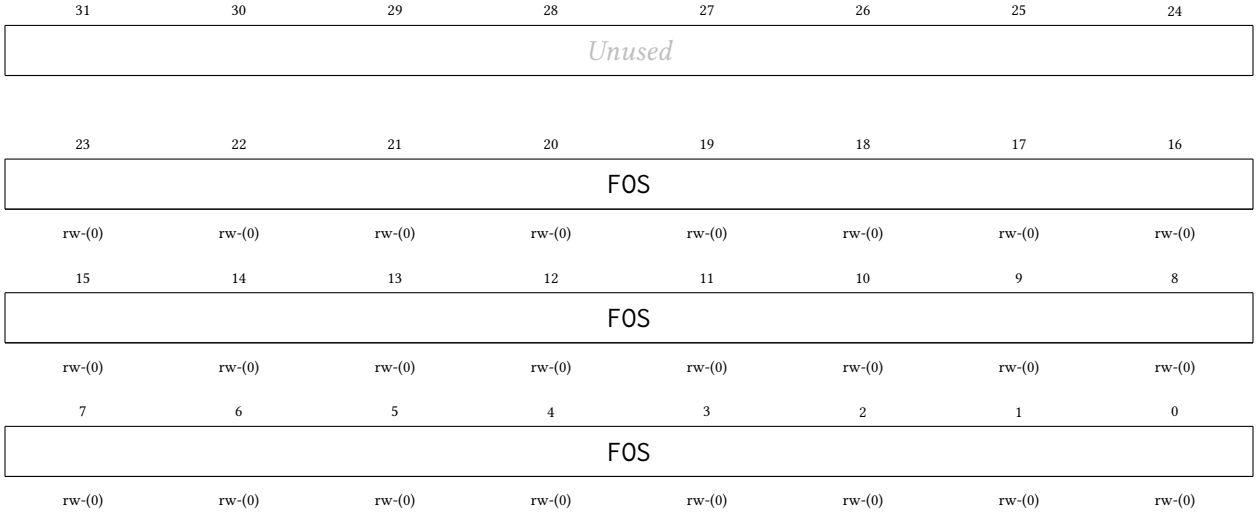
SPI receive buffer register. Contains the data received during the most recent SPI transfer. Reading this register also clears the SPITCIF flag. Valid data width depends on SPIDL setting; unused upper bits read as 0.



Bitfield	Range	Description
RX	Bits 31-0	

11.11.5 SPIFOS Register

Register memory slot: 4, *offset:* 16 bytes, *size:* 4 bytes
SPI Flash memory address offset. This 24-bit value is added to memory access addresses when flash extended memory mode is enabled (SPIFEN=1). Allows remapping of flash memory to different virtual addresses. The addition wraps around at 0x00FFFFFF. Available only on SPI0; reads as 0 on SPI1.



Bitfield	Range	Description
Unused	Bits 31-24	
FOS	Bits 23-0	

12 UARTx Peripheral

The Universal Asynchronous Receiver/Transmitter (UART) is an asynchronous, full-duplex serial interface with hardware parity support. The main features of the UART are listed below:

- Full-duplex, independent receive and transmit operation
- Asynchronous (no serial clock wire)
- Two wires, one for TX and one for RX
- Communicates with one external device
- 8-bit data frames
- Adjustable baud generator with 16× oversampling
- Hardware parity generation and checking (even or odd)
- Error detection: framing error, parity error, and receive overflow flags
- Three separate interrupt sources: receive complete, transmit complete, and TX register empty
- Latched register reads for noise immunity

12.1 UART Pins

The UARTx peripheral has two pins: RXx and TXx.

The TXx pin transmits each data bit from this UARTx peripheral to the external device. It must be attached to the RX pin of the external device. When the corresponding PxSEL[y] bit is a 1, the UARTx peripheral seizes control of the DIR, OUT, and REN controls of TXx, making it an output.

The RXx pin receives each data bit from the external device. It must be attached to the TX pin of the external device. When the corresponding PxSEL[y] bit is a 1, the UARTx peripheral seizes control of the DIR, OUT, and REN controls of RXx, making it a high impedance input.

The UART transmitter and receiver are independent of one another. Thus, transmissions and receptions need not occur at the same time or with the same external device. Indeed, both pins do not necessarily need to be used if one-way communications are all that is needed.

A connection diagram for the UART is displayed in Figure 8.

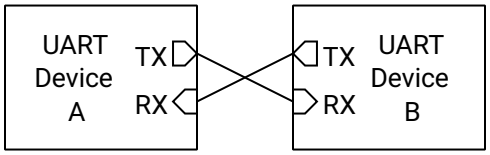


Figure 8: UART Connection Diagram

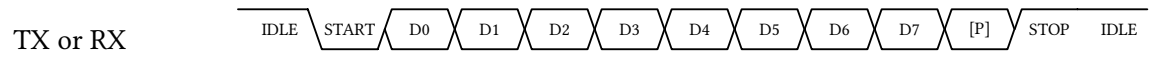


Figure 9: UART Data Frame Timing Diagram

12.2 Generalized UART Transmission Process

Each UART wire is unidirectional, with the TX pin of the transmitting device driving the logic level of the wire and the RX pin of the receiving device reading the level of the pin. The only distinction between the two wires is the direction that data travels, and as such, this section will only depict one wire.

When no UART transmission is occurring, the UART wire is idle, and the TX pin drives the UART wire high. To start a transmission, the TX pin transmits a start bit by driving the wire low for $T_B = 1/\text{baud}$ seconds. It then begins transmitting each data bit, LSB-first, every T_B seconds until all 8 bits have been transmitted. Then, it optionally transmits a parity bit, which helps detect transmission errors. Finally, it transmits a stop bit by driving the wire high for T_B seconds. After that point, it may begin another transmission by sending another start bit, or it can let the wire be idle.

Figure 9 shows a timing diagram for the UART transmission process. Note that the parity bit [P] is optional.

12.3 UART Baud Generation

Both devices communicating over the UART must agree on the baud rate, or the number of bits transmitted per second. In the absence of a clock wire, the UART uses the baud rate to sample the RX wire every $T_B = 1/$ baud seconds, recording the logic levels in sequence. The transmitter must send bits at that same rate. Any significant deviation of the baud rate can result in bit errors.

The UART uses 16× oversampling for improved noise immunity and accurate bit detection. The baud rate for the `UARTx` peripheral is configured through the 12-bit `UARTxBR` register:

$$\text{UART baud} = \frac{f_{\text{SMCLK}}}{16 \times (\texttt{BR} + 1)}$$

For example, with SMCLK running at 48 MHz:

- `BR` = 25 gives 115,200 baud
- `BR` = 51 gives 57,600 baud
- `BR` = 103 gives 28,800 baud

12.4 Parity Bit

A UART frame has an optional parity bit after the last data bit and before the stop bit. The purpose of the parity bit is to verify that the data received is indeed the data that was transmitted. If the received parity bit does not match the expected parity bit, then the receiver knows that the received data has been corrupted.

If the parity bit is enabled with `UPEN`, the transmitter calculates the parity of the byte to transmit by taking the exclusive OR of all 8 bits in the transmitted byte, the result of which is then put through a final exclusive OR gate with the parity select bit `UPSEL`:

$$p = b_7 \oplus b_6 \oplus \cdots \oplus b_1 \oplus b_0 \oplus \texttt{UPSEL}$$

When `UPSEL` is 0, the parity is said to be “even”, and when it is 1, “odd”. This operation is functionally counting how many bits are 1s in the transmitted byte. When even parity is selected, the parity bit is 0

when the number of 1s is even and 1 when the number of 1s is odd. That result is inverted when odd parity is selected.

On the receiver end, the parity bit is capable of detecting a single bit error in the transmission. That is, if one of the eight data bits (or the parity bit itself) is incorrectly received, then the parity error flag UPEF will detect a bit error. However, if two bits are incorrectly received, then the parity error flag will not detect the bit error. Note that if the number of bit errors is an odd number, the parity error flag will detect the error, but will not when the number of bit errors is even.

If a parity bit is enabled on the transmitter side, it must also be enabled on the receiver side with UPEN.

12.5 Transmit Buffer Register Queue

Writing to the UART transmit buffer register UARTxTX queues up the byte written to it as the next frame to transmit. If a transmission is not currently ongoing, the UARTx peripheral immediately begins transmitting the byte written to UARTxTX, and TEIF is set immediately. If a transfer is currently ongoing, the UARTx peripheral waits for the current frame to finish transferring. As soon as it finishes, the UARTx peripheral begins transmitting the new frame written to UARTxTX and sets TEIF.

The TX queue allows the UARTx peripheral to constantly transfer data without any dead time in between frames, ensuring maximum transfer speed. However, the user must take care not to overflow the queue, which happens if the UARTxTX is written two or more times during a single frame transfer. Queue overflows can be prevented by waiting for the TEIF flag to be 1 or TXBF to be 0 before writing to UARTxTX.

12.6 Flags and Interrupts

The UARTx peripheral has five status flags and three interrupt flags in the UARTxSR register. If the corresponding interrupts are enabled in the UARTxCR register and the UARTx ISR is not masked, then the three interrupt flags will generate an interrupt whenever they are equal to 1. Any and all interrupt flags must be cleared before the UARTx ISR returns, else the ISR will mistakenly re-execute immediately upon the prior return.

The UARTxSR and UARTxRX registers are latched on the falling edge of the memory enable signal for improved noise immunity during reads.

The UART receiver busy flag RXBF shows when the UARTx peripheral is currently receiving a byte of data when it is 1. Likewise, TXBF indicates that a transmission is ongoing or pending when 1.

The UART framing error flag FEF indicates that a framing error occurred on the last reception when it is a 1. A framing error typically occurs if the UART transmitter settings do not match the receiver settings, such as baud rate, number of bits to transmit, or the presence/absence of a parity bit. Specifically, a framing error is detected if the receiver does not detect a high level at the expected stop bit time. If FEF is a 1 after a reception, that reception must be treated as unrecoverable, corrupted data. FEF is cleared when the UARTxRX register is read.

The UART parity error flag PEF indicates that a parity error occurred on the last reception when it is a 1. A parity error can only happen if the parity bit is enabled by setting PEN to 1. A parity error occurs when the parity bit appended to the received data byte does not match the expected value calculated from the received data. If PEF is a 1 after a reception, that reception must be treated as corrupted data. PEF is cleared when the UARTxRX register is read.

The UART receive data overflow flag OVF indicates that the user did not read the UARTxRX register in time before another byte of data was received by the UARTx peripheral when it is a 1. In this case, the previous received data has been lost and must be treated as incomplete. OVF is cleared when the UARTxRX register is read.

The UART receive complete interrupt flag RCIF is set to 1 immediately when the UARTx peripheral finishes receiving a frame of data from the transmitter. The new data byte becomes available in the UARTxRX register as soon as RCIF is set. This is also the moment that FEF, PEF, and OVF flags will be updated, if the proper conditions are met. RCIF is cleared when the UARTxRX register is read or by writing a 1 to this bit in UARTxSR.

The UART transmit register empty interrupt flag TEIF is set the moment the UART TX buffer register UARTxTX becomes empty (i.e., is ready to queue up the next byte to transmit). If a frame is not currently being transmitted, the flag is set immediately when a new byte is written to UARTxTX. If a frame is currently being transmitted, it is set as soon as the current transfer is completed, whereupon the value in the UARTxTX register begins to be transmitted. Write a 1 to this bit in UARTxSR to clear TEIF.

The UART transmit complete interrupt flag TCIF is set the moment the UART transmission completes (all bits sent) and the transmitter becomes idle. Write a 1 to this bit in UARTxSR to clear TCIF.

12.7 Transmit Procedure

To transmit a byte of data via the UART, first initialize the UARTx peripheral by configuring the UARTxCR register and setting the baud rate with the UARTxBR register. Of note, the UART enable bit EN must be a 1 to activate the UART. When EN is 0, the UART peripheral is held in reset state. The GPIO pin multiplexed with TXx must be set to secondary/peripheral mode in the corresponding PxSEL register.

To transmit a byte of data over the UART, first clear the status register by reading it and discarding its value. Then, wait until the UART is not transmitting (when TXBF is 0) or the UART TX buffer is empty (when TEIF is 1). Next, write the byte to transmit to the UARTxTX register. The byte will be finished transmitting once TXBF is 0.

A simplified procedure for transmitting a byte over the UART is shown in Listing 2.

```
1  /** Includes **/
2  #include <MemoryMap.h>
3
4  /** Function Definitions **/
5  void uartx_putc(char tx_data)
6  {
7      // Wait for the transmitter to finish any previous transmissions
8      while (UARTxSR & TXBF) { }
9
10     // Send the new data byte
11     UARTxTX = tx_data;
12 }
```

Listing 2: Example Function for UART Transmissions

12.8 Receive Procedure

To receive a byte of data via the UART, first initialize the UARTx peripheral by configuring the UARTxCR register and setting the baud rate with the UARTxBR register. Of note, the UART enable bit EN must be a 1 to activate the UART. The GPIO pin multiplexed with RXx must be set to secondary/peripheral mode in the corresponding PxSEL register.

Once done setting up the UART, wait for the UART receive complete interrupt flag RCIF to be 1. Then, read the received byte contained in the UARTxRX register. Reading UARTxRX automatically clears the RCIF, FEF, PEF, and OVF flags.

A simplified procedure for receiving a byte over the UART is shown in Listing 3.

```
1  /** Includes **/
2  #include <MemoryMap.h>
3
```

```
4  /** Function Definitions **/
5  char uartx_getc()
6  {
7      // Wait for the UART to receive a new character
8      while ((UARTxSR & RCIF) == 0) { }
9
10     // Return the new character that is in the receive buffer register
11     return UARTxRX;
12 }
```

Listing 3: Example Function for UART Receptions

12.9 `UART0` Boot Behavior

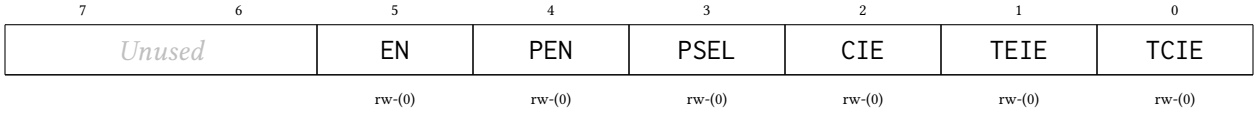
When booting in Forth interpreter mode, the `UART0` peripheral provides an interactive command processor with an external device, such as computer with a USB to UART device attached. When the MCU is powered up or reset and the `BOOT` pin is low, the Forth interpreter will launch over `UART0`, which can be used to program or debug the MCU.

Section 4 elaborates on the power up and reset behavior of the MCU.

12.10 Register Description

12.10.1 UARTCR Register

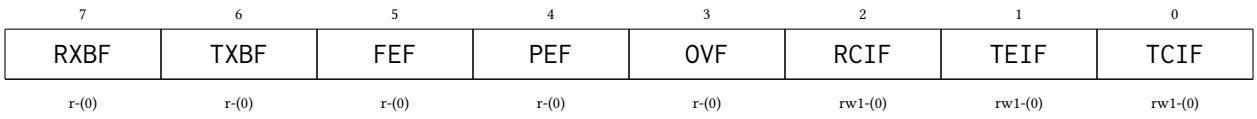
Register memory slot: 0, *offset:* 0 bytes, *size:* 1 byte
UART control register. Controls UART enable, parity configuration, and interrupt enable bits. When UCR.EN is disabled, the UART peripheral is held in reset state.



Bitfield	Range	Description
Unused	Bits 7-6	
EN	Bit 5	UART enable. When disabled, the UART peripheral is in reset state and the transmitter is idle. 0 UART disabled 1 UART enabled
PEN	Bit 4	Parity enable. Enables parity generation on transmit and parity checking on receive. 0 Parity disabled 1 Parity enabled
PSEL	Bit 3	Parity select. Selects even or odd parity when parity is enabled. 0 Even parity 1 Odd parity
CIE	Bit 2	Receive complete interrupt enable. Enables interrupt generation when a byte is successfully received. 0 RX complete interrupt disabled 1 RX complete interrupt enabled
TEIE	Bit 1	Transmit empty interrupt enable. Enables interrupt generation when the transmit buffer becomes empty and ready for new data. 0 TX empty interrupt disabled 1 TX empty interrupt enabled
TCIE	Bit 0	Transmit complete interrupt enable. Enables interrupt generation when transmission is complete and the transmitter is idle. 0 TX complete interrupt disabled 1 TX complete interrupt enabled

12.10.2 UARTSR Register

Register memory slot: 1, *offset:* 4 bytes, *size:* 1 byte
UART status register. Contains receiver and transmitter status flags and interrupt flags. Error flags (FEF, PEF, OVF, RCIF) are cleared by reading UARTxRX. Interrupt flags (RCIF, TEIF, TCIF) can also be cleared by writing a 1 to the respective bit.

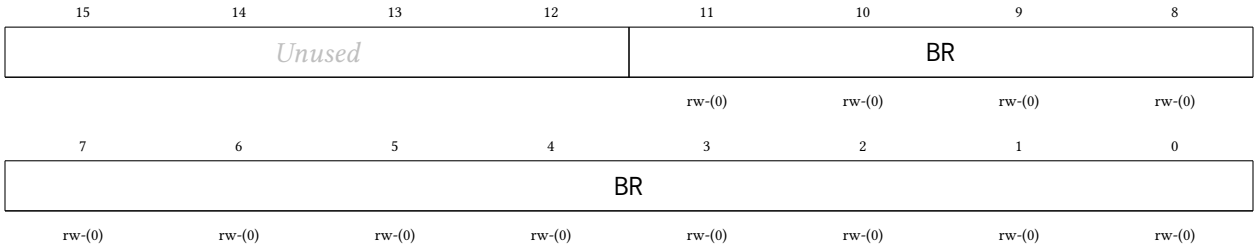


7	6	5	4	3	2	1	0
RXBF	TXBF	FEF	PEF	OVF	RCIF	TEIF	TCIF
r-(0)	r-(0)	r-(0)	r-(0)	r-(0)	rw1-(0)	rw1-(0)	rw1-(0)

Bitfield	Range	Description
RXBF	Bit 7	Receiver busy flag. Set while the UART receiver is actively receiving a byte. 0 Receiver idle 1 Reception in progress
TXBF	Bit 6	Transmitter busy flag. Set while the UART transmitter is actively transmitting a byte or has a pending transmission. 0 Transmitter idle 1 Transmission in progress
FEF	Bit 5	Framing error flag. Set when the stop bit is not detected (RX line not high at expected stop bit time). Cleared by reading UARTxRX. 0 No framing error 1 Framing error detected on last reception
PEF	Bit 4	Parity error flag. Set when received parity does not match expected parity (when parity is enabled). Cleared by reading UARTxRX. 0 No parity error 1 Parity error detected on last reception
OVF	Bit 3	Receive overflow flag. Set when a new byte is received before the previous byte in UARTxRX was read by the processor. Cleared by reading UARTxRX. 0 No receive data overrun 1 Receive data overflow detected
RCIF	Bit 2	Receive complete interrupt flag. Set when a byte is successfully received and placed in UARTxRX. Cleared by reading UARTxRX or writing a 1 to this bit. 0 No pending RX complete interrupt 1 RX complete interrupt pending
TEIF	Bit 1	Transmit empty interrupt flag. Set when the transmit buffer is empty and ready to accept new data. Write a 1 to this bit to clear it. 0 No pending TX empty interrupt 1 TX empty interrupt pending
TCIF	Bit 0	Transmit complete interrupt flag. Set when transmission is complete (all bits sent) and the transmitter is idle. Write a 1 to this bit to clear it. 0 No pending TX complete interrupt 1 TX complete interrupt pending

12.10.3 UARTBR Register

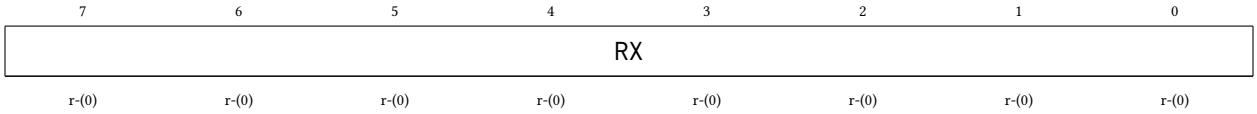
Register memory slot: 2, *offset:* 8 bytes, *size:* 2 bytes
UART baud rate register. Configures the baud rate divisor for the UART. The UART uses 16× oversampling.



Bitfield	Range	Description
Unused	Bits 15-12	
BR	Bits 11-0	Baud rate divisor. UART baud rate = SMCLK ÷ (16 × (BR + 1)). For example, with SMCLK = 48 MHz: BR = 25 gives 115,200 baud; BR = 51 gives 57,600 baud; BR = 103 gives 28,800 baud.

12.10.4 UARTRX Register

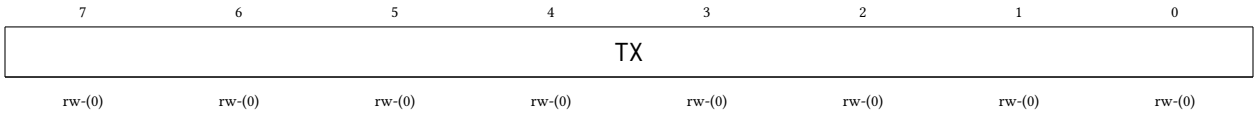
Register memory slot: 3, *offset:* 12 bytes, *size:* 1 byte
UART receive buffer register. Contains the last received byte. Reading this register clears the FEF, PEF, OVF, and RCIF flags in UARTxSR. The value is latched on the falling edge of the memory enable signal.



Bitfield	Range	Description
RX	Bits 7-0	Received data byte

12.10.5 UARTTX Register

Register memory slot: 4, *offset:* 16 bytes, *size:* 1 byte
UART transmit buffer register. Writing to this register loads the byte to transmit and initiates transmission. Do not write to this register while TXBF is set in UARTxSR.



Bitfield	Range	Description
TX	Bits 7-0	Transmit data byte

13 TIMERx Peripheral

The TIMERx peripheral is a 32-bit general purpose timer/counter unit with glitch-free clock source switching. Its main features are listed below:

- 32-bit timer values, from 0 to 4,294,967,295
- Memory mapped register to read and write current timer value
- Three output compare units, two of which can generate pulse width modulation (PWM) waveforms
- Two input capture units for precise timing measurements of external signals
- Four clock sources: SMCLK, MCLK, LFXT, HFXT
- Configurable clock divider (÷1 to ÷32,768)
- Glitch-free clock source multiplexer
- Six separate interrupts: 2 capture, 3 compare, 1 overflow
- Latched register reads for noise immunity

13.1 Timer Pins

The TIMERx peripheral has four pins: TxCMP0, TxCMP1, TxCAP0, and TxCAP1.

The TxCMP0 pin outputs the PWM waveform generated by the output compare unit 0. When the corresponding PxSEL[y] bit is a 1, the TIMERx peripheral seizes the DIR, OUT, and REN controls of the pin, forcing it to be in output mode. Similarly, TxCMP1 outputs the PWM waveform of output compare unit 1.

The TxCAP0 pin is an input that waits for a pin state change and then notifies the input capture unit 0, allowing precise timing of pin changes. The corresponding PxSEL[y] bit has no effect on this pin, since its secondary/peripheral function only ever acts as an input. Similarly, TxCAP1 is attached to input capture unit 1.

13.2 Clock Sources and Divider

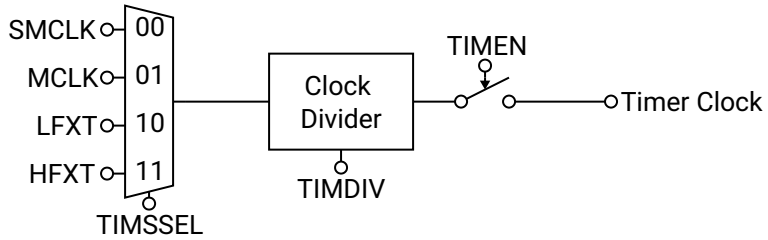


Figure 10: TIMERx Clock Diagram

The clock source for the TIMERx peripheral can be selected using TSSEL from four options: SMCLK, MCLK, LFXT (low frequency external clock), and HFXT (high frequency external clock). A glitch-free clock multiplexer built into the clock source selector prevents spurious clock transitions on the timer clock, allowing the user to switch clock sources freely even while the timer is enabled. However, switching clocks will cause the timer clock to be inactive for several clock cycles of the clock that is being switched from or to, whichever has a smaller frequency.

After the clock multiplexer, the timer clock frequency may optionally be reduced with a clock divider controlled by TDIV. The timer value register TIMxVAL increments by 1 on every rising edge of the divided timer clock when TIMERx is enabled with TEN.

A clock diagram for the TIMERx peripheral is shown in Figure 10.

13.3 Starting, Halting, Setting, and Reading the Timer

To start the timer, set the TEN bit to 1, whereupon the timer will begin incrementing by 1 with each rising edge of the divided timer clock. To halt the timer at its present value, clear the TEN bit to 0. The timer will retain its value until it is enabled again, its value is set manually, or the MCU is reset. If the timer is reenabled, it will begin counting from the timer value just before it was reenabled.

The timer value may be manually set at any time by writing the new 32-bit timer value to the TIMxVAL register. The timer value is updated immediately, regardless of if the timer is enabled or disabled.

The timer value may be read at any time by reading its value from the TIMxVAL register, regardless of if the timer is enabled or disabled. The TIMxVAL register is latched on the falling edge of the memory enable signal for stable reads during timer operation.

13.4 Overflow and Rollover Behavior

When the timer is enabled and its 32-bit value reaches its maximum of 4,294,967,295, the value will roll over back to zero on the next rising edge of the timer clock and the timer overflow interrupt flag TOVIF will be set. The timer will then continue incrementing as normal.

Optionally, the user may configure the timer to roll over before the 32-bit limit is reached by setting the new rollover value in the TIMxCMP2 register and setting the TCMP2RST bit to 1. In this mode, when the timer value becomes equivalent to the value of TIMxCMP2, the timer value will roll over back to zero on the next rising edge of the timer clock and then continue incrementing as normal, as depicted in Figure 11. This mechanism is used for setting the period of the PWM signals for the output compare units. Note: the timer rolls over back to zero when its value is *equal* to that of TIMxCMP2, but not if its value is greater than TIMxCMP2. This becomes important if the user manually sets the timer value above TIMxCMP2, or if the user sets TIMxCMP2 below the current timer value, whereupon the timer will continue counting all the way to the 32-bit maximum value.

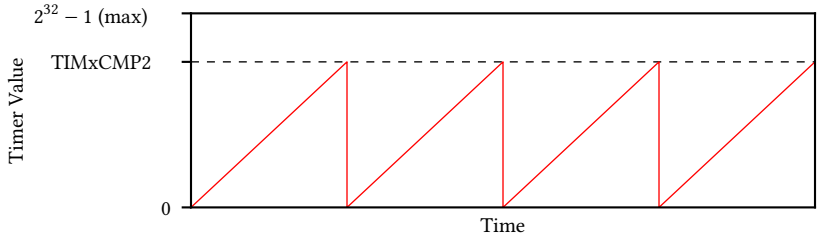


Figure 11: Timer Rollover Operation

13.5 Output Compare Units and Pulse Width Modulation

The TIMERx peripheral contains two output compare units that can be used to create two pulse width modulation (PWM) output signals on GPIO pins. Each output compare unit has an output compare register (TIMxCMP0 and TIMxCMP1), which is continually compared to the timer value TIMxVAL. The corresponding output compare pins (TxCMP0 and TxCMP1) toggle when either the timer value TIMxVAL equals the output

compare register TIMxCMPy, or the timer rolls over. The TCMP0IH and TCMP1IH bits control the initial state of the TxCMP0 and TxCMP1 pins when the timer value is less than the output compare register. Figure 12 provides an example of PWM operation using an output compare unit.

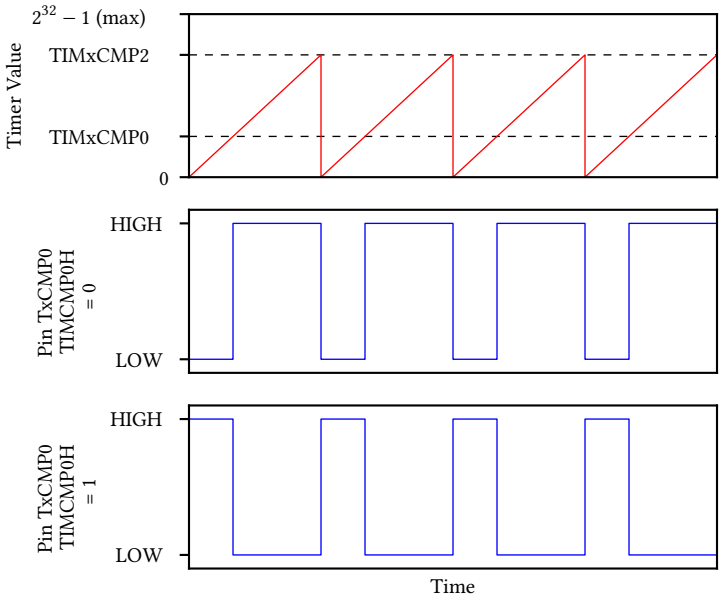


Figure 12: Timer Output Compare and PWM Operation

13.6 Input Capture Units

The TIMERx peripheral contains two input capture units which allow the precise timestamping of pin change events. When enabled with TCAPyEN, the input capture unit waits for the TxCAPy pin to toggle in the appropriate edge direction. The TCAPyFE bit selects either rising edge or falling edge sensitivity. When the pin toggles in the appropriate edge direction, the input capture unit stores the current timer value in the TIMxCAPy register and sets the TCAPyIF interrupt flag. The TCAPyIF flag must be cleared before the next input capture event by writing a 1 to it in the TIMxSR register.

Note: changing the value of TCAPyFE while TCAPyEN is enabled can inadvertently cause false input capture triggers. The user should not change TCAPyFE while TCAPyEN is enabled.

13.7 Flags and Interrupts

The TIMERx peripheral has two indicator bits and six interrupt flags in the timer status register TIMxSR. The TIMxSR register is latched on the falling edge of the memory enable signal for stable reads.

The two indicator bits TCMP0OUT and TCMP1OUT show the current value of the output compare pins TxCMP0 and TxCMP1 respectively, regardless of if the corresponding GPIO pins are set to primary/GPIO mode or secondary/peripheral mode.

The timer value overflow interrupt flag TOVIF is set when the 32-bit timer value overflows from $2^{32} - 1$ back to 0. Note that when TCMP2RST is enabled, the timer resets at the TIMxCMP2 value and does not

overflow to $2^{32} - 1$, so TOVIF will not be set in this mode. The overflow interrupt request is generated when both TOVIF and TOVIE are set. TOVIF is cleared by writing a 1 to it in the status register TIMxSR.

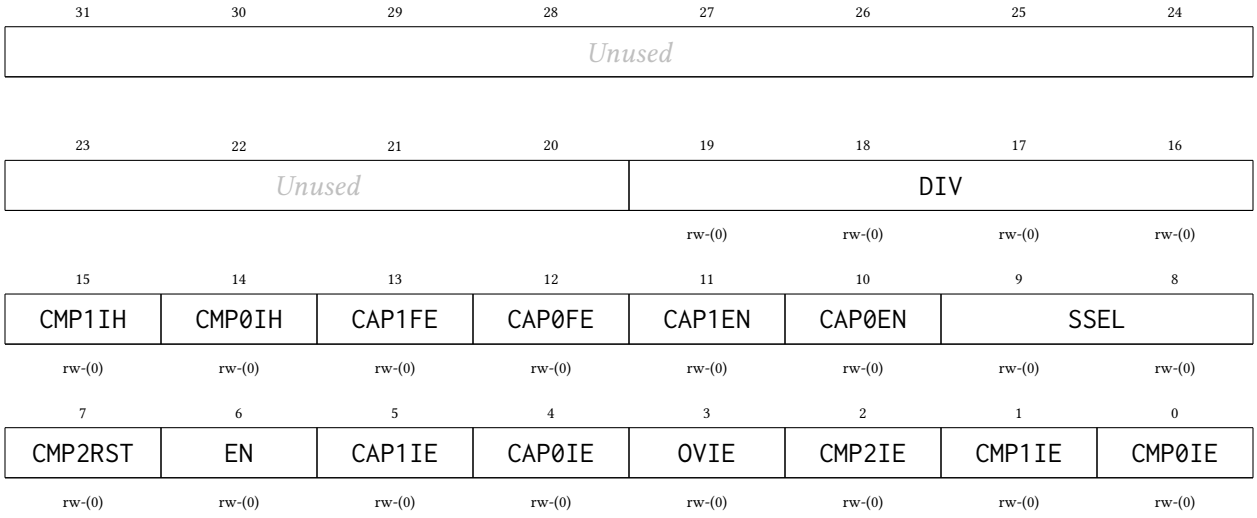
The output compare interrupt flags TCMP0IF, TCMP1IF, and TCMP2IF are set when the timer value equals the respective output compare value (TIMxCMP0, TIMxCMP1, and TIMxCMP2). Each compare interrupt request is generated when both the interrupt flag (TCMPyIF) and the corresponding interrupt enable bit (TCMP0IE, TCMP1IE, or TCMP2IE) are set. Each TCMPyIF flag is cleared by writing a 1 to it in the status register TIMxSR. Note that TCMP2IF is only set when TCMP2RST is enabled and the timer resets at the TIMxCMP2 value.

The input capture interrupt flags TCAP0IF and TCAP1IF are set when a new input capture event occurs on the TxCAP0 and TxCAP1 pins, respectively. The input capture register TIMxCAPy can be read as soon as TCAPyIF is set. Each capture interrupt request is generated when both the interrupt flag (TCAPyIF) and the corresponding interrupt enable bit (TCAP0IE or TCAP1IE) are set. Each TCAPyIF flag is cleared by writing a 1 to it in the status register TIMxSR.

13.8 Register Description

13.8.1 `TIMCR` Register

Register memory slot: 0, *offset:* 0 bytes, *size:* 4 bytes
Timer/Counter control register. Configures clock source, clock divider, capture/compare settings, and interrupt enables.



Bitfield	Range	Description
Unused	Bits 31-20	
`DIV`	Bits 19-16	Timer/Counter clock divider. Divides the clock source selected by SSEL to generate the timer clock for TIMxVAL increments. `0000` DIV_1: /1 (no division) `0001` DIV_2: /2 `0010` DIV_4: /4 `0011` DIV_8: /8 `0100` DIV_16: /16 `0101` DIV_32: /32 `0110` DIV_64: /64 `0111` DIV_128: /128 `1000` DIV_256: /256 `1001` DIV_512: /512 `1010` DIV_1024: /1,024 `1011` DIV_2048: /2,048 `1100` DIV_4096: /4,096 `1101` DIV_8192: /8,192 `1110` DIV_16384: /16,384 `1111` DIV_32768: /32,768
`CMP1IH`	Bit 15	Compare 1 initial PWM output level. Sets the TxCMP1 pin level when TIMxVAL < TIMxCMP1. `0` PWM output starts LOW

		1 PWM output starts HIGH
CMP0IH	Bit 14	Compare 0 initial PWM output level. Sets the TxCMP0 pin level when TIMxVAL < TIMxCMP0. 0 PWM output starts LOW 1 PWM output starts HIGH
CAP1FE	Bit 13	Capture 1 edge select. Selects which edge on TxCAP1 pin triggers a capture event. 0 Capture on rising edge 1 Capture on falling edge
CAP0FE	Bit 12	Capture 0 edge select. Selects which edge on TxCAP0 pin triggers a capture event. 0 Capture on rising edge 1 Capture on falling edge
CAP1EN	Bit 11	Capture 1 enable. Enables input capture on TxCAP1 pin. 0 Capture 1 disabled 1 Capture 1 enabled
CAP0EN	Bit 10	Capture 0 enable. Enables input capture on TxCAP0 pin. 0 Capture 0 disabled 1 Capture 0 enabled
SSEL	Bits 9-8	Timer/Counter clock source select. Glitch-free multiplexer prevents spurious transitions when switching sources. 00 SSEL_SMCLK: SMCLK 01 SSEL_MCLK: MCLK 10 SSEL_LFXT: LFXT (low frequency crystal) 11 SSEL_HFXT: HFXT (high frequency crystal)
CMP2RST	Bit 7	Timer/Counter reset on Compare 2 enable. Resets TIMxVAL to 0 when TIMxVAL equals TIMxCMP2. 0 Free-running mode (resets at 2^{32}-1) 1 Resets on TIMxVAL = TIMxCMP2
EN	Bit 6	Timer/Counter enable. When disabled, timer clock is gated off and TIMxVAL holds its value. 0 Timer disabled 1 Timer enabled
CAP1IE	Bit 5	Capture 1 interrupt enable. Enables interrupt when CAP1IF flag is set. 0 Interrupt disabled 1 Interrupt enabled
CAP0IE	Bit 4	Capture 0 interrupt enable. Enables interrupt when CAP0IF flag is set. 0 Interrupt disabled 1 Interrupt enabled
OVIE	Bit 3	Timer/Counter overflow interrupt enable. Enables interrupt when OVIF flag is set. 0 Interrupt disabled 1 Interrupt enabled
CMP2IE	Bit 2	Compare 2 interrupt enable. Enables interrupt when CMP2IF flag is set. 0 Interrupt disabled 1 Interrupt enabled
CMP1IE	Bit 1	Compare 1 interrupt enable. Enables interrupt when CMP1IF flag is set.

		0 Interrupt disabled
		1 Interrupt enabled
CMP0IE	Bit 0	Compare 0 interrupt enable. Enables interrupt when CMP0IF flag is set.
		0 Interrupt disabled
		1 Interrupt enabled

13.8.2 TIMSR Register

Register memory slot: 1, *offset:* 4 bytes, *size:* 1 byte
Timer/Counter status register. Contains current compare output levels and interrupt flags. The register is latched on the falling edge of the memory enable signal for stable reads.

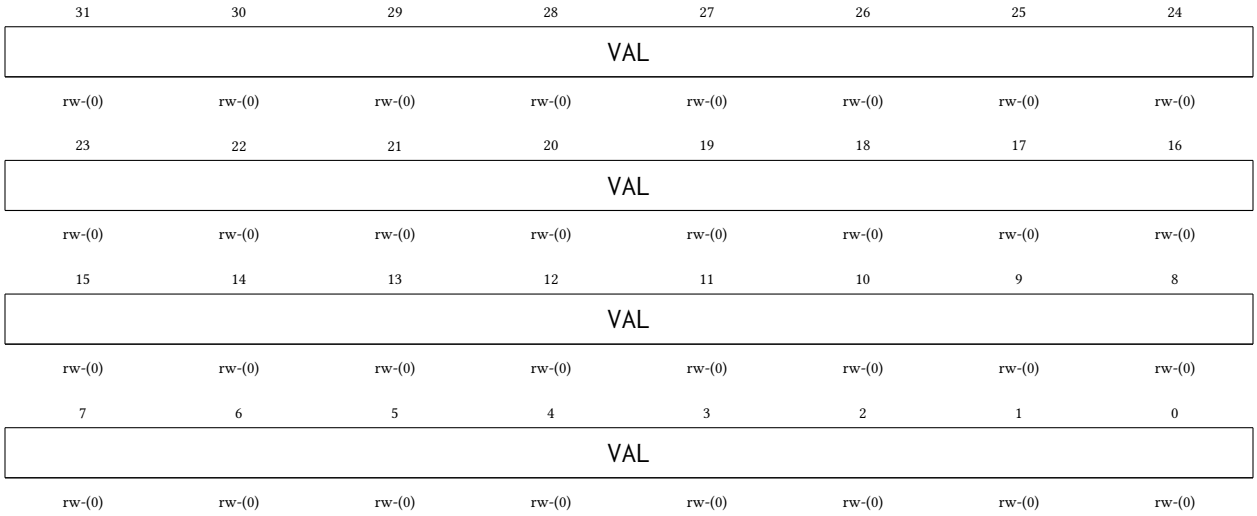
7	6	5	4	3	2	1	0
CMP1OUT	CMP0OUT	CAP1IF	CAP0IF	OVIF	CMP2IF	CMP1IF	CMP0IF
r-(0)	r-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)	rw1-(0)

Bitfield	Range	Description
CMP1OUT	Bit 7	Current value of the Compare 1 output pin TxCMP1. Reflects the actual PWM output level.
		0 TxCMP1 output LOW
		1 TxCMP1 output HIGH
CMP0OUT	Bit 6	Current value of the Compare 0 output pin TxCMP0. Reflects the actual PWM output level.
		0 TxCMP0 output LOW
		1 TxCMP0 output HIGH
CAP1IF	Bit 5	Capture 1 interrupt flag. Set when TxCAP1 pin triggers a capture event. Write 1 to clear.
		0 No pending capture 1 interrupt
		1 Capture 1 interrupt pending
CAP0IF	Bit 4	Capture 0 interrupt flag. Set when TxCAP0 pin triggers a capture event. Write 1 to clear.
		0 No pending capture 0 interrupt
		1 Capture 0 interrupt pending
OVIF	Bit 3	Timer/Counter overflow interrupt flag. Set when timer overflows from 2^{32}-1 to 0. Write 1 to clear.
		0 No pending overflow interrupt
		1 Overflow interrupt pending
CMP2IF	Bit 2	Compare 2 interrupt flag. Set when TIMxVAL equals TIMxCMP2. Write 1 to clear.
		0 No pending compare 2 interrupt
		1 Compare 2 interrupt pending
CMP1IF	Bit 1	Compare 1 interrupt flag. Set when TIMxVAL equals TIMxCMP1. Write 1 to clear.
		0 No pending compare 1 interrupt
		1 Compare 1 interrupt pending

CMP0IF	Bit 0	Compare 0 interrupt flag. Set when TIMxVAL equals TIMxCMP0. Write 1 to clear. 0 No pending compare 0 interrupt 1 Compare 0 interrupt pending

13.8.3 TIMVAL Register

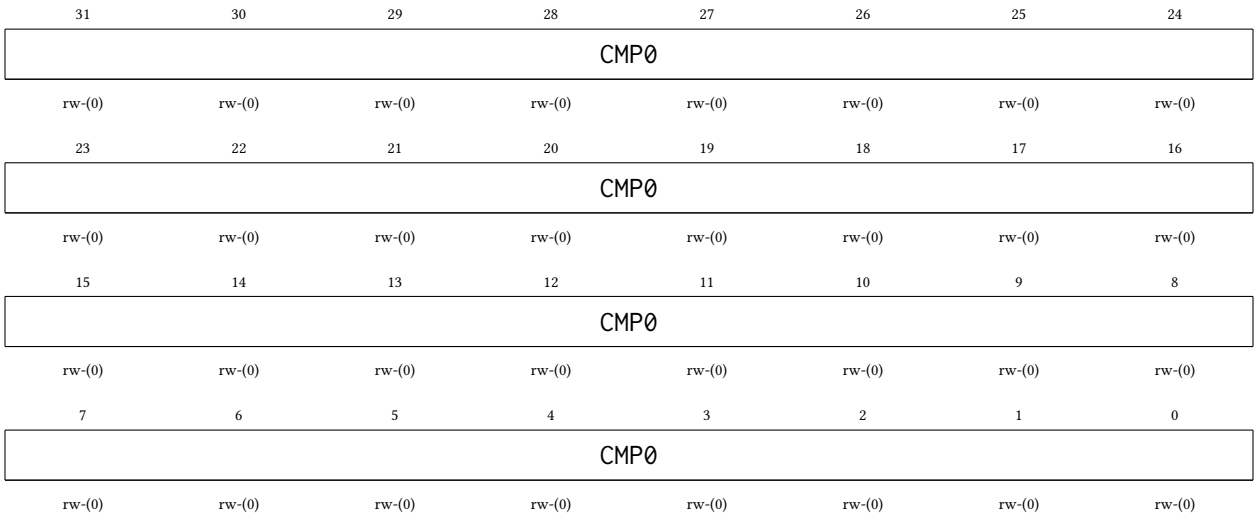
Register memory slot: 2, *offset:* 8 bytes, *size:* 4 bytes
Timer/Counter value register. Reads the current timer count. Writes immediately update the timer value regardless of enable state. The value is latched on the falling edge of the memory enable signal for stable reads.



Bitfield	Range	Description
VAL	Bits 31-0	Current timer count value

13.8.4 TIMCMP0 Register

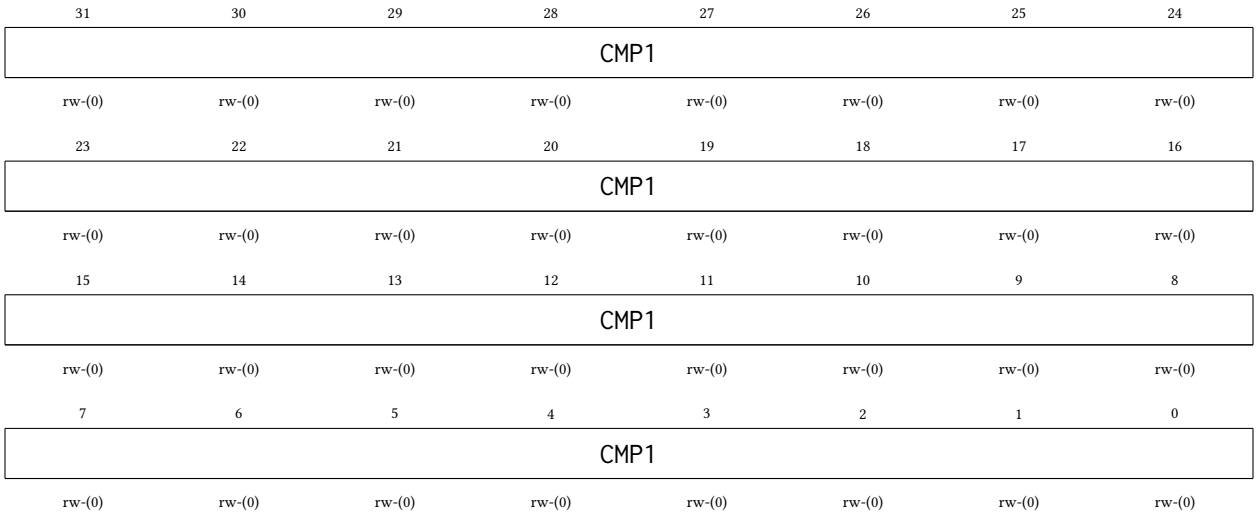
Register memory slot: 3, *offset:* 12 bytes, *size:* 4 bytes
Timer/Counter Compare 0 threshold register. When TIMxVAL equals this value, CMP0IF is set and the TxCMP0 PWM output toggles.



Bitfield	Range	Description
CMP0	Bits 31-0	Compare 0 threshold value

13.8.5 TIMCMP1 Register

Register memory slot: 4, *offset:* 16 bytes, *size:* 4 bytes
Timer/Counter Compare 1 threshold register. When TIMxVAL equals this value, CMP1IF is set and the TxCMP1 PWM output toggles.

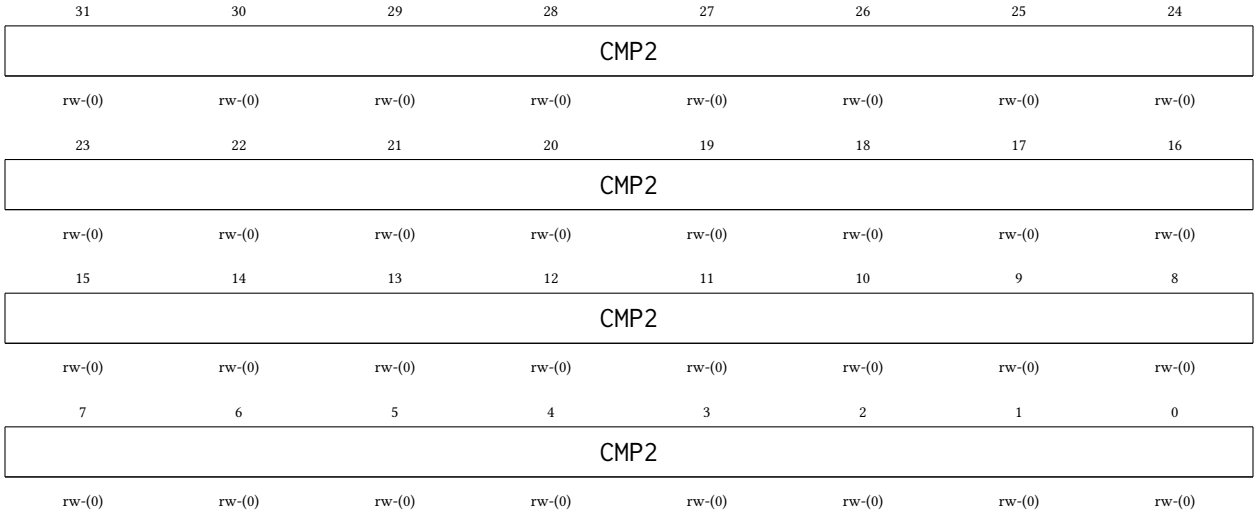


Bitfield	Range	Description
CMP1	Bits 31-0	Compare 1 threshold value

13.8.6 TIMCMP2 Register

Register memory slot: 5, *offset:* 20 bytes, *size:* 4 bytes

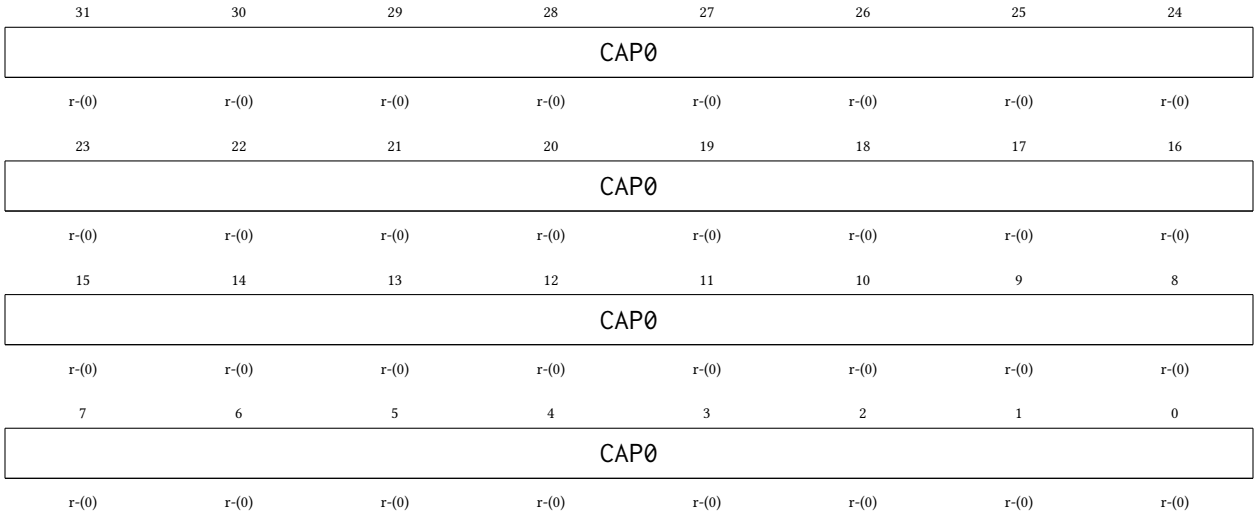
Timer/Counter Compare 2 threshold register. When TIMxVAL equals this value, CMP2IF is set. If CMP2RST is enabled, timer resets to 0 (PWM period control).



Bitfield	Range	Description
CMP2	Bits 31-0	Compare 2 threshold value (PWM period)

13.8.7 `TIMCAP0` Register

Register memory slot: 6, *offset:* 24 bytes, *size:* 4 bytes
Timer/Counter Capture 0 value register. Automatically latches TIMxVAL when TxCAP0 pin edge triggers a capture event. The captured value is latched on the falling edge of the memory enable signal for stable reads.

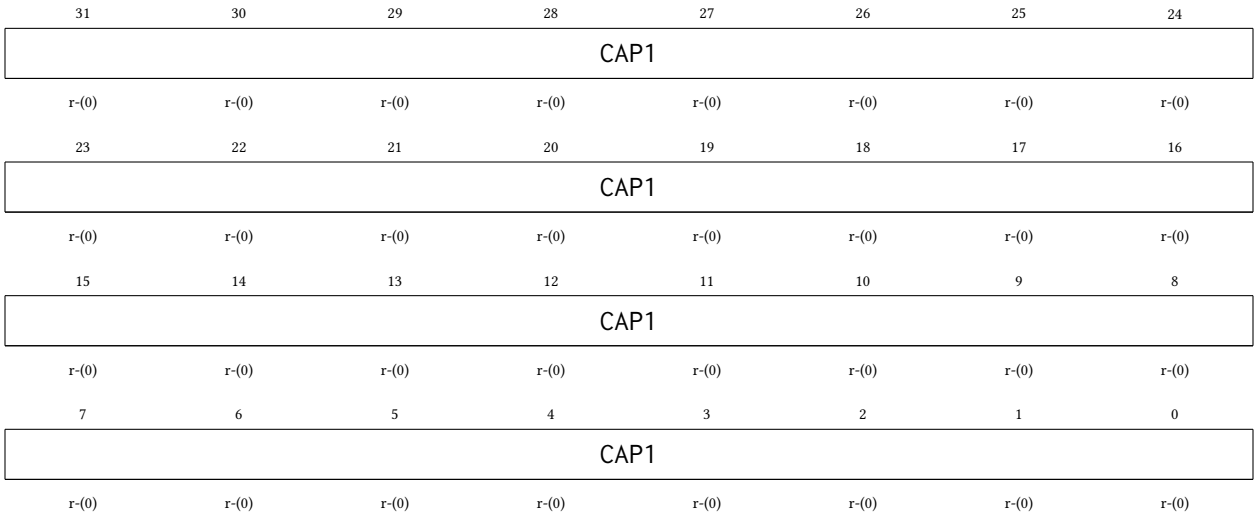


Bitfield	Range	Description

CAP0	Bits 31-0	Captured timer value from TxCAP0 event

13.8.8 TIMCAP1 Register

Register memory slot: 7, *offset:* 28 bytes, *size:* 4 bytes
Timer/Counter Capture 1 value register. Automatically latches TIMxVAL when TxCAP1 pin edge triggers a capture event. The captured value is latched on the falling edge of the memory enable signal for stable reads.



Bitfield	Range	Description
CAP1	Bits 31-0	Captured timer value from TxCAP1 event

14 SYSTEM Peripheral

The SYSTEM peripheral manages a potpourri of system-level blocks, including the MCU clocks, interrupt control, watchdog timer, and a CRC data verification module.

14.1 System Clocks and Clock Management

The MCU has four clock sources: HFXT, LFXT, DCO0, and DCO1. HFXT and LFXT are external clock sources, which must be provided to the MCU via the ClkHFXT and ClkLFXT pins, respectively. DCO0 and DCO1 are internal, programmable frequency clock sources provided by digitally controllable oscillator circuits.

The Myshkin MCU is designed and characterised for operation with a maximum clock frequency of 24 MHz. HFXT should therefore not exceed 24 MHz. LFXT is suggested to be exactly 32.768 kHz to provide an accurate method of measuring time. The frequencies of DCO0 and DCO1 are configured with the DCO0BIAS and DCO1BIAS registers; higher bias values produce higher frequencies.

Note on overclocking: It is possible to operate the Myshkin MCU beyond 24 MHz by supplying a higher-frequency signal on the ClkHFXT pin or by configuring the internal DCOs to produce frequencies above 24 MHz. As with any digital circuit operated beyond its rated frequency, doing so may produce setup-time violations, incorrect computational results, and unpredictable behaviour. Operation above 24 MHz is **not guaranteed** and is undertaken entirely at the user's risk.

Two clocks, the main clock (MCLK) and the submain clock (SMCLK), form the roots of the MCU clock tree, shown in Figure 13. MCLK and SMCLK can each select one of the four source clocks as their base and optionally divide the frequency further using CLKDIVCR. MCLK drives the CPU, while SMCLK drives most of the peripherals. Both clocks use glitch-free multiplexers to prevent spurious transitions when switching sources.

MCLK cannot be turned off, but SMCLK and individual source clocks can be independently disabled via SYSCLKCR. DCO0 and DCO1 are off by default and must be explicitly powered on by setting DCO0ON or DCO1ON. HFXT and LFXT are enabled by default and may be disabled with HFXTOFF and LFXTOFF. If a source clock is currently selected by MCLK or SMCLK, that source will be automatically kept enabled regardless of its disable bit in SYSCLKCR.

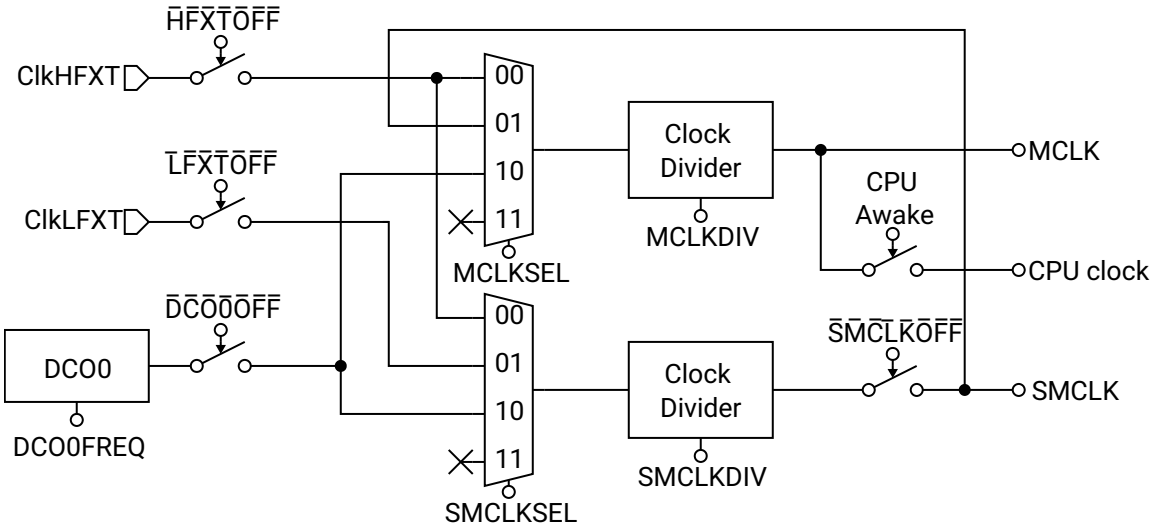


Figure 13: MCU Clock System

14.2 Power Saving Features

The MCU has several power-saving features.

The CPU is the most power-hungry part of the MCU, and its dynamic power consumption is proportional to the CPU clock frequency. Reducing the frequency of MCLK is the primary avenue for reducing total dynamic power consumption. This can be done by utilising the MCLK clock divider in `CLKDIVCR`, by switching MCLK to a lower-frequency source via `MCLKSEL`, or by reducing the frequency of the currently-selected source. The internal DCOs offer a wide range of programmable frequencies through their bias settings and can be used as a low-power clock source when HFXT is disabled.

Another method to save power is to turn off unused clocks. Remember, however, that the `ClkHFXT` pin is required to be a valid clock signal while the MCU boots. Failure to provide a valid clock may cause the chip to boot improperly, or not at all. For this reason, the start-HIGH pin(s) (`SHx`) have been provided. On the PCB, the start-HIGH pins should be connected to the enable input of the external clock source driving `ClkHFXT`, ensuring the clock is always present during reset. After boot, the user may switch MCLK to another source and then safely disable `ClkHFXT`.

The static (leakage) power can be reduced by powering off unused on-chip memory blocks with `BLOCKPWR`. The ROM and RAM blocks include power-gating circuitry that physically disconnects their supply rail. The ROM can be powered down by setting `ROMOFF`, and the two RAM blocks by setting `RAM0OFF` and `RAM1OFF`, respectively. Each powered-down block reduces leakage. Any access to a powered-down block will fail and produce undefined results. The entire contents of a RAM block are lost when it is powered down; data are undefined when it is powered back on until explicitly overwritten. Care must be taken to track the power state of each block to avoid undefined behaviour.

14.3 CPU Sleep Mode

Another way to conserve power is to run the CPU only when needed and switch it off when idle. Because the CPU accounts for the majority of dynamic power, duty-cycling its clock can yield significant savings. Peripherals can continue to operate while the CPU sleeps and may wake it when events occur. Long-term average CPU power is:

$$P_{\text{CPU}} = P_{\text{CPU, 100\%}} \times \frac{t_{\text{on}}}{t_{\text{on}} + t_{\text{off}}}$$

Sleep mode is activated with the custom `sleep` instruction, which gates off the CPU clock and reduces its dynamic power to zero. The CPU can only be woken by an interrupt or a system reset. When an interrupt fires, the CPU clock is re-enabled for the duration of that interrupt service routine (ISR). Once all pending interrupts have been serviced the CPU returns to sleep, unless one of the ISRs executed the custom `wake` instruction, in which case the CPU remains awake and returns to the point in the main program where it went to sleep.

A code example to put the CPU to sleep is included in Listing 4.

```
/** Includes **/
#include <MemoryMap.h>
#include <irq.h>

/** Interrupt Service Routine Declaration Macros **/
RVISR(IRQ_TIMER1_VECTOR, ISR_TIMER1)

/** Main Function **/
int main()
{
    // Set up TIMER1 here

    // Enable interrupts by unmasking them
    enable_all_interrupts();
```

```
15
16      // Go to sleep
17      cpu_sleep();
18
19      // Insert code to execute after ISR wakes the CPU
20
21      // Wait forever
22      while (1) {}
23  }
24
25  /** Interrupt Service Routines **/
26  void ISR_TIMER1()
27  {
28      // Do ISR processing here
29
30      // Wake the CPU
31      cpu_wake();
32  }
```

Listing 4: Example program to use CPU sleep mode

14.4 CRC Module

The CRC module enables the computation of a CRC16 checksum over an array of bytes. The checksum can be used to verify the integrity of data in a communication channel or in memory.

The module implements the CRC16-CDMA2000 standard with the following parameters:

- CRC width: 16 bits
- Polynomial: `0xC857`
- Initial value: `0xFFFF`
- No output XOR
- No input reflection
- No output reflection

To compute a CRC over a byte array, first write `0xFFFF` to `CRCSTATE` to initialise the accumulator. Then write each byte of the array sequentially to `CRCDATA`; each write updates the running CRC. After the last byte has been written, read `CRCSTATE` to obtain the final CRC16 value.

14.5 Interrupt Control

The `SYSTEM` peripheral manages global interrupt control. The `IRQCR` register contains the global interrupt enable bit `IRQGEN`, which must be set before any interrupt can fire. It also contains `IRQRECEN`, the interrupt recursion enable, which permits a running ISR to be preempted by a higher-priority interrupt when set.

Individual interrupt enables are controlled through the three 32-bit registers `IRQENL`, `IRQENM`, and `IRQENU`, providing enable bits for up to 96 interrupt vectors (83 are used on the Myshkin MCU). Each bit position corresponds directly to an IRQ vector number. An interrupt request will only trigger its ISR if both its individual enable bit and the global enable `IRQGEN` are set.

Interrupt priority is managed through the three 32-bit registers `IRQPRIL`, `IRQPRIM`, and `IRQPRIU`. Each bit determines the priority tier of the corresponding IRQ: a **0** places the IRQ in the high-priority tier, a **1** places it in the low-priority tier. Within each tier, lower IRQ vector numbers carry higher priority. A high-priority interrupt (bit = 0) will always preempt a low-priority ISR (bit = 1) that is currently executing,

regardless of vector number. Interrupt nesting to arbitrary depth is supported when IRQRECEN is set, limited only by available stack space.

14.6 Watchdog Timer

The watchdog timer improves system reliability by automatically taking corrective action if software fails to service the watchdog within a configurable timeout period. The watchdog is controlled by WDTCR, which is write-protected and must be unlocked via WDTPASS before any configuration change.

The watchdog uses a 24-bit up-counter that increments on every MCLK cycle when enabled via WDTEN. The timeout period is selected by WDTCDIV, which chooses which bit of the counter triggers the watchdog event. A watchdog event occurs on the 0-to-1 transition of the selected bit, giving a timeout of $2^{\text{WDTCDIV}+16}$ MCLK cycles. At 24 MHz the timeout ranges from approximately 2.73 ms (WDTCDIV = 0000, bit 16) to approximately 89.5 s (WDTCDIV = 1111, bit 31).

On a watchdog event, the response depends on the configuration:

- If WDTIE is set **and** the watchdog IRQ is enabled in IRQENL, the watchdog ISR executes. If WDTHWRST is also set, the system resets immediately after the ISR returns.
- If WDTIE is clear or the IRQ is disabled in IRQENL, the system resets immediately on timeout provided WDTHWRST is set.

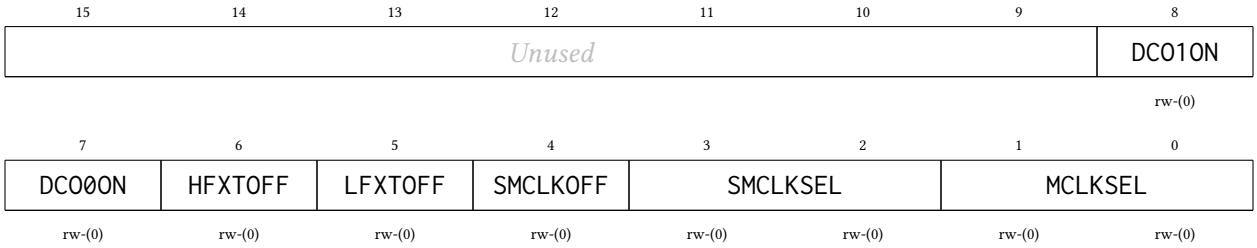
To prevent timeout, software must periodically write the clear password (0xD6F402BC) to WDTPASS, which resets the counter to zero. To modify WDTCR, first write the unlock password (0x3FB0AD1C) to WDTPASS; this opens a 64-MCLK-cycle window during which WDTCR is writable.

The WDTSR status register holds two sticky flags. WDTRF is set when the last system reset was caused by the watchdog and persists across resets; it is cleared by writing 1 to the bit. WDTIF is set when a watchdog timeout event occurs and is likewise cleared by writing 1. The current counter value is readable at any time from WDTVAL; the register reads as zero when the watchdog is disabled.

14.7 Register Description

14.7.1 SYSCLKCR Register

Register memory slot: 0, *offset:* 0 bytes, *size:* 2 bytes
System clock control register

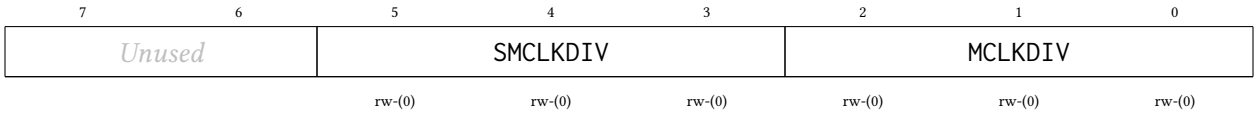


Bitfield	Range	Description
Unused	Bits 15-9	
DCO1ON	Bit 8	Digitally controlled oscillator 1 (DCO1) power enable. Set to power on DCO1. DCO1 is automatically kept on if it is currently selected as the source for MCLK or SMCLK, regardless of this bit. 0 DCO1 powered off 1 DCO1 powered on
DCO0ON	Bit 7	Digitally controlled oscillator 0 (DCO0) power enable. Set to power on DCO0. DCO0 is automatically kept on if it is currently selected as the source for MCLK or SMCLK, regardless of this bit. 0 DCO0 powered off 1 DCO0 powered on
HFXTOFF	Bit 6	High frequency external crystal clock disable. Cannot be disabled if it is currently selected as the source for MCLK or SMCLK. 0 HFXT enabled 1 HFXT disabled
LFXTOFF	Bit 5	Low frequency external crystal clock disable. Cannot be disabled if it is currently selected as the source for SMCLK. 0 LFXT enabled 1 LFXT disabled
SMCLKOFF	Bit 4	Submain clock disable. Globally and unconditionally disables SMCLK, gating all peripherals clocked from SMCLK. 0 SMCLK enabled 1 SMCLK disabled
SMCLKSEL	Bits 3-2	Submain clock source select 00 SMCLKSEL_HFXT: High Frequency Crystal Clock 01 SMCLKSEL_LFXT: Low Frequency Crystal Clock 10 SMCLKSEL_DCO0: Digitally Controlled Oscillator 0 11 SMCLKSEL_DCO1: Digitally Controlled Oscillator 1
MCLKSEL	Bits 1-0	Main clock source select (also CPU clock source select) 00 MCLKSEL_HFXT: High Frequency Crystal Clock 01 MCLKSEL_SMCLK: Submain Clock

10	MCLKSEL_DCO0: Digitally Controlled Oscillator 0
11	MCLKSEL_DCO1: Digitally Controlled Oscillator 1

14.7.2 CLKDIVCR Register

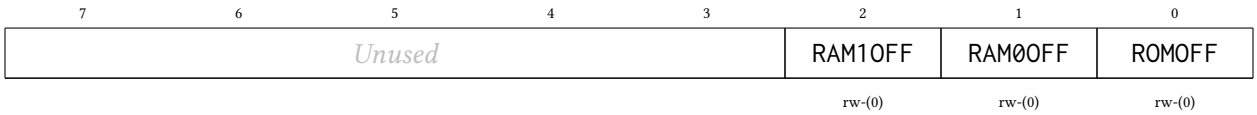
Register memory slot: 1, *offset:* 4 bytes, *size:* 1 byte
MCLK and SMCLK clock divider control register. Configures clock division for main and submain clocks using glitch-free multiplexers.



Bitfield	Range	Description
Unused	Bits 7-6	
SMCLKDIV	Bits 5-3	SMCLK clock division selection. Division is applied after clock source selection through glitch-free divider multiplexer. 000 SYSSMCLKDIV_1: /1 (no division) 001 SYSSMCLKDIV_2: /2 010 SYSSMCLKDIV_4: /4 011 SYSSMCLKDIV_8: /8 100 SYSSMCLKDIV_16: /16 101 SYSSMCLKDIV_32: /32 110 SYSSMCLKDIV_64: /64 111 SYSSMCLKDIV_128: /128
MCLKDIV	Bits 2-0	MCLK clock division selection. Division is applied after clock source selection through glitch-free divider multiplexer. 000 SYSMCLKDIV_1: /1 (no division) 001 SYSMCLKDIV_2: /2 010 SYSMCLKDIV_4: /4 011 SYSMCLKDIV_8: /8 100 SYSMCLKDIV_16: /16 101 SYSMCLKDIV_32: /32 110 SYSMCLKDIV_64: /64 111 SYSMCLKDIV_128: /128

14.7.3 BLOCKPWR Register

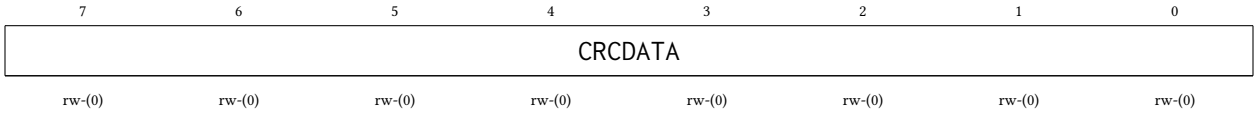
Register memory slot: 2, *offset:* 8 bytes, *size:* 1 byte
Block power control register. Controls power gating for on-chip memory blocks.



Bitfield	Range	Description
Unused	Bits 7-3	
RAM1OFF	Bit 2	RAM block 1 power control. When set, RAM block 1 is powered off. All data becomes undefined and the block no longer responds to memory access. Reduces static power consumption. 0 RAM block 1 powered on 1 RAM block 1 powered off
RAM0OFF	Bit 1	RAM block 0 power control. When set, RAM block 0 is powered off. All data becomes undefined and the block no longer responds to memory access. Reduces static power consumption. 0 RAM block 0 powered on 1 RAM block 0 powered off
ROMOFF	Bit 0	ROM power control. When set, boot ROM is powered off. ROM no longer responds to read access. Reduces static power consumption. 0 ROM powered on 1 ROM powered off

14.7.4 CRCDATA Register

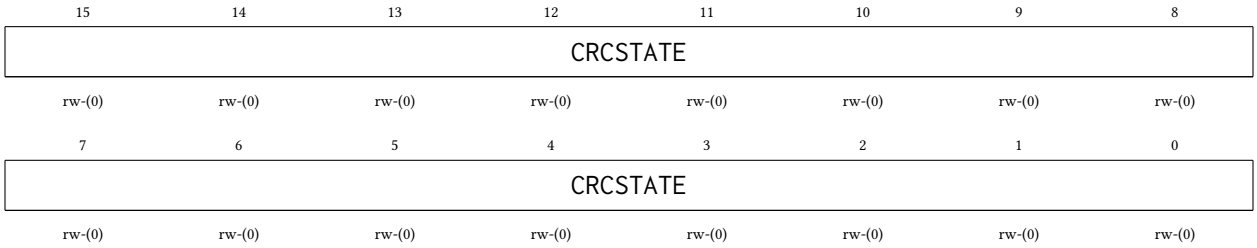
Register memory slot: 3, *offset:* 12 bytes, *size:* 1 byte
CRC input data register. Write the next byte of data to this register to update the CRC calculation. Uses CRC16-CDMA2000 polynomial 0xC857.



Bitfield	Range	Description
CRCDATA	Bits 7-0	CRC data input byte. Writing to this register feeds the byte into the CRC calculation and updates CRCSTATE.

14.7.5 CRCSTATE Register

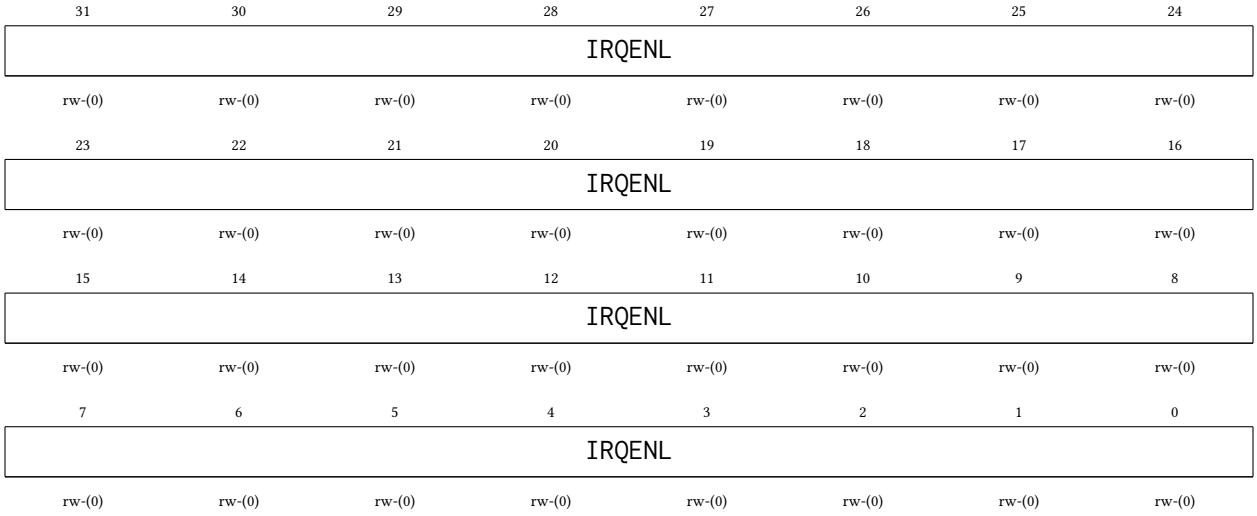
Register memory slot: 4, *offset:* 16 bytes, *size:* 2 bytes
CRC state register. Contains the current CRC16 calculation result. Write to this register to initialize or restart the CRC calculation. Read to obtain the computed CRC16 checksum.



Bitfield	Range	Description
CRCSTATE	Bits 15-0	CRC state value. Initialize to 0xFFFF before starting CRC calculation. Read after processing all data bytes to get final CRC16 checksum.

14.7.6 IRQENL Register

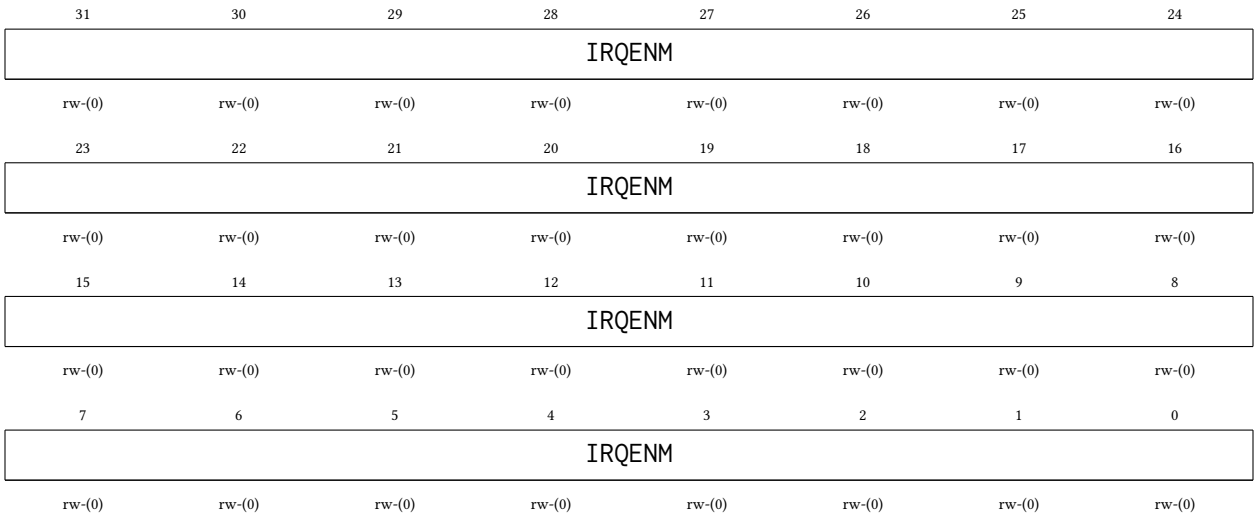
Register memory slot: 5, *offset:* 20 bytes, *size:* 4 bytes
IRQ enable register (lower 32 bits). Each bit enables the corresponding IRQ when set to 1. Bit position corresponds to IRQ vector number 0-31. IRQ also requires global enable in IRQCR.



Bitfield	Range	Description
IRQENL	Bits 31-0	IRQ enable bits for IRQ vectors 0-31. Set bit to 1 to enable corresponding IRQ.

14.7.7 IRQENM Register

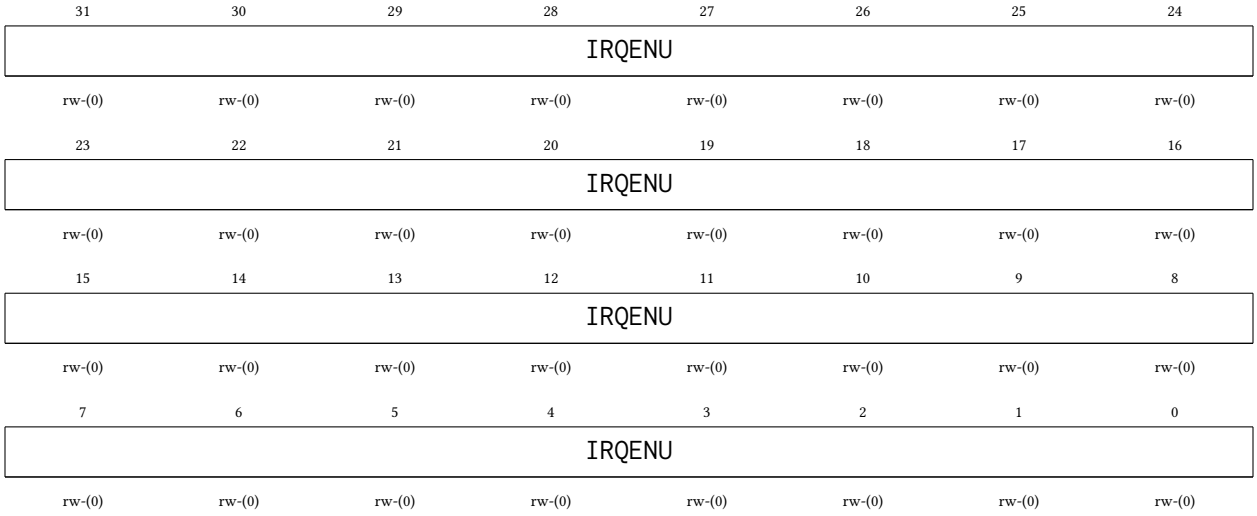
Register memory slot: 6, *offset:* 24 bytes, *size:* 4 bytes
IRQ enable register (middle 32 bits). Each bit enables the corresponding IRQ when set to 1. Bit position corresponds to IRQ vector number 32-63.



Bitfield	Range	Description
IRQENM	Bits 31-0	IRQ enable bits for IRQ vectors 32-63. Set bit to 1 to enable corresponding IRQ.

14.7.8 `IRQENU` Register

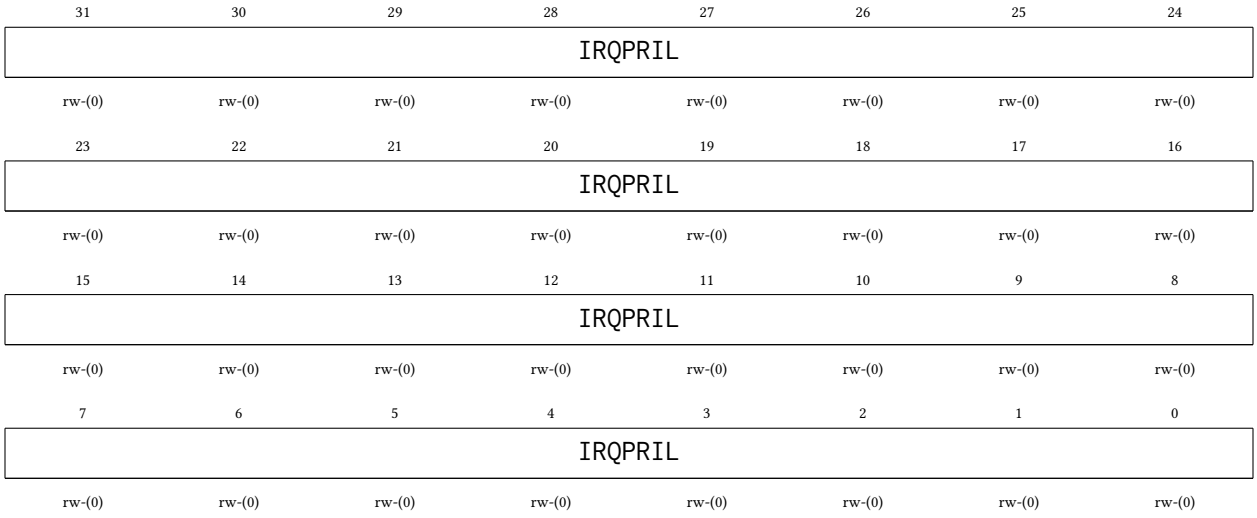
Register memory slot: 7, *offset:* 28 bytes, *size:* 4 bytes
IRQ enable register (upper bits). Each bit enables the corresponding IRQ when set to 1. Bit position corresponds to IRQ vector number 64 and above.



Bitfield	Range	Description
IRQENU	Bits 31-0	IRQ enable bits for IRQ vectors 64 and above. Set bit to 1 to enable corresponding IRQ.

14.7.9 `IRQPRIL` Register

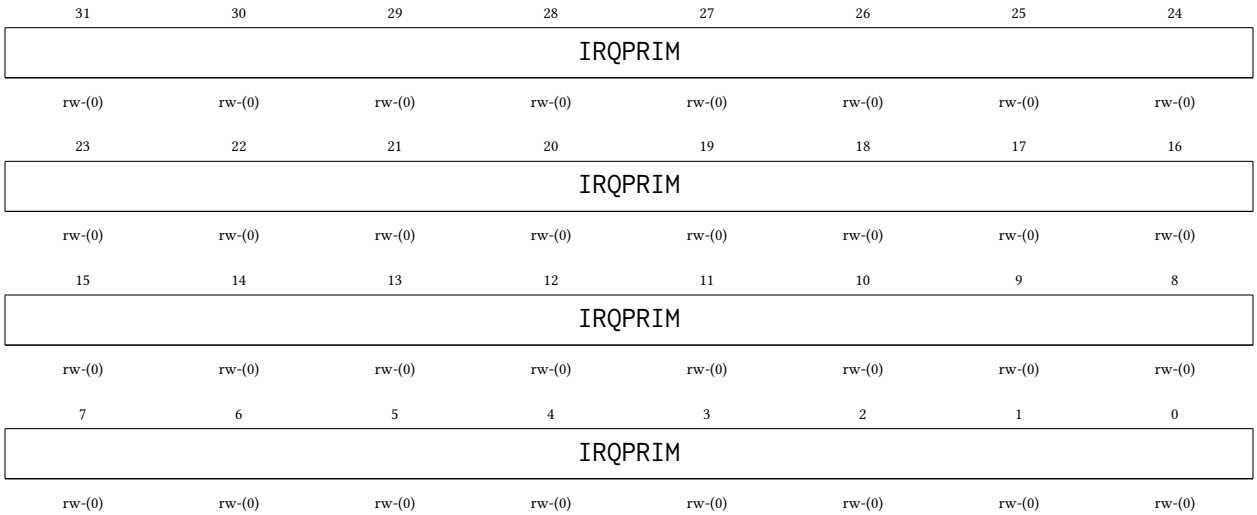
Register memory slot: 8, *offset:* 32 bytes, *size:* 4 bytes
IRQ priority register (lower 32 bits). Each bit sets priority tier for corresponding IRQ. Bit position corresponds to IRQ vector number 0-31. Priority determined first by tier (0=high, 1=low), then by IRQ number (lower number = higher priority within tier).



Bitfield	Range	Description
`IRQPRIL`	Bits 31-0	IRQ priority bits for IRQ vectors 0-31. Clear bit (0) for high priority tier, set bit (1) for low priority tier.

14.7.10 `IRQPRIM` Register

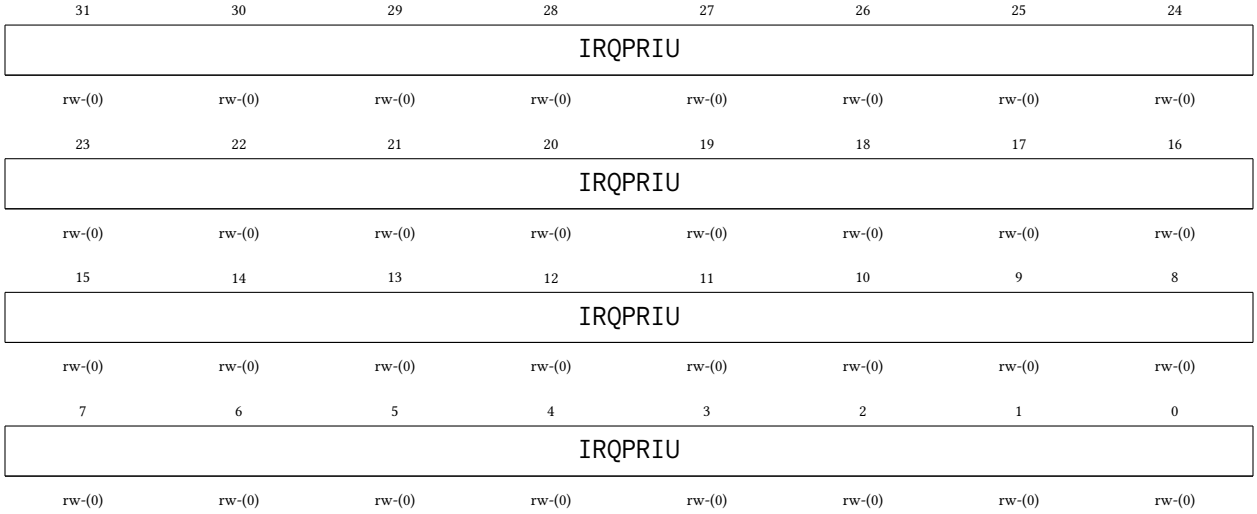
Register memory slot: 9, *offset:* 36 bytes, *size:* 4 bytes
IRQ priority register (middle 32 bits). Each bit sets priority tier for corresponding IRQ. Bit position corresponds to IRQ vector number 32-63.



Bitfield	Range	Description
IRQPRIM	Bits 31-0	IRQ priority bits for IRQ vectors 32-63. Clear bit (0) for high priority tier, set bit (1) for low priority tier.

14.7.11 IRQPRIU Register

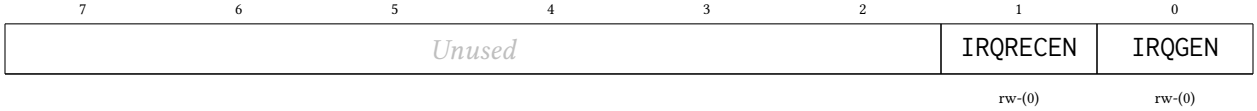
Register memory slot: 10, *offset:* 40 bytes, *size:* 4 bytes
IRQ priority register (upper bits). Each bit sets priority tier for corresponding IRQ. Bit position corresponds to IRQ vector number 64 and above.



Bitfield	Range	Description
IRQPRIU	Bits 31-0	IRQ priority bits for IRQ vectors 64 and above. Clear bit (0) for high priority tier, set bit (1) for low priority tier.

14.7.12 IRQCR Register

Register memory slot: 11, *offset:* 44 bytes, *size:* 1 byte
IRQ control register. Controls global IRQ enable and interrupt recursion settings.



Bitfield	Range	Description
Unused	Bits 7-2	
IRQRECEN	Bit 1	IRQ recursion enable. When set, allows interrupt service routines to be interrupted by higher priority IRQs. 0 IRQ recursion disabled 1 IRQ recursion enabled

IRQGEN	Bit 0	Global IRQ enable. Master enable for all interrupts. When cleared, all IRQs are disabled regardless of individual enable bits. 0 All IRQs globally disabled 1 IRQs enabled per IRQEN registers

14.7.13 WDTCR Register

Register memory slot: 12, *offset:* 48 bytes, *size:* 1 byte
Watchdog timer control register. This register is protected and requires password unlock via WDTPASS before writing. Configures watchdog operation mode and timeout period.

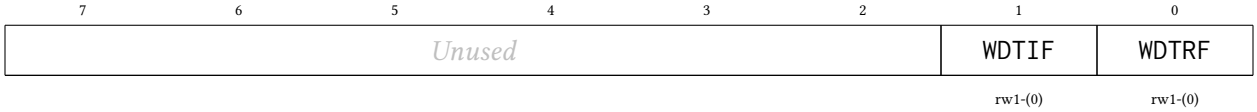
7	6	5	4	3	2	1	0
WDTEN	*Unused*	WDTCDIV				WDTIE	WDTHWRST
rw-(0)		rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)	rw-(0)

Bitfield	Range	Description
WDTEN	Bit 7	Watchdog timer enable. When set, watchdog counter increments on MCLK. Watchdog generates interrupt and/or reset when counter bit selected by WDTCDIV transitions from 0 to 1. Register is write-protected; unlock with WDTPASS first. 0 Watchdog disabled 1 Watchdog enabled
Unused	Bit 6	
WDTCDIV	Bits 5-2	Watchdog timer clock divider select. Selects which bit of the 24-bit counter triggers watchdog event. Event occurs when selected bit transitions from 0 to 1. Timeout period = 2^(WDTCDIV+16) MCLK cycles. 0000 SYSWDTCDIV_65536: Bit 16: 65,536 MCLK cycles 0001 SYSWDTCDIV_131072: Bit 17: 131,072 MCLK cycles 0010 SYSWDTCDIV_262144: Bit 18: 262,144 MCLK cycles 0011 SYSWDTCDIV_524288: Bit 19: 524,288 MCLK cycles 0100 SYSWDTCDIV_1048576: Bit 20: 1,048,576 MCLK cycles 0101 SYSWDTCDIV_2097152: Bit 21: 2,097,152 MCLK cycles 0110 SYSWDTCDIV_4194304: Bit 22: 4,194,304 MCLK cycles 0111 SYSWDTCDIV_8388608: Bit 23: 8,388,608 MCLK cycles 1000 SYSWDTCDIV_16777216: Bit 24: 16,777,216 MCLK cycles 1001 SYSWDTCDIV_33554432: Bit 25: 33,554,432 MCLK cycles 1010 SYSWDTCDIV_67108864: Bit 26: 67,108,864 MCLK cycles 1011 SYSWDTCDIV_134217728: Bit 27: 134,217,728 MCLK cycles 1100 SYSWDTCDIV_268435456: Bit 28: 268,435,456 MCLK cycles 1101 SYSWDTCDIV_536870912: Bit 29: 536,870,912 MCLK cycles 1110 SYSWDTCDIV_1073741824: Bit 30: 1,073,741,824 MCLK cycles 1111 SYSWDTCDIV_2147483648: Bit 31: 2,147,483,648 MCLK cycles

WDTIE	Bit 1	Watchdog timer interrupt enable. When set, watchdog timeout generates an interrupt before reset. If SYSWDTEN is also set and SYSIRQEN is enabled for watchdog IRQ, system executes watchdog ISR. Upon ISR return, system resets if SYSWDTHWRST is set. 0 Watchdog interrupt disabled 1 Watchdog interrupt enabled
WDTHWRST	Bit 0	Watchdog hardware reset enable. When set, watchdog timeout causes system reset. If WDTIE is set, reset occurs after interrupt service routine completes. If WDTIE is cleared or IRQ is not enabled, reset occurs immediately on timeout. 0 Watchdog reset disabled 1 Watchdog reset enabled

14.7.14 WDTSR Register

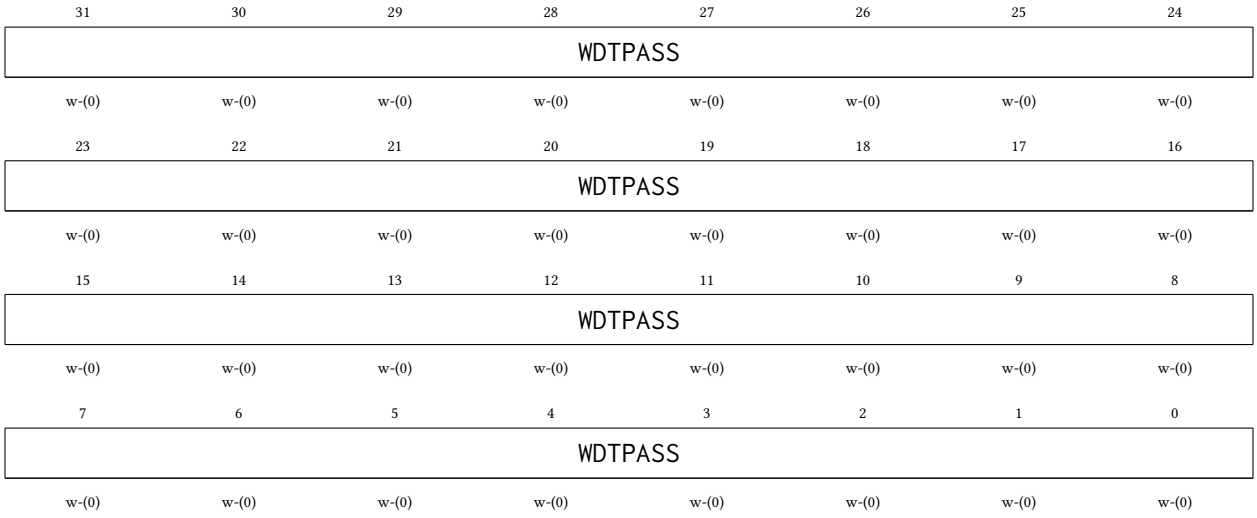
Register memory slot: 13, *offset:* 52 bytes, *size:* 1 byte
Watchdog timer status register. Contains flags indicating watchdog reset and interrupt events. Flags are cleared by writing 1 to the respective bit.



Bitfield	Range	Description
Unused	Bits 7-2	
WDTIF	Bit 1	Watchdog timer interrupt flag. Set when watchdog timeout occurs and WDTIE is enabled. Cleared by writing 1 to this bit. If not cleared before next timeout, indicates watchdog event occurred. 0 No watchdog interrupt pending 1 Watchdog interrupt occurred
WDTRF	Bit 0	Watchdog timer reset flag. Set when system reset was caused by watchdog timer. Persists across resets until cleared by software. Cleared by writing 1 to this bit. 0 Reset not caused by watchdog 1 Reset caused by watchdog

14.7.15 WDTPASS Register

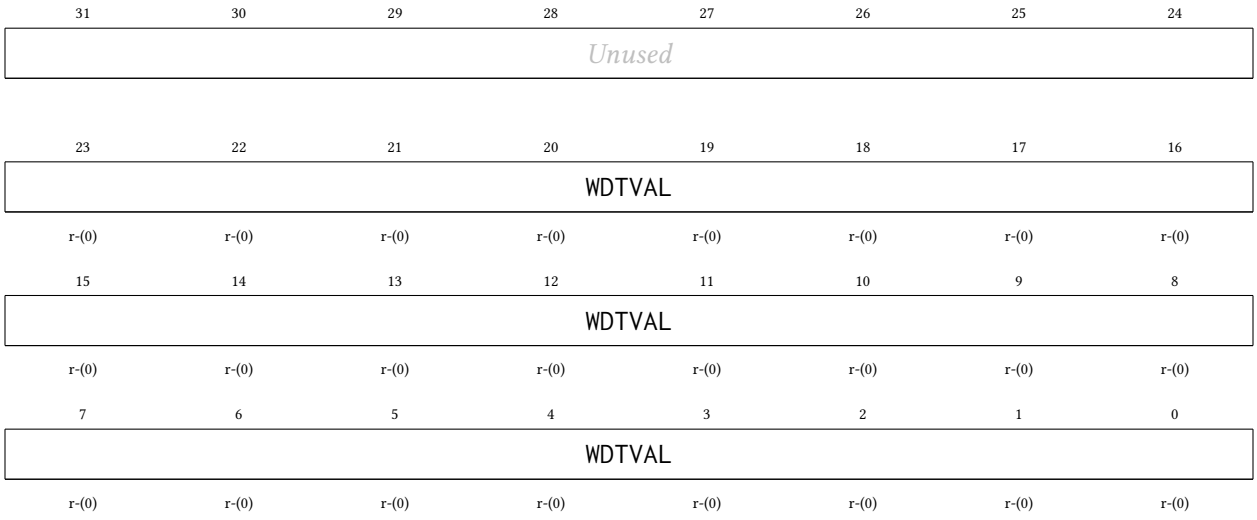
Register memory slot: 14, *offset:* 56 bytes, *size:* 4 bytes
Watchdog timer password register. Write-only register for two security functions: (1) Write 0x3FB0AD1C to unlock WDTCR for 64 MCLK cycles, enabling writes to watchdog configuration. (2) Write 0xD6F402BC to clear watchdog counter to 0, preventing timeout. Reading always returns 0.



Bitfield	Range	Description
WDTPASS	Bits 31-0	Watchdog password. Write 0x3FB0AD1C (unlock password) to enable WDTCR writes for 64 MCLK cycles. Write 0xD6F402BC (clear password) to reset watchdog counter to 0.

14.7.16 WDTVAL Register

Register memory slot: 15, *offset:* 60 bytes, *size:* 4 bytes
Watchdog timer value register. Read-only register containing current watchdog counter value. Counter increments on MCLK when watchdog is enabled. Returns 0 when watchdog is disabled.

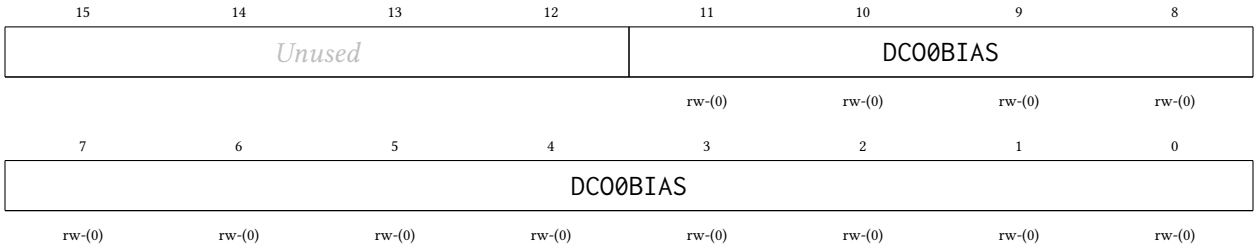


Bitfield	Range	Description
Unused	Bits 31-24	

WDTVAL	Bits 23-0	Watchdog counter value. 24-bit up-counter that increments on MCLK cycles. Watchdog event occurs when bit selected by WDTCDIV transitions from 0 to 1.

14.7.17 DCO0BIAS Register

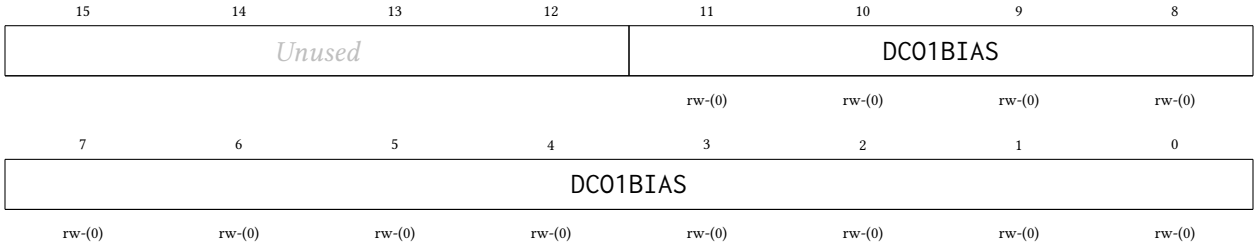
Register memory slot: 16, *offset:* 64 bytes, *size:* 2 bytes
Digitally controlled oscillator 0 bias register. Controls DCO0 output frequency through bias voltage adjustment. Higher bias values generally produce higher frequencies.



Bitfield	Range	Description
Unused	Bits 15-12	
DCO0BIAS	Bits 11-0	DCO0 bias adjustment value. 12-bit bias control for DCO0 frequency tuning. Default value loaded from constants on reset.

14.7.18 DCO1BIAS Register

Register memory slot: 17, *offset:* 68 bytes, *size:* 2 bytes
Digitally controlled oscillator 1 bias register. Controls DCO1 output frequency through bias voltage adjustment. Higher bias values generally produce higher frequencies.



Bitfield	Range	Description
Unused	Bits 15-12	
DCO1BIAS	Bits 11-0	DCO1 bias adjustment value. 12-bit bias control for DCO1 frequency tuning. Default value loaded from constants on reset.

15 NPU Peripheral

The NPU is a fixed-point neural network processing unit that accelerates the computation of a single fully-connected (dense) layer of a multilayer perceptron (MLP). The peripheral operates on data stored in a multiplexed 16 KiB SRAM shared with the CPU. The CPU is responsible for loading input data and weights into the SRAM, configuring the NPU, and reading the results after the computation is complete.

Data Formats

All NPU operands are fixed-point numbers, each stored in its own 32-bit SRAM word:

- **Inputs**: signed Q0.15 format (16 bits: 1 sign bit + 15 fractional bits). Stored in bits 15:0 of a 32-bit SRAM word; bits 31:16 are unused.
- **Weights**: signed Q3.15 format (19 bits: 1 sign bit + 3 integer bits + 15 fractional bits). Stored in bits 18:0 of a 32-bit SRAM word; bits 31:19 are unused. Bit 18 is the sign bit.
- **Outputs**: signed Q3.15 format (19 bits: 1 sign bit + 3 integer bits + 15 fractional bits). Stored in bits 18:0 of a 32-bit SRAM word; bits 31:19 are unused. Bit 18 is the sign bit.

Memory Layout

The input vector, weight matrix, and output vector must all reside within the same 16 KiB multiplexed SRAM, and all three start addresses must be 4-byte aligned. Each operand (input, weight, or output) occupies exactly one 32-bit SRAM word. See the register descriptions for NPUIVSAR, NPUWVSAR, and NPUOVSAR for address details.

If bias is enabled (NPUBEN = 1), the weight matrix contains NPUNN + 1 rows of NPUNI + 2 words each: one bias weight followed by NPUNI + 1 synaptic weights per output neuron. If bias is disabled, each row contains NPUNI + 1 synaptic weights.

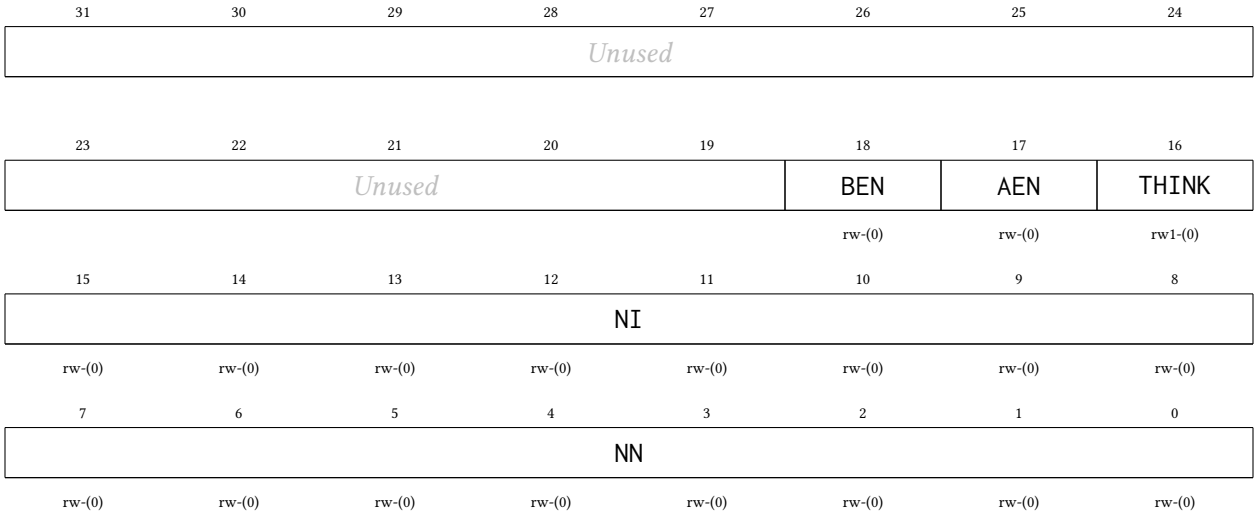
Operation

1. Load the input vector into the SRAM at the desired address and write that address to NPUIVSAR.
2. Load the weight matrix into the SRAM at the desired address and write that address to NPUWVSAR.
3. Write the desired output address to NPUOVSAR.
4. Configure NPUCR: set NPUNI to (number of inputs − 1), NPUNN to (number of output neurons − 1), and configure NPUBEN and NPUAEN as required.
5. Set NPUTHINK to 1 to start the computation.
6. Poll NPUTHINK until it reads 0, indicating that the computation is complete.
7. Read the output vector from the SRAM at the address stored in NPUOVA.

15.1 Register Description

15.1.1 NPUCR Register

Register memory slot: 0, *offset:* 0 bytes, *size:* 4 bytes
NPU control register

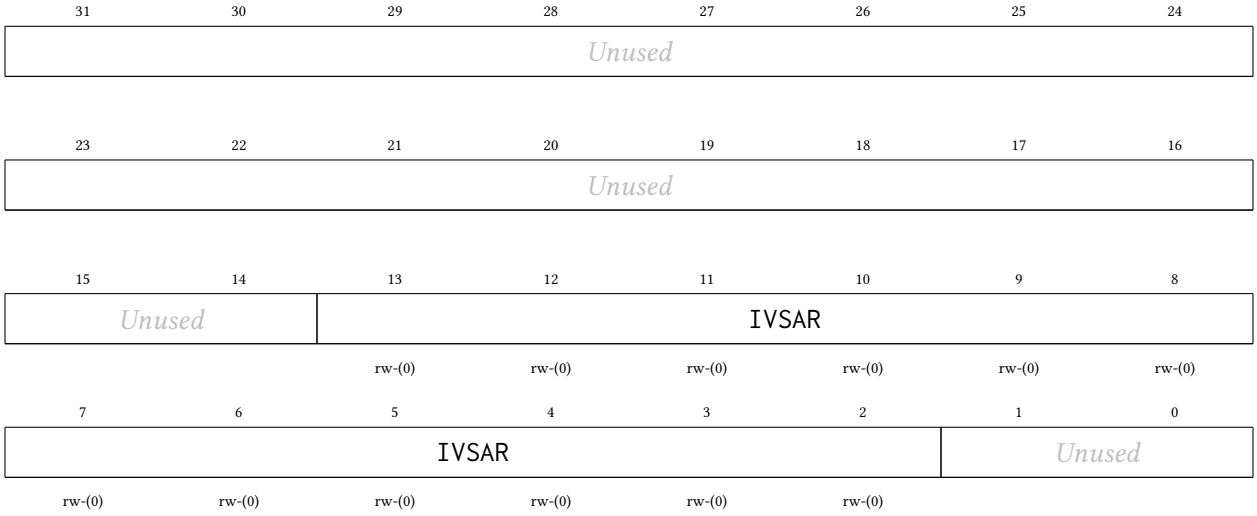


Bitfield	Range	Description
Unused	Bits 31-19	
BEN	Bit 18	Bias enable. When set, the last weight fetched for each output neuron is used as a bias term added to the accumulator before the activation function. 0 Disabled 1 Enabled
AEN	Bit 17	Activation function enable. When set, the logistic sigmoid approximation activation function is applied to the accumulator output. When cleared, the raw accumulator output is used (linear/identity). 0 Disabled (linear output) 1 Enabled (logistic sigmoid approximation)
THINK	Bit 16	NPU computation start and status bit. Write 1 to start the NPU. Self-clears when the computation is complete. Poll this bit to determine when results are ready. 0 Idle (computation complete or not started) 1 Running (write 1 to start)
NI	Bits 15-8	Number of inputs in the input vector minus 1. The actual number of inputs is NPUNI + 1.
NN	Bits 7-0	Number of output neurons minus 1. The actual number of outputs is NPUNN + 1.

15.1.2 NPUIVSAR Register

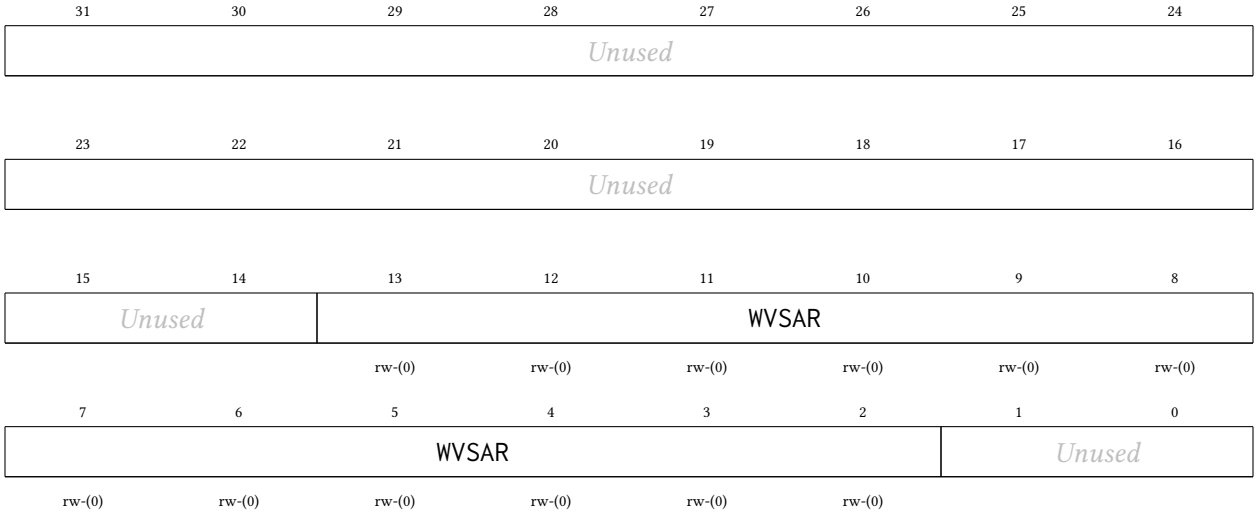
Register memory slot: 1, *offset:* 4 bytes, *size:* 4 bytes

Input vector start address (word address, i.e. byte address divided by 4). Each input is a signed Q0.15 (signed int16) value stored in bits 15:0 of its 32-bit SRAM word; bits 31:16 are ignored. The input at index 0 is at word address NPUIVSAR. The input at index 1 is at word address NPUIVSAR + 1. The rest of the inputs follow in consecutive word addresses. Bits 1:0 are unused because the start address must be aligned to a 32-bit word boundary.



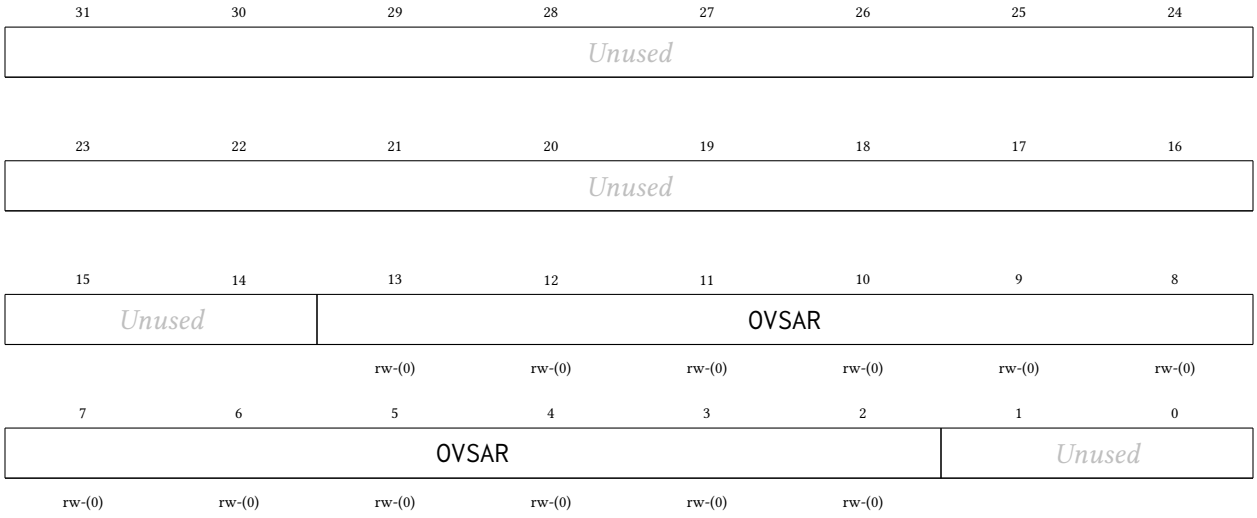
15.1.3 NPUWVSAR Register

Register memory slot: 2, *offset:* 8 bytes, *size:* 4 bytes
Synaptic weight matrix start address (word address, i.e. byte address divided by 4). Each weight is a signed Q3.15 (signed 19-bit) value stored in bits 18:0 of its 32-bit SRAM word; bits 31:19 are ignored. Bit 18 is the sign bit. Weights are stored row-major, one per 32-bit word, in the following order: for each output neuron (0 through NPUNN), if bias is enabled (NPUBEN = 1), the first word in the row is the bias weight (multiplied by an implicit input of 1.0), followed by NPUNI + 1 synaptic weights for inputs 0 through NPUNI. If bias is disabled, each row contains NPUNI + 1 synaptic weights only. Bits 1:0 are unused because the start address must be aligned to a 32-bit word boundary.



15.1.4 NPUOVSAR Register

Register memory slot: 3, *offset:* 12 bytes, *size:* 4 bytes
Output vector start address (word address, i.e. byte address divided by 4). Each output is a signed Q3.15 (signed 19-bit) value stored in bits 18:0 of its 32-bit SRAM word; bits 31:19 are unused. Bit 18 is the sign bit. The output at index 0 is written to word address NPUOVSAR. The output at index 1 is at word address NPUOVSAR + 1, and so on. Bits 1:0 are unused because the start address must be aligned to a 32-bit word boundary.



16 SARADC Peripheral

The SARADC is a 10-bit successive-approximation register (SAR) analog-to-digital converter based on a capacitive DAC (CAPDAC) architecture, optimized for compact layout and low energy consumption. A single trigger initiates one conversion cycle; continuous conversion mode (CONTMEAS) allows repeated conversions without further software intervention.

Resolution

The ADC produces a 10-bit unsigned result, available in bits 9:0 of the SARADC_DATA register. Bits 31:10 read as zero.

Conversion Timing

The conversion cycle is synchronized to the peripheral clock and consists of three sequential phases: a clear phase, a sample phase, and a successive-approximation conversion phase. The duration of the sample phase is controlled by the SAMPLESTEP field, which sets the initial value of a 4-bit countdown counter that runs during the sample phase. A value of 0 gives the minimum sample time. Refer to the ADC characterization data for the relationship between sample step value and the resulting sample duration.

Interrupts and Flags

Two status flags in SARADC_SR indicate conversion state:

- **DATAVALID** (SR bit 1): set by hardware when a new conversion result is available in SARADC_DATA. Clear by writing 1 to this bit. If this flag is already set when a new conversion completes, the overflow flag OVF is also set.
- **OVF** (SR bit 2): set when a conversion completes while DATAVALID is still set, indicating that the previous result was not read in time. Clear by writing 1 to this bit.

An interrupt is asserted when DATAVALID is set and the data-valid interrupt enable bit DATAIE is 1.

Operation

1. Set SAMPLESTEP to the desired sample phase duration (0 for minimum).
2. Optionally set CONTMEAS to 1 for continuous conversion mode, and/or set DATAIE to 1 to enable the data-valid interrupt.
3. Enable the ADC by setting EN to 1 in SARADC_CR. This starts the first conversion.
4. Wait for conversion to complete by polling BUSY until it reads 0, or wait for the IRQ if interrupts are enabled.
5. Read the 10-bit result from bits 9:0 of SARADC_DATA.
6. Clear DATAVALID by writing 1 to SR bit 1. In continuous mode, the next conversion begins automatically after the previous one completes.

Digital Test Ports

The SARADC_TPR (Test Port Register) provides a hardware debugging aid that routes internal ADC signals to two external digital test pins, DTP0 and DTP1. These pins can be probed with a logic analyser or oscilloscope to observe the internal state of the peripheral in real silicon, where internal nets are otherwise inaccessible. The test ports are only active when debug mode is enabled (DEBUG = 1); both outputs are driven to 0 otherwise.

The 4-bit field DTP0SEL (bits 3:0 of SARADC_TPR) selects which internal debug signal is driven onto DTP0; the 4-bit field DTP1SEL (bits 7:4) selects the signal for DTP1. Both fields use the same index space, so any two signals can be observed simultaneously. The available debug signal indices are listed in Table 45.

Index	Signal	Description
0	SAMPLESTEP[0]	Bit 0 (LSB) of the SAMPLESTEP field currently loaded from SARADC_CR.
1	SAMPLESTEP[1]	Bit 1 of SAMPLESTEP.
2	SAMPLESTEP[2]	Bit 2 of SAMPLESTEP.
3	SAMPLESTEP[3]	Bit 3 (MSB) of SAMPLESTEP.
4	EN	ADC enable bit (CR bit 5). High when the ADC is enabled.
5	DATAIE	Data-valid interrupt enable bit (CR bit 7).
6	CONTMEAS	Continuous measurement mode enable bit (CR bit 8).
7	ADC_ready	Registered copy of the ready strobe from the analog ADC core. Goes high on each cycle where new data is presented.
8	BUSY	Conversion busy flag (SR bit 0). High while the SAR conversion cycle is in progress.
9	OVF	Overflow interrupt flag (SR bit 2). High when a conversion completed while DATAVALID was already set.
10	DATAVALID	Data-valid flag (SR bit 1). High when a conversion result is ready to be read from SARADC_DATA.
11	clr_ovf	Internal clear-OVF pulse. Briefly high when software writes 1 to SR bit 2 to clear the overflow flag.
12	clr_valid	Internal clear-DATAVALID pulse. Briefly high when software writes 1 to SR bit 1 to clear the data-valid flag.
13–15	—	Tied low. Not connected to any internal signal.

Table 45: SARADC digital test port debug signal index map.

16.1 Register Description

16.1.1 SARADC_CR Register

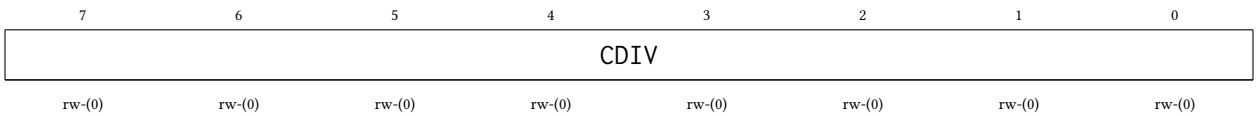
Register memory slot: 0, *offset:* 0 bytes, *size:* 2 bytes
SAR ADC control register



Bitfield	Range	Description
Unused	Bits 15-9	
CONTMEAS	Bit 8	Continuous measurement mode enable 0 Disabled 1 Enabled
DATAIE	Bit 7	Data valid interrupt enable 0 Interrupt disabled 1 Interrupt enabled
DEBUG	Bit 6	Debug mode 0 Normal operation 1 Debug mode
EN	Bit 5	ADC enable 0 Disabled 1 Enabled
SAMPLESTEP	Bits 4-1	Initial value of the 4-bit sample-phase countdown counter. 0 gives the minimum sample time; larger values extend the sample phase by the corresponding number of clock cycles.
RESET	Bit 0	ADC reset 0 Normal operation 1 Reset

16.1.2 SARADC_CDIV Register

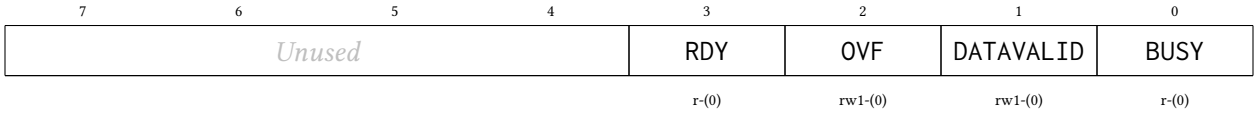
Register memory slot: 1, *offset:* 4 bytes, *size:* 1 byte
Reserved. This register slot is allocated in the memory map but is not implemented in hardware. Writes are ignored and reads return 0.



Bitfield	Range	Description
CDIV	Bits 7-0	Reserved. Not implemented; reads as 0.

16.1.3 SARADC_SR Register

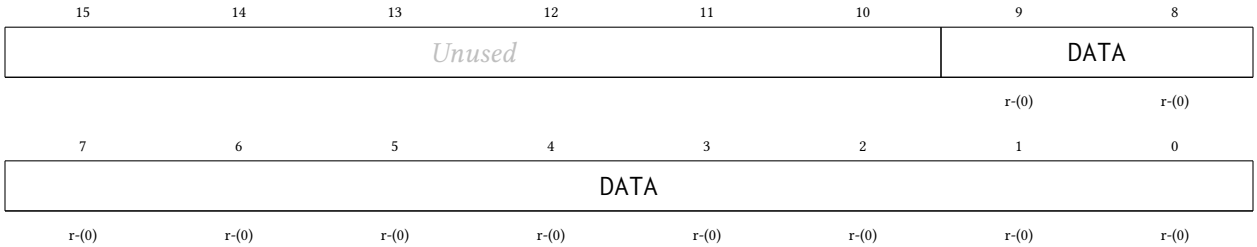
Register memory slot: 2, *offset:* 8 bytes, *size:* 1 byte
SAR ADC status register



Bitfield	Range	Description
Unused	Bits 7-4	
RDY	Bit 3	ADC ready 0 ADC not ready 1 ADC ready
OVF	Bit 2	ADC overflow interrupt flag 0 No overflow 1 Overflow occurred
DATAVALID	Bit 1	Data valid flag. Set by hardware when a conversion result is available. Write 1 to clear. 0 Data not valid 1 Data valid
BUSY	Bit 0	Conversion busy 0 Idle 1 Busy

16.1.4 SARADC_DATA Register

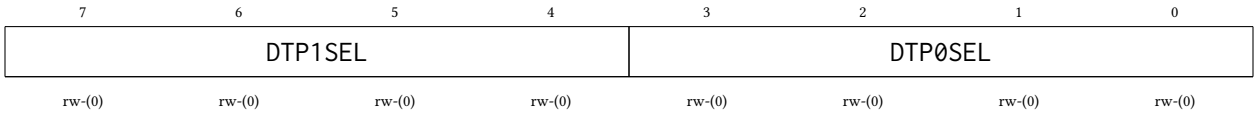
Register memory slot: 3, *offset:* 12 bytes, *size:* 2 bytes
SAR ADC data register. Holds the most recent 10-bit conversion result. Bits 15:10 read as zero.



Bitfield	Range	Description
Unused	Bits 15-10	
DATA	Bits 9-0	ADC conversion result (10-bit)

16.1.5 SARADC_TPR Register

Register memory slot: 4, *offset:* 16 bytes, *size:* 1 byte
SAR ADC test port register. Selects which internal debug signal is routed to each digital test port when debug mode (DEBUG) is enabled. DTP0SEL (bits 3:0) controls DTP0; DTP1SEL (bits 7:4) controls DTP1. Both outputs are driven to 0 when DEBUG = 0. See the Digital Test Ports section for the full signal index table.



Bitfield	Range	Description
DTP1SEL	Bits 7-4	Digital test port 1 select
DTP0SEL	Bits 3-0	Digital test port 0 select

17 AFE Peripheral

The AFE integrates a three-electrode potentiostat analog front end with a 12-bit dual-slope integrating ADC (DSADC) and a fully programmable bias generation block, forming a self-contained electrochemical sensing platform. The potentiostat controls the potential across a working and reference electrode while measuring the resulting cell current; the DSADC digitises the resulting voltage directly on chip. Together they are designed to support amperometric and voltammetric techniques such as chronoamperometry and cyclic voltammetry for chemical concentration measurements. All analog operating points—bias currents, DAC codes, integration time, gain—are configurable through memory-mapped registers, enabling complete software control without any external trim components.

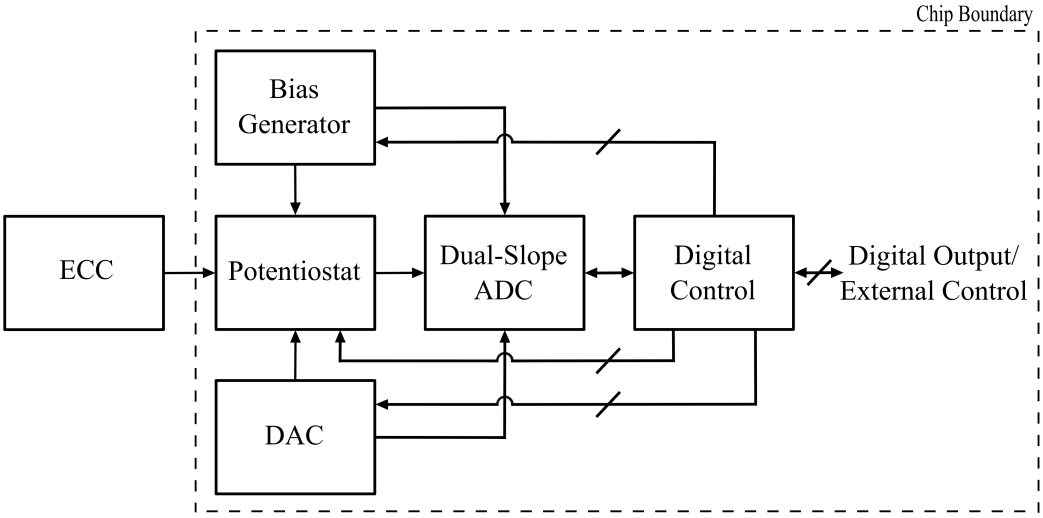


Figure 14: AFE system block diagram. The potentiostat front end (left) conditions the electrode current; the dual-slope ADC (centre) digitises the result; the digital peripheral (right) exposes both subsystems through memory-mapped registers.

Potentiostat Front End

The potentiostat implements the classical three-electrode electrochemical cell interface: a working electrode (WE), a reference electrode (RE), and a counter electrode (CE). A folded-cascode *control amplifier* (A_1) forms the outer feedback loop: its non-inverting input is driven by the *excitation voltage* programmed through BIAS_REV_POT (14-bit R-2R DAC), while its inverting input senses the reference electrode potential. A_1 drives the counter electrode such that $V_{\mathrm{RE}} = v_{\mathrm{pattern}}$, holding the WE–RE potential precisely at the programmed value. This feedback is stable across the full electrochemical bandwidth (≤ 300 Hz) with a phase margin exceeding 60°.

The cell current i_{cell} flowing through the counter electrode is duplicated by a *regulated cascode current mirror* (Q_1–Q_6) with a unity mirroring ratio, presenting high output impedance and $R^2 > 0.999$ current replication accuracy across process corners over a measured input current range of 1 nA to 20 μA.

A *transimpedance amplifier* (TIA, A_2) converts the mirrored current into a voltage

$$v_{\mathrm{out}} = v_{\mathrm{cm}} - i_{\mathrm{out}} \cdot R_f$$

where $v_{\mathrm{cm}} = V_{\mathrm{DD}}/2$ is the analog common-mode voltage (programmed via BIAS_DSADC_VCM, midscale = 8192). Referencing the output to v_{cm} centres the swing on the ADC input range so that both anodic and cathodic

currents can be represented without a negative supply. The feedback resistance R_f is set by `BIAS_TIA_G_POT` (17-bit thermometer code): the register controls 16 series 60 kΩ resistors plus one 1 MΩ resistor, each switchable in parallel by an analog switch. Asserting a bit closes the corresponding switch, shorting that resistor and *reducing* R_f. Larger codes therefore give lower gain and a wider measurable current range. The reset value 0x1FFFF closes all switches (minimum gain); writing 0x00000 gives maximum gain ($R_f \approx 2$ MΩ, current sensitivity ≈ 56 pA).

The TIA amplifier (A_2) is a two-stage Miller-compensated op-amp; its tail-current bias is trimmed by `BIAS_TC_POT` and its Miller lead-compensation resistor by `BIAS_LC_POT`.

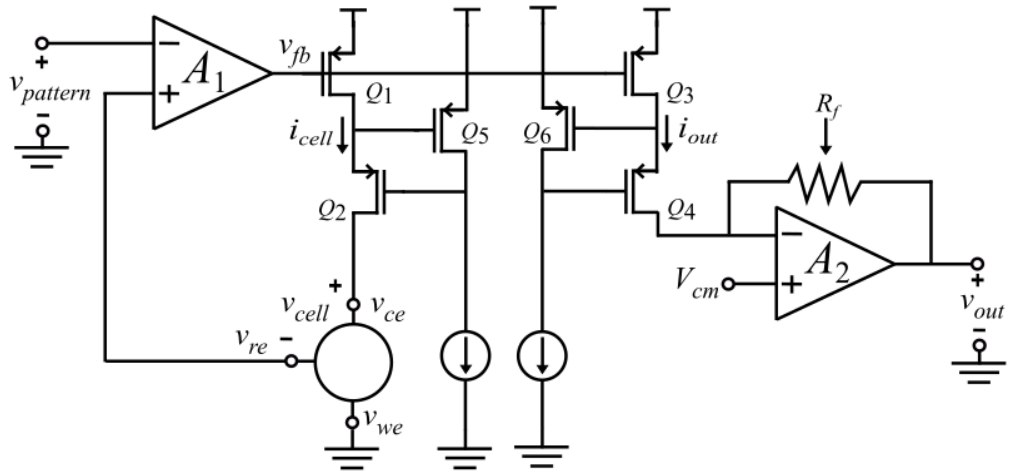


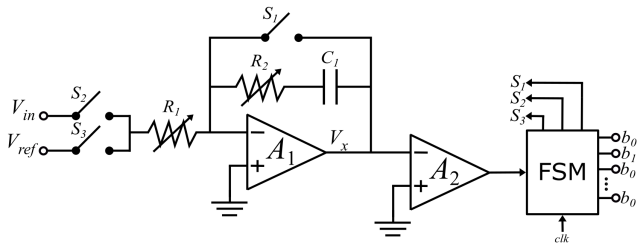
Figure 15: Potentiostat front-end schematic. A_1 (folded-cascode) is the control amplifier; Q_1–Q_6 form the regulated cascode current mirror with unity gain; A_2 with R_f implements the transimpedance amplifier. The excitation voltage v_{pattern} is generated by the `BIAS_REV_POT` DAC.

Dual-Slope ADC (DSADC)

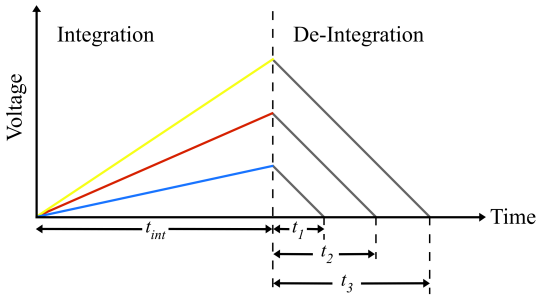
The DSADC is a 12-bit integrating converter. A conversion proceeds in two phases:

1. **Integration**: the TIA output (v_{out}) is integrated for a fixed interval $T_{\text{int}} = \texttt{RAMPNUM} \times T_{\text{clk}}$ clock cycles (12-bit field `AFE_CR`[23:12]). The internal counter increments from 0 to 4095.

2. **Deintegration**: the integrator input is switched to a reference of opposite polarity ($V_{\text{ref}} = V_{\text{DD}}$). The counter decrements from 4095 toward zero. When the comparator detects the integrator output crossing the common-mode voltage v_{cm}, conversion completes and the counter value is latched as the result. The deintegration time $T_{\text{deint}} = T_{\text{int}} \cdot V_{\text{in}}/V_{\text{ref}}$ is directly proportional to the input magnitude.



(a) Dual-slope ADC schematic with programmable R_1 (sets R_1C_1 time constant) and R_2 (lead-compensation at the integration-to-deintegration transition).

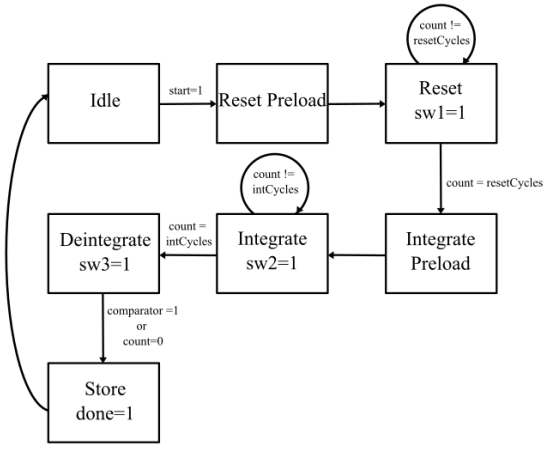


(b) Conversion waveform. The integrator ramps up during the fixed integration phase (T_{int}) then ramps back to v_{cm} during deintegration ($T_{\text{deint}} \propto V_{\text{in}}$).

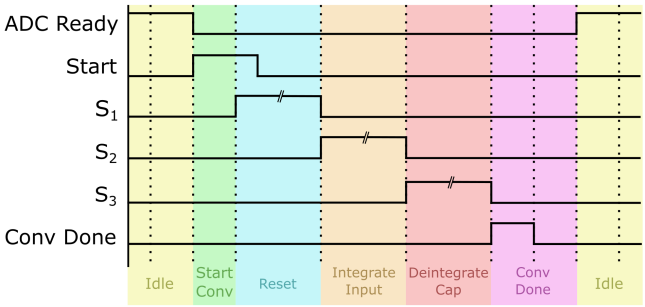
Figure 16: Dual-slope ADC circuit and conversion waveform.

The 12-bit result is available in AFE_ADC_VAL bits 11:0 after DATARDYIF asserts. The integrating architecture provides inherent low-frequency noise rejection and high linearity; measured deviations from the ideal transfer curve are ≤ 1 LSB across the full input range [0, 1.25 V]. The worst-case sampling rate at a 6 MHz ADC clock is 732 Hz (when $V_{\text{in}} = V_{\text{ref}}$, so $T_{\text{deint}} = T_{\text{int}}$).

A single conversion is triggered by writing any value to AFE_ADC_VAL while ADCEN = 1 and CONTMEAS = 0. In continuous mode (CONTMEAS = 1), the FSM restarts immediately after each conversion without software intervention.



(a) DSADC FSM state diagram. start = 1 (a write to AFE_ADC_VAL, or the automatic restart in continuous mode) leaves Idle; the Reset, Integrate and Deintegrate states each dwell until their cycle counter expires, and Deintegrate also exits on the comparator. Store latches the result and raises done, which sets DATARDYIF. The sw1–sw3 annotations are the integrator switch controls asserted in each state.



(b) DSADC control-signal timing across one conversion. ADC Ready deasserts for the whole conversion; the switch controls S_1, S_2 and S_3 select the reset, integration and deintegration phases in turn, and Conv Done pulses as the result is latched. The double slashes mark the phases whose length is set by a cycle count and is not to scale.

Figure 17: DSADC finite state machine and timing signals.

The DSADC input is selected by ADCEXTIN (AFE_CR bit 8): 1 = potentiostat output pad, 0 = external pin. The integrating amplifier (also a two-stage BTS op-amp) has its tail-current bias trimmed by BIAS_TC_DSADC

and its Miller resistor by BIAS_LC_DSADC. The DSADC integration resistor R_1 (which sets the R_1C_1 time constant) is trimmed by BIAS_RIN_DSADC, and the lead-compensation resistor R_2 (which mitigates the integrator overshoot at the integration-to-deintegration transition) is trimmed by BIAS_RFB_DSADC. Both on-chip resistors can vary ±20 % from nominal, and these 6-bit trim codes correct for process variation.

Bias Generation

Both the potentiostat and DSADC require supply-independent bias voltages and currents. The source is selected by BIAS_USEDAC (BIAS_CR bit 4):

- **Internal bias generator** (BIAS_USEDAC = 0): a wide-swing cascode generator produces the global bias voltages V_{bp}, V_{bpc}, V_{bnc}, V_{bn} distributed to all analog blocks. Enable with BIAS_EN (BIAS_CR bit 2). The overall current level is trimmed by BIAS_ADJ (6-bit; higher codes produce smaller currents). Enable output buffers with BIAS_BUFEN (BIAS_CR bit 3).
- **External bias DACs** (BIAS_USEDAC = 1): four 14-bit R-2R DACs (BIAS_DBP, BIAS_DBPC, BIAS_DBNC, BIAS_DBN) set the P, P-cascode, N-cascode, and N bias voltages independently, allowing fine-grained tuning or corner compensation without relying on the on-chip generator.

Two additional 14-bit R-2R DACs generate voltages that must be enabled via DACEN (AFE_CR bit 3) before a conversion is started:

- BIAS_REV_POT: generates the excitation voltage v_{pattern} applied to the non-inverting input of the control amplifier. Programming this register sets the WE–RE electrode potential. For cyclic voltammetry, firmware sweeps this register over time; for chronoamperometry, it is stepped to a fixed overpotential.
- BIAS_DSADC_VCM: generates the analog common-mode voltage $v_{\text{cm}} = V_{\text{DD}}/2$ applied to both A_1 and A_2. Midscale code 8192 produces $V_{\text{DD}}/2$, centring the TIA output and DSADC input swing.

Interrupts and Flags

Two status flags in AFE_SR indicate conversion state:

- **DATARDYIF** (SR bit 1): set by hardware when a DSADC conversion completes and the result is latched in AFE_ADC_VAL. Write 1 to clear. An IRQ is generated if DATARDYIE (AFE_CR bit 2) is also set.
- **OVFIF** (SR bit 2): set when a conversion completes while DATARDYIF is still asserted, indicating the previous result was overwritten before it was read. Write 1 to clear.

Operation

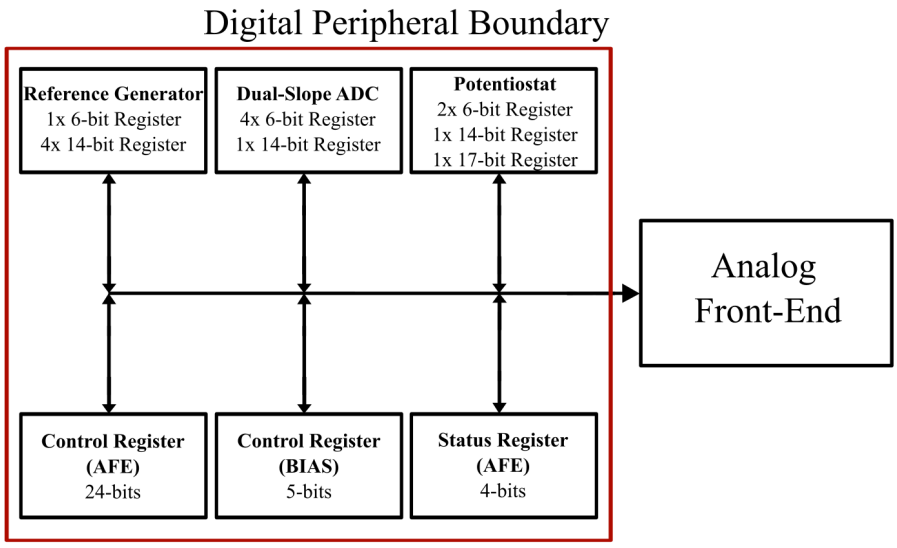


Figure 18: AFE digital peripheral block. Inside the peripheral boundary the memory-mapped register file splits into the reference-generator, DSADC and potentiostat trim groups (top) and the AFE_CR, BIAS_CR and AFE_SR registers (bottom); every group drives the analog front end over the same control bus. Widths are the implemented register widths, not the memory-map word size.

1. Enable and configure bias generation: set BIAS_EN and BIAS_BUFEN in BIAS_CR. Write BIAS_ADJ (nominal code is chip-dependent; see characterization data), or configure the individual bias DACs and set BIAS_USEDAC = 1.
2. Write the TIA and DSADC amplifier bias trim codes: BIAS_TC_POT, BIAS_LC_POT, BIAS_TC_DSADC, BIAS_LC_DSADC.
3. Write the DSADC resistor trim codes: BIAS_RIN_DSADC (integration resistor R_1) and BIAS_RFB_DSADC (lead-compensation resistor R_2).
4. Set the TIA gain by writing BIAS_TIA_G_POT (0x1FFFF = minimum gain / maximum current range; 0x00000 = maximum gain / minimum current range).
5. Program the electrode potential: write BIAS_REV_POT to set v_{pattern} (i.e. the WE−RE voltage).
6. Program the analog common mode: write BIAS_DSADC_VCM = 8192 for $V_{\text{DD}}/2$.
7. Enable the DACs by setting DACEN = 1 in AFE_CR.
8. Set the integration length: write RAMPNUM (AFE_CR bits 23:12). The integration time is RAMPNUM $\times T_{\text{clk}}$.
9. Enable the AFE clock and ADC: set EN = 1 and ADCEN = 1 in AFE_CR.
10. Trigger a conversion by writing any value to AFE_ADC_VAL (single-shot), or set CONTMEAS = 1 for continuous mode.
11. Wait for DATARDYIF (poll AFE_SR bit 1 or wait for the IRQ).

12. Read the 12-bit result from AFE_ADC_VAL bits 11:0. Clear DATARDYIF by writing 1 to AFE_SR bit 1.

For **cyclic voltammetry**, repeat steps 10–12 while periodically updating BIAS_REV_POT to sweep the electrode potential over the desired voltage window. For **chronoamperometry**, hold BIAS_REV_POT at the step potential and use continuous mode (CONTMEAS = 1) to sample the decaying cell current as a function of time.

Digital Test Ports

The AFE provides four digital test ports (DTP0–DTP3), each driven by a selected signal from a 32-entry internal debug vector. The 5-bit fields DTP0SEL through DTP3SEL in AFE_TPR select the signal for each port. All four ports are always configured as outputs. The available signals are listed in Table 52.

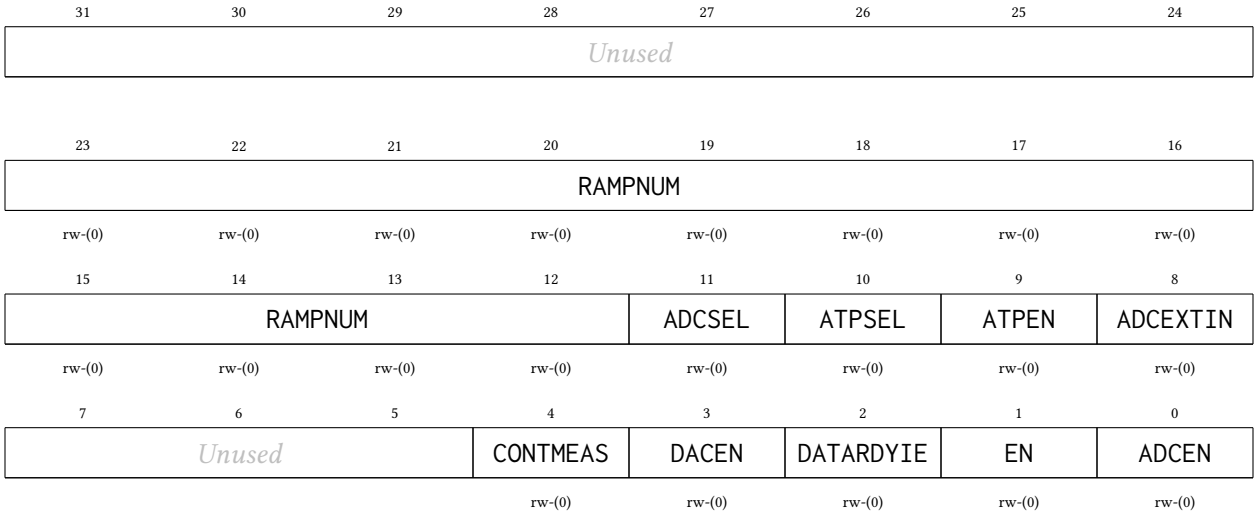
Index	Signal	Description
0	ADCACTIVE	High while the DSADC FSM is running a conversion (AFE_SR bit 0).
1	DATARDYIF	Data-ready interrupt flag (AFE_SR bit 1). High when a conversion result is available.
2	OVFIF	Overflow flag (AFE_SR bit 2). High when a conversion completed before the previous result was read.
3	adc_done	Internal done pulse from the AFE FSM. High for one clock cycle when a conversion completes.
4	adc_clk	DSADC conversion clock output, derived from the AFE clock.
5	adc_switch[0]	DSADC switch control bit 0 from the AFE FSM.
6	adc_switch[1]	DSADC switch control bit 1.
7	adc_switch[2]	DSADC switch control bit 2.
8	adc_start	ADC start signal. High continuously in continuous mode; single pulse when AFE_ADC_VAL is written in single-shot mode.
9–19	—	Tied low. Not connected to any internal signal.
20	DSADC_CNT[0]	Bit 0 (LSB) of the live DSADC ramp counter.
21	DSADC_CNT[1]	Bit 1 of the DSADC ramp counter.
22	DSADC_CNT[2]	Bit 2.
23	DSADC_CNT[3]	Bit 3.
24	DSADC_CNT[4]	Bit 4.
25	DSADC_CNT[5]	Bit 5.
26	DSADC_CNT[6]	Bit 6.
27	DSADC_CNT[7]	Bit 7.
28	DSADC_CNT[8]	Bit 8.
29	DSADC_CNT[9]	Bit 9.
30	DSADC_CNT[10]	Bit 10.
31	DSADC_CNT[11]	Bit 11 (MSB) of the live DSADC ramp counter. Observing bits 31:20 simultaneously gives the full 12-bit live counter value.

Table 52: AFE digital test port debug signal index map.

17.1 Register Description

17.1.1 AFE_CR Register

Register memory slot: 0, *offset:* 0 bytes, *size:* 4 bytes
AFE control register. Controls the DSADC, potentiostat, and analog test port. Bits 7:5 are not routed to any internal signal and should be written 0.

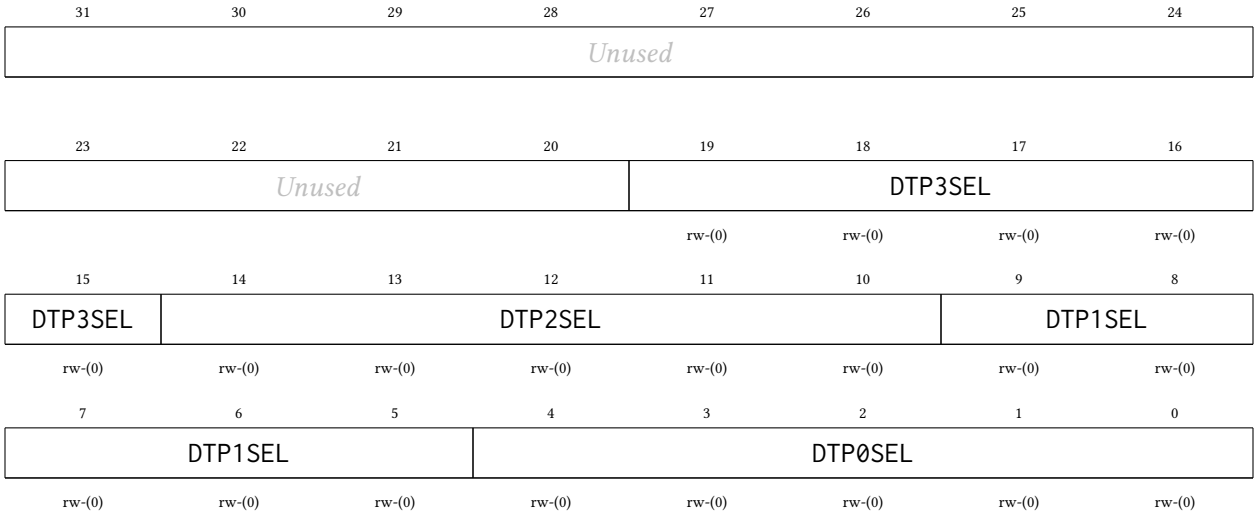


Bitfield	Range	Description
Unused	Bits 31-24	
RAMPNUM	Bits 23-12	Number of DSADC ramp cycles per conversion (12-bit). Larger values give longer integration time and increased sensitivity at the cost of conversion speed.
ADCSEL	Bit 11	ADC output multiplexer select. 0 DSADC result 1 SARADC result
ATPSEL	Bit 10	Analog test port signal select. 0 Potentiostat output 1 DSADC output
ATPEN	Bit 9	Analog test port enable. 0 Disabled 1 Enabled
ADCEXTIN	Bit 8	DSADC input source select. 0 External pin 1 Potentiostat output pad
Unused	Bits 7-5	
CONTMEAS	Bit 4	Continuous measurement mode. When set, the DSADC restarts automatically after each conversion without a software trigger. 0 Single conversion (triggered by writing AFE_ADC_VAL) 1 Continuous

DACEN	Bit 3	Enable the bias DACs for DSADC common-mode voltage (BIAS_DSADC_VCM) and potentiostat reference electrode voltage (BIAS_REV_POT). Must be set before starting a conversion. 0 DACs disabled 1 DACs enabled
DATARDYIE	Bit 2	Data-ready interrupt enable. When 1, an IRQ is generated when DATARDYIF is set. 0 Interrupt disabled 1 Interrupt enabled
EN	Bit 1	AFE enable. Gates the AFE peripheral clock. Must be 1 for any AFE operation. 0 AFE clock disabled 1 AFE clock enabled
ADCEN	Bit 0	DSADC enable. Must be 1 before triggering a conversion. 0 Disabled 1 Enabled

17.1.2 AFE_TPR Register

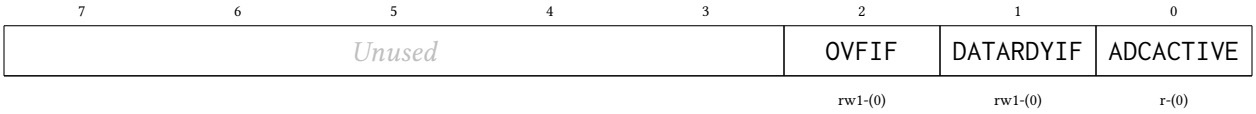
Register memory slot: 1, *offset:* 4 bytes, *size:* 4 bytes
AFE test port register. Four 5-bit fields select which internal debug signal is driven onto each of the four digital test ports DTP0-DTP3. See the Digital Test Ports section for the full signal index table.



Bitfield	Range	Description
Unused	Bits 31-20	
DTP3SEL	Bits 19-15	Digital test port 3 signal select (5-bit index into the debug vector)
DTP2SEL	Bits 14-10	Digital test port 2 signal select
DTP1SEL	Bits 9-5	Digital test port 1 signal select
DTP0SEL	Bits 4-0	Digital test port 0 signal select

17.1.3 AFE_SR Register

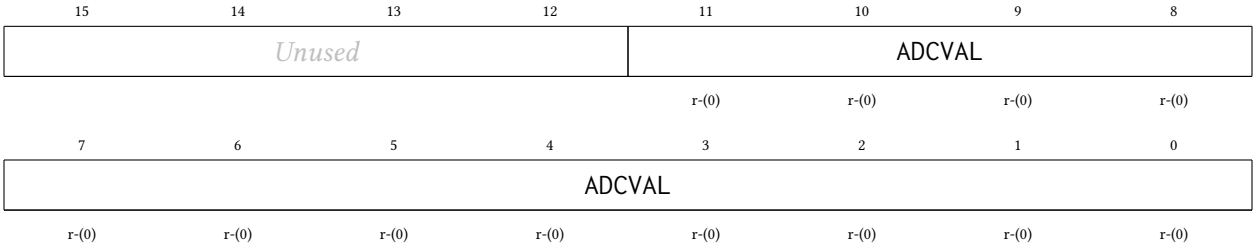
Register memory slot: 2, *offset:* 8 bytes, *size:* 1 byte
AFE status register.



Bitfield	Range	Description
Unused	Bits 7-3	
OVFIF	Bit 2	Overflow interrupt flag. Set when a conversion completes while DATARDYIF is still set. Write 1 to clear. 0 No overflow 1 Overflow: previous result was not read before new conversion completed
DATARDYIF	Bit 1	Data-ready interrupt flag. Set by hardware when a DSADC conversion completes. Write 1 to clear. 0 No data ready 1 Conversion result available in AFE_ADC_VAL
ADCACTIVE	Bit 0	ADC active flag. High while the DSADC FSM is running a conversion. 0 Idle 1 Conversion in progress

17.1.4 AFE_ADC_VAL Register

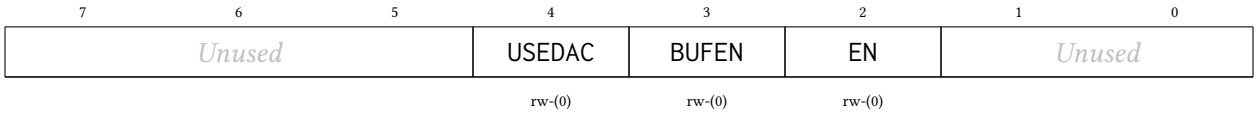
Register memory slot: 3, *offset:* 12 bytes, *size:* 2 bytes
DSADC result register. Reading returns the 12-bit result of the most recent conversion in bits 11:0; bits 15:12 read as zero. Writing any value to this register (when ADCEN = 1 and CONTMEAS = 0) triggers a new single-shot conversion.



Bitfield	Range	Description
Unused	Bits 15-12	
ADCVAL	Bits 11-0	12-bit DSADC conversion result. Represents the number of ramp clock cycles counted before the comparator fired.

17.1.5 BIAS_CR Register

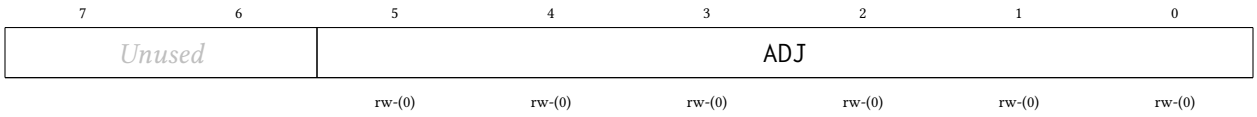
Register memory slot: 4, *offset:* 16 bytes, *size:* 1 byte
Bias control register. Selects the bias source (internal generator or DACs) and enables the generator and its output buffers.



Bitfield	Range	Description
Unused	Bits 7-5	
USEDAC	Bit 4	Bias source select. 0 Internal bias generator (use BIAS_ADJ to trim) 1 Bias DACs (BIAS_DBP/DBPC/DBN/DBNC)
BUFEN	Bit 3	Bias voltage buffer enable. Enables the output buffers on the internal global bias voltages. 0 Buffers disabled 1 Buffers enabled
EN	Bit 2	Internal bias generator enable. 0 Generator disabled 1 Generator enabled
Unused	Bits 1-0	

17.1.6 BIAS_ADJ Register

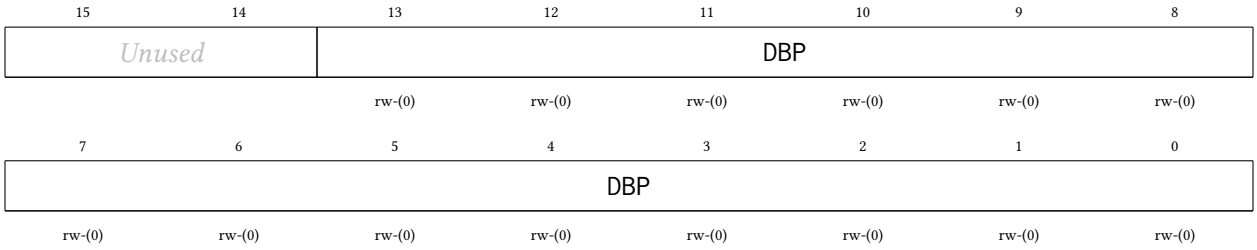
Register memory slot: 5, *offset:* 20 bytes, *size:* 1 byte
Wide-swing cascode bias generator adjustment register. Higher codes produce smaller bias currents (smaller voltage across the bias resistor). Used when BIAS_USEDAC = 0. The nominal trim code depends on process corner; see characterization data.



Bitfield	Range	Description
Unused	Bits 7-6	
ADJ	Bits 5-0	6-bit bias generator trim code (nominal = 37).

17.1.7 BIAS_DBP Register

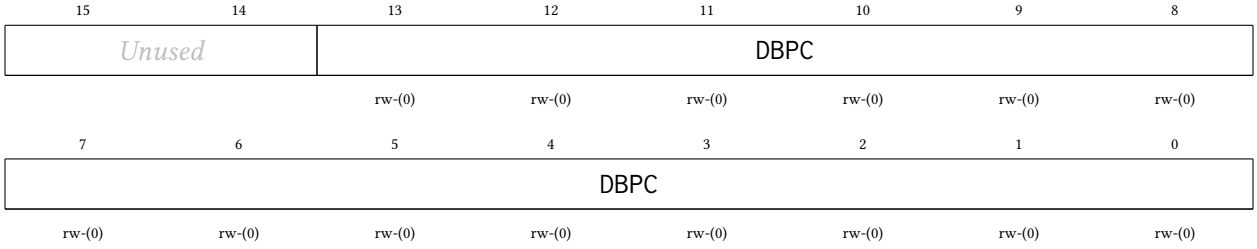
Register memory slot: 6, *offset:* 24 bytes, *size:* 2 bytes
P-branch bias DAC register. Sets the global V_bp voltage for PMOS current sources when BIAS_USEDAC = 1.



Bitfield	Range	Description
Unused	Bits 15-14	
DBP	Bits 13-0	14-bit P bias DAC code.

17.1.8 BIAS_DBPC Register

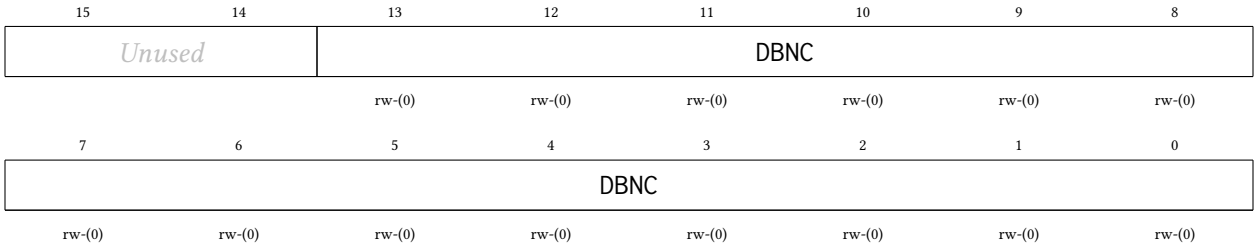
Register memory slot: 7, *offset:* 28 bytes, *size:* 2 bytes
P-cascode bias DAC register. Sets the global V_bpc voltage for PMOS cascode transistors when BIAS_USEDAC = 1.



Bitfield	Range	Description
Unused	Bits 15-14	
DBPC	Bits 13-0	14-bit P cascode bias DAC code.

17.1.9 BIAS_DBNC Register

Register memory slot: 8, *offset:* 32 bytes, *size:* 2 bytes
N-cascode bias DAC register. Sets the global V_bnc voltage for NMOS cascode transistors when BIAS_USEDAC = 1.



Bitfield	Range	Description
Unused	Bits 15-14	
DBNC	Bits 13-0	14-bit N cascode bias DAC code.

17.1.10 BIAS_DBN Register

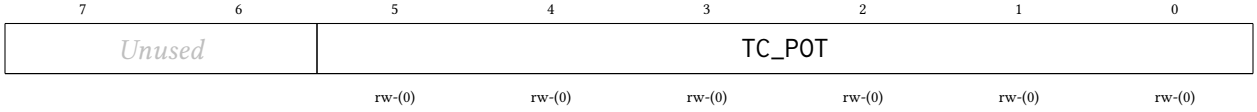
Register memory slot: 9, *offset:* 36 bytes, *size:* 2 bytes
N-branch bias DAC register. Sets the global V_bn voltage for NMOS current sources when BIAS_USEDAC = 1.



Bitfield	Range	Description
Unused	Bits 15-14	
DBN	Bits 13-0	14-bit N bias DAC code.

17.1.11 BIAS_TC_POT Register

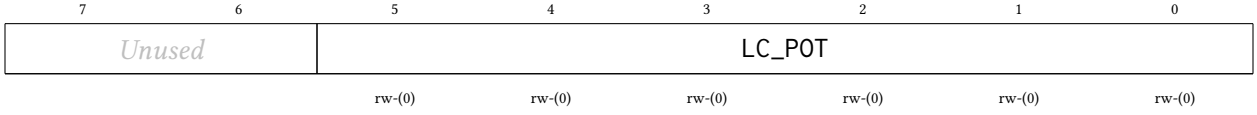
Register memory slot: 10, *offset:* 40 bytes, *size:* 1 byte
Potentiostat TIA tail-current bias trim register. Sets the quiescent current in the two-stage op-amp (A2) used as the transimpedance amplifier. Larger codes produce smaller currents.



Bitfield	Range	Description
Unused	Bits 7-6	
TC_POT	Bits 5-0	6-bit tail-current trim code for the potentiostat TIA amplifier (A2).

17.1.12 BIAS_LC_POT Register

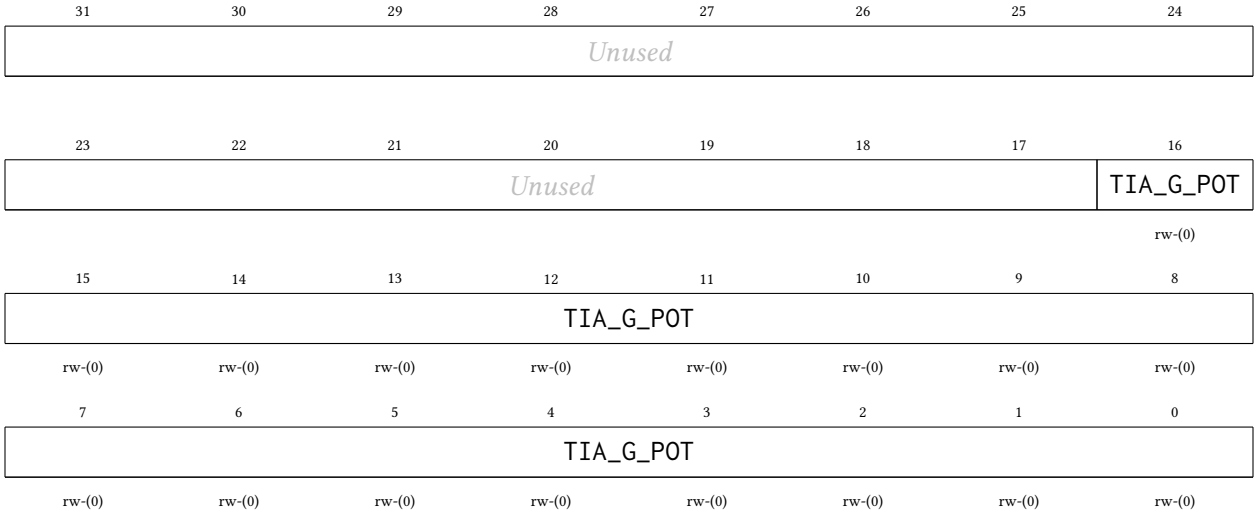
Register memory slot: 11, *offset:* 44 bytes, *size:* 1 byte
Potentiostat TIA Miller lead-compensation resistor trim register. The two-stage op-amp (A2) uses a Miller capacitor with a series resistor Rc to improve phase margin. This register trims Rc to cancel right-half-plane zeros introduced by process variation.



Bitfield	Range	Description
Unused	Bits 7-6	
LC_POT	Bits 5-0	6-bit Miller lead-compensation resistor Rc trim code for the potentiostat TIA (A2).

17.1.13 BIAS_TIA_G_POT Register

Register memory slot: 12, *offset:* 48 bytes, *size:* 4 bytes
Potentiostat TIA feedback resistor Rf register. Encoded as a 17-bit thermometer code controlling 16 series 60 kohm resistors plus one 1 Mohm resistor; each bit closes a parallel analog switch that shorts the corresponding resistor. Asserting more bits reduces Rf and therefore lowers TIA gain, increasing the measurable current range. Maximum gain (Rf = ~2 Mohm, sensitivity = ~56 pA) requires all bits clear (0x00000); minimum gain requires all bits set (0x1FFFF, reset value).



Bitfield	Range	Description
Unused	Bits 31-17	
TIA_G_POT	Bits 16-0	17-bit TIA gain resistance DAC code.

17.1.14 BIAS_DSADC_VCM Register

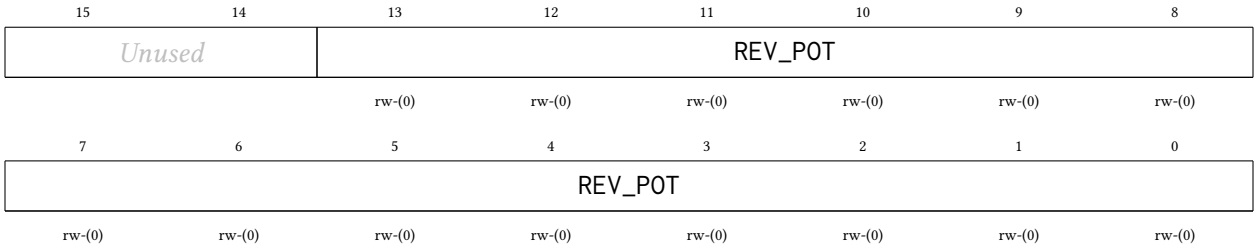
Register memory slot: 13, *offset:* 52 bytes, *size:* 2 bytes
Analog common-mode voltage (v_cm) DAC register. The generated voltage is applied to the non-inverting inputs of both the control amplifier (A1) and the TIA (A2), centring the output swing at V_DD/2 for single-supply operation. Midscale code 8192 produces V_DD/2. Must be enabled via DACEN (AFE_CR bit 3) before starting a conversion.



Bitfield	Range	Description
Unused	Bits 15-14	
DSADC_VCM	Bits 13-0	14-bit v_cm R-2R DAC code. Midscale = 8192 gives V_DD/2.

17.1.15 BIAS_REV_POT Register

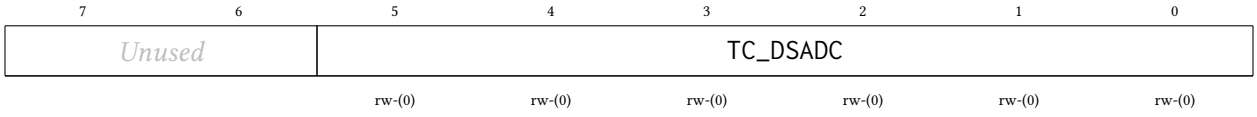
Register memory slot: 14, *offset:* 56 bytes, *size:* 2 bytes
Potentiostat excitation voltage (v_pattern) DAC register. The generated voltage is applied to the non-inverting input of the folded-cascode control amplifier (A1), which drives the counter electrode such that V_RE = v_pattern. Programming this register sets the working-to-reference electrode potential. For cyclic voltammetry, firmware increments or decrements this register over time; for chronoamperometry, it is stepped once to the desired overpotential. Must be enabled via DACEN (AFE_CR bit 3).



Bitfield	Range	Description
Unused	Bits 15-14	
REV_POT	Bits 13-0	14-bit v_pattern R-2R DAC code. Sets the WE-RE electrode potential.

17.1.16 BIAS_TC_DSADC Register

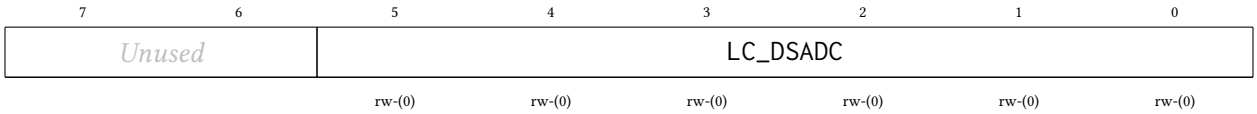
Register memory slot: 15, *offset:* 60 bytes, *size:* 1 byte
DSADC bias current source (BTS) trim register.



Bitfield	Range	Description
Unused	Bits 7-6	
TC_DSADC	Bits 5-0	6-bit tail-current trim code for the DSADC integrating amplifier.

17.1.17 BIAS_LC_DSADC Register

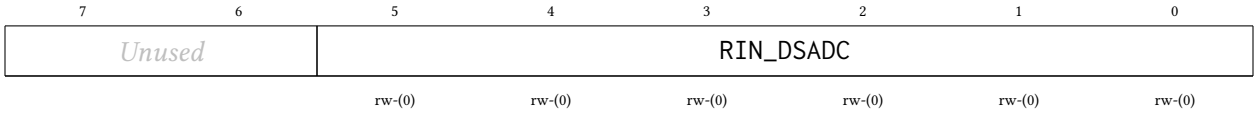
Register memory slot: 16, *offset:* 64 bytes, *size:* 1 byte
DSADC integrating amplifier Miller lead-compensation resistor trim register. Adjusts the Rc zero-placement for proper phase margin in the two-stage integrator op-amp.



Bitfield	Range	Description
Unused	Bits 7-6	
LC_DSADC	Bits 5-0	6-bit Miller lead-compensation resistor Rc trim code for the DSADC integrating amplifier.

17.1.18 BIAS_RIN_DSADC Register

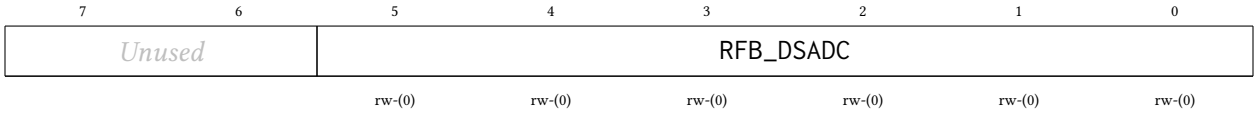
Register memory slot: 17, *offset:* 68 bytes, *size:* 1 byte
DSADC integration resistor R1 trim register. R1 and the integration capacitor C1 define the ramp slope during the integration phase. On-chip resistors can vary up to +/-20% from nominal; this 6-bit trim code corrects the R1 value to maintain the desired R1*C1 time constant.



Bitfield	Range	Description
Unused	Bits 7-6	
RIN_DSADC	Bits 5-0	6-bit R1 trim code for the DSADC integration resistor.

17.1.19 BIAS_RFB_DSADC Register

Register memory slot: 18, *offset:* 72 bytes, *size:* 1 byte
DSADC lead-compensation resistor R2 trim register. R2 is placed in series with the integration capacitor C1 to mitigate integrator output overshoot at the transition from integration to deintegration phase. On-chip resistors can vary up to +/-20% from nominal; this 6-bit trim code corrects R2 accordingly.



Bitfield	Range	Description
Unused	Bits 7-6	
RFB_DSADC	Bits 5-0	6-bit R2 lead-compensation resistor trim code for the DSADC.

18 I2Cx Peripheral

I2C serial port interface. The master and slave I2C interfaces are split between two sets of registers.

To use master transmitter mode, first configure the I2C peripheral by setting I2CMEN, clearing I2CSEN, and configuring I2CMDIV with the appropriate clock division factor, noting that the I2C clock source is SMCLK. To send a start condition, set I2CMST, wait for the I2CMSTS flag to be set (if the bus is busy, the I2C peripheral will wait for it to become idle and then send a start condition), and then clear the status register. I2CMCB will now indicate that the I2C peripheral now has control of the bus as its master. Next, write to I2CxMTX the desired slave address in the most significant 7 bits followed by the desired read/write bit (0 for write/master transmitter) in the least significant bit. Then, wait for I2CMXC or I2CMARB to be set. If I2CMARB is set, then the I2C peripheral has lost the bus arbitration contest and has released control of the bus. If I2CMNR is set, then the desired slave has not acknowledged itself. Clear the status register. Next, send the slave a byte of data by writing the desired data to I2CxMTX. When the I2C peripheral is ready for another byte of data to be queued for transmission, the I2CMTXE flag will be set. Again, wait for I2CMXC or I2CMARB, then check I2CMARB and I2CMNR, and finally clear the status register. Once finished sending all of the desired bytes, either send a stop condition to release control of the bus by setting I2CMSP, or send a repeated start condition to retain control of the bus with a new transmission (and possibly a new slave and read/write mode) by setting I2CMST. Once a stop condition is sent, wait for I2CMSTS to be set, indicating that a stop condition has been sent. Clear the status register.

To use master receiver mode, first configure the I2C peripheral by setting I2CMEN, clearing I2CSEN, and configuring I2CMDIV with the appropriate clock division factor, noting that the I2C clock source is SMCLK. To send a start condition, set I2CMST, wait for the I2CMSTS flag to be set (if the bus is busy, the I2C peripheral will wait for it to become idle and then send a start condition), and then clear the status register. I2CMCB will now indicate that the I2C peripheral now has control of the bus as its master. Next, write to I2CxMTX the desired slave address in the most significant 7 bits followed by the desired read/write bit (1 for read/master receiver) in the least significant bit. Then, wait for I2CMXC or I2CMARB to be set. If I2CMARB is set, then the I2C peripheral has lost the bus arbitration contest and has released control of the bus. If I2CMNR is set, then the desired slave has not acknowledged itself. Clear the status register. Next, begin to receive a byte of data from the slave by setting I2CMRB. Wait for I2CMXC to be set. Clear the status register. Read I2CxMRX to get the byte of data received from the slave. To send the slave an ACK and begin to read another byte from the slave, set I2CMRB. Or, to send the slave a NACK and send a stop condition, set I2CMSP. Or, to send the slave a NACK and send a repeated start condition, set I2CMST. Wait for the appropriate flag, then clear the status register.

To use slave receiver mode, first configure the I2C peripheral by setting I2CSEN, clearing I2CSEN, clearing I2CSN, and configuring I2CSCS and I2CGCE to the desired values. Note that if clock stretching is enabled with I2CSCS, the I2C peripheral will seize control of the bus by driving SCL low during every ACK/NACK bit transfer (if the I2C peripheral was addressed) until I2CSC is set, which requires user intervention to prevent indefinite hold-ups of the I2C bus. But, if clock stretching is not enabled, the master will be allowed full control of the rate data is sent over the bus, which opens the possibility that the software running on this MCU does not notice that a byte has been transferred in time before the next is transferred. Note that if I2CGCE is set, the I2C peripheral will be addressed if either its address is received or if the general call is received. Wait for the I2C peripheral to be addressed when I2CSA is set. Check I2CSTM to see if the master has requested slave receiver or slave transmitter mode (0 indicates slave receiver). Clear the status register. If clock stretching is enabled, send an ACK or NACK by clearing or setting I2CSN, and then set I2CSC to release SDA and continue with the transfer. If clock stretching is not enabled, the I2C peripheral will automatically ACK or NACK depending on the value of I2CSN. Next, wait for the slave to receive a data byte from the master when I2CSXC is set. If I2CSOVF is set, then the MCU has failed to read one of the bytes sent by the master in the past. Clear the status register, and then read I2CSRX to get

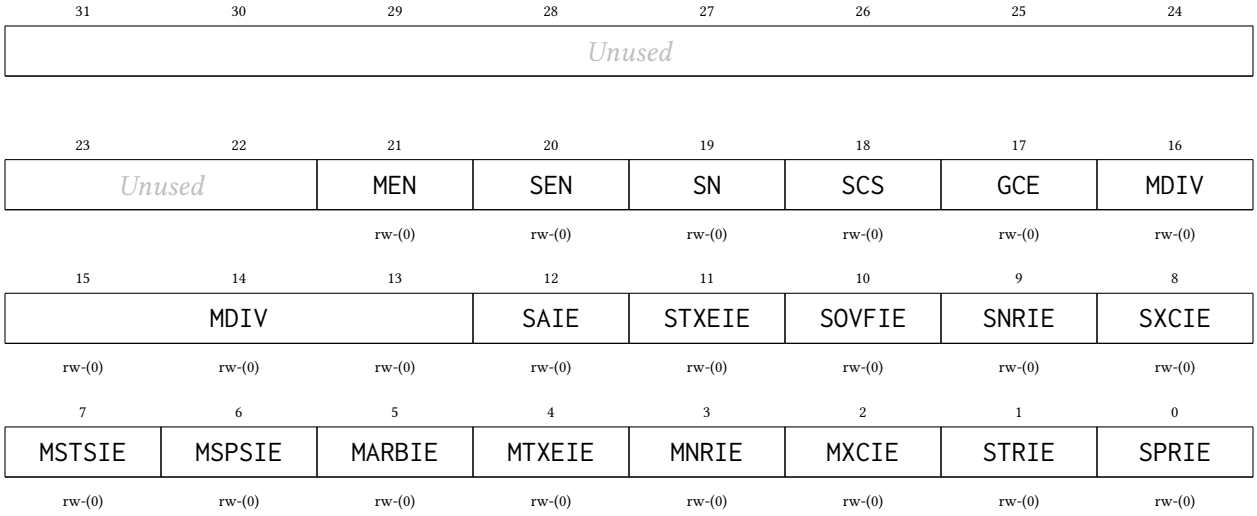
the data byte sent from the master. If clock stretching is enabled, send an ACK or NACK by clearing or setting I2CSN, and then set I2CSC to release SDA and continue with the transfer. If clock stretching is not enabled, the I2C peripheral will automatically ACK or NACK depending on the value of I2CSN. Next, wait for I2CSXC, I2CSPR, or I2CSTR to be set, indicating the I2C peripheral has received a new byte of data, a stop condition, or a repeated start condition. If a stop or start condition has been received, clear the status register.

To use slave transmitter mode, first configure the I2C peripheral by setting I2CSEN, clearing I2CSEN, clearing I2CSN, and configuring I2CSCS and I2CGCE to the desired values. Note that if clock stretching is enabled with I2CSCS, the I2C peripheral will seize control of the bus by driving SCL low during every ACK/NACK bit transfer (if the I2C peripheral was addressed) until I2CSC is set, which requires user intervention to prevent indefinite hold-ups of the I2C bus. But, if clock stretching is not enabled, the master will be allowed full control of the rate data is sent over the bus, which opens the possibility that the software running on this MCU does not notice that a byte has been transferred in time before the next is transferred. Check I2CSTM to see if the master has requested slave receiver or slave transmitter mode (1 indicates slave transmitter). Clear the status register. Queue the byte of data to transmit to the master by writing the byte to I2CSTX. If clock stretching is enabled, set I2CSC to release SDA and continue with the transfer. If clock stretching is not enabled, the I2C peripheral will automatically ACK or NACK depending on the value of I2CSN. Wait for I2CSTXE to be set, indicating that the I2C peripheral is ready to queue the next byte to send to the master. Clear the status register, and write the next byte to send to the master to I2CxSTX. If clock stretching is enabled, wait for I2CSXC to be set, clear the status register, and set I2CSC. Wait for I2CSTXE, I2CSPR, or I2CSTR to be set, then clear the status register.

18.1 Register Description

18.1.1 `I2CCR` Register

Register memory slot: 0, *offset:* 0 bytes, *size:* 4 bytes
I2C control register



Bitfield	Range	Description
Unused	Bits 31-22	

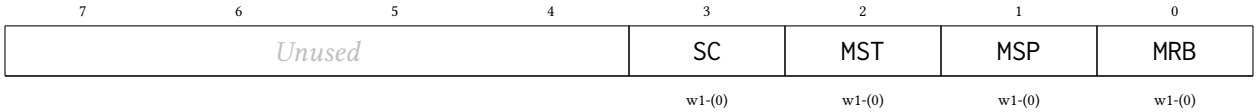
MEN	Bit 21	I2C master enable. When enabled, this device awaits a command to send a start condition with I2CMST, and then begins acting as a master until commanded to send a stop condition with I2CMSP. If master mode is also enabled on this device, then this device will act as a slave until commanded to send a start condition with I2CMST, whereupon it will begin acting as a master. Once the master transfer is complete, it will resume acting as a slave. 0 Disabled 1 Enabled
SEN	Bit 20	I2C slave enable. When enabled, this device behaves as an I2C slave and begins listening for its address. If master mode is also enabled on this device, then this device will act as a slave until commanded to send a start condition with I2CMST, whereupon it will begin acting as a master. Once the master transfer is complete, it will resume acting as a slave. 0 Disabled 1 Enabled
SN	Bit 19	I2C slave NACK next byte received. When enabled, this slave will reply with a NACK whenever it receives its address or whenever it receives a byte from a master. When disabled, it will send an ACK in those situations. If clock stretching is enabled, this bit can be changed in accordance with the desired ACK/NACK reply before allowing a rising edge of SCL. 0 ACK 1 NACK
SCS	Bit 18	I2C slave clock stretching enable. When enabled, this slave will hold the SCL line low during the ACK phase of the transmission to allow this slave more time. Note that the master will be left waiting for the ACK/NACK as long as this bit is set to 1. 0 Disabled 1 Enabled
GCE	Bit 17	I2C slave general call enable. When enabled, this slave will be addressed if a global call is issued on the bus. 0 Disabled 1 Enabled
MDIV	Bits 16-13	I2C master mode clock divider. The master mode finite state machine clock source is SMCLK, which is divided by a factor of 4 * 2**I2CMDIV. 0000 I2CMDIV_1: /1 (no division) 0001 I2CMDIV_2: /2 0010 I2CMDIV_4: /4 0011 I2CMDIV_8: /8 0100 I2CMDIV_16: /16 0101 I2CMDIV_32: /32 0110 I2CMDIV_64: /64 0111 I2CMDIV_128: /128 1000 I2CMDIV_256: /256 1001 I2CMDIV_512: /512 1010 I2CMDIV_1024: /1,024 1011 I2CMDIV_2048: /2,048

		1100 I2CMDIV_4096: /4,096
		1101 I2CMDIV_8192: /8,192
		1110 I2CMDIV_16384: /16,384
		1111 I2CMDIV_32768: /32,768
SAIE	Bit 12	I2C slave mode addressed interrupt enable
		0 Interrupt disabled
		1 Interrupt enabled
STXEIE	Bit 11	I2C slave transmit register empty interrupt enable
		0 Interrupt disabled
		1 Interrupt enabled
SOVFIE	Bit 10	I2C slave receive register overflow interrupt enable
		0 Interrupt disabled
		1 Interrupt enabled
SNRIE	Bit 9	I2C slave mode NACK received from master interrupt enable
		0 Interrupt disabled
		1 Interrupt enabled
SXCIE	Bit 8	I2C slave mode transfer complete interrupt enable
		0 Interrupt disabled
		1 Interrupt enabled
MSTSIE	Bit 7	I2C master mode start condition sent interrupt enable
		0 Interrupt disabled
		1 Interrupt enabled
MSPSIE	Bit 6	I2C master mode stop condition sent interrupt enable
		0 Interrupt disabled
		1 Interrupt enabled
MARBIE	Bit 5	I2C master mode arbitration error interrupt enable
		0 Interrupt disabled
		1 Interrupt enabled
MTXEIE	Bit 4	I2C master transmit register empty interrupt enable
		0 Interrupt disabled
		1 Interrupt enabled
MNRIE	Bit 3	I2C master mode NACK received interrupt enable
		0 Interrupt disabled
		1 Interrupt enabled
MXCIE	Bit 2	I2C master transfer complete interrupt enable
		0 Interrupt disabled
		1 Interrupt enabled
STRIE	Bit 1	I2C start received interrupt enable
		0 Interrupt disabled
		1 Interrupt enabled
SPRIE	Bit 0	I2C stop received interrupt enable
		0 Interrupt disabled
		1 Interrupt enabled

18.1.2 I2CFCR Register

Register memory slot: 1, *offset:* 4 bytes, *size:* 1 byte

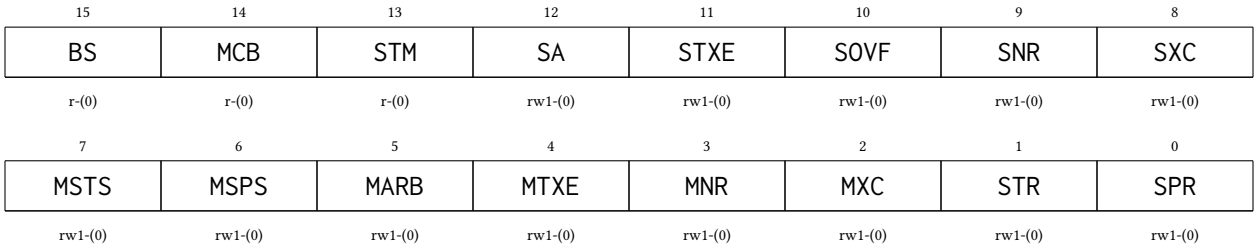
I2C flow control register. Writing a 1 to a bit in this register initiates or queues the associated command. Writing a 0 to a bit does nothing. Reading this register always returns the value 0.



Bitfield	Range	Description
Unused	Bits 7-4	
SC	Bit 3	I2C slave continue command. When clock stretching is enabled in slave mode, set this bit to tell the slave to continue with the ACK/NACK phase of the current byte by releasing SCL. This may only be set if clock stretching is enabled, slave mode is enabled, and the slave transfer complete flag has just been set. 0 No effect 1 Send a start condition
MST	Bit 2	I2C master send start condition command. Set this bit to 1 to send a start condition or a repeated start condition. Once this bit is set, this master is required to send at least one address frame before initiating this command again. If another master has control of the bus when this bit is set, this master will wait until the bus is idle before seizing control of it and sending the start condition. If this master is busy with a transaction when this bit is set, then it will send a restart condition immediately after it finishes the transaction. 0 No effect 1 Send a start condition
MSP	Bit 1	I2C master send stop condition command. Set this bit to 1 while this master is in control of the bus to send a stop condition. This master is required to have already sent a start condition and at least one address frame before initiating this command. If this master is busy with a transaction when this bit is set, then it will send the stop condition immediately after it finishes the transaction. 0 No effect 1 Send a stop condition
MRB	Bit 0	I2C master read byte command. Set this bit to 1 while in master receiver mode to read one byte from the slave. This master is required to have already sent the slave address and received an ACK from the slave before initiating this command. 0 No effect 1 Read next byte

18.1.3 I2CSR Register

Register memory slot: 2, *offset:* 8 bytes, *size:* 2 bytes
I2C status register



Bitfield	Range	Description
BS	Bit 15	I2C bus state indicator. This bit cannot be cleared by writing to the status register. 0 The I2C bus is idle 1 The I2C bus is active
MCB	Bit 14	I2C master controls bus indicator. This bit cannot be cleared by writing to the status register. 0 This master does not control the bus 1 This master controls the bus
STM	Bit 13	I2C slave transmitter mode indicator. Indicates for which mode this slave has been addressed. Only valid if I2CSA is 1. This bit cannot be cleared by writing to the status register. 0 Slave receiver mode 1 Slave transmitter mode
SA	Bit 12	I2C slave mode addressed interrupt flag. Indicates that this slave has been addressed by another master. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
STXE	Bit 11	I2C slave transmit register empty interrupt flag. This bit is set when this slave latches the data stored in the masslaveter transmit register to indicate that the slave transmit register is ready to accept another byte and queue it for transmission. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
SOVF	Bit 10	I2C slave receive register overflow interrupt flag. Indicates that this slave has failed to read one or more bytes from the I2CxSRX register before they were overwritten by another transmission. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
SNR	Bit 9	I2C slave mode NACK received from master interrupt flag. This bit is set in slave transmitter mode if the master responds with a NACK after this slave sends it a byte of data. This bit is not set if the master responds with an ACK. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
SXC	Bit 8	I2C slave mode transfer complete interrupt flag. This bit is set after this slave receives a byte of data from a master, but before this slave sends an ACK/NACK. Write a 1 to this bit to clear it.

		0 No pending interrupt 1 Pending interrupt
MSTS	Bit 7	I2C master mode start condition sent interrupt flag. This flag is set after this master sends a start condition or repeated start condition. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
MSPS	Bit 6	I2C master mode stop condition sent interrupt flag. This flag is set after this master sends a stop condition. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
MARB	Bit 5	I2C master mode arbitration loss interrupt flag. This bit is set when this master detects that the value it tried to write to SDA is being overridden by another master. After it detects the arbitration loss, this master releases control of the bus. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
MTXE	Bit 4	I2C master transmit register empty interrupt flag. This bit is set when this master latches the data stored in the master transmit register to indicate that the master transmit register is ready to accept another byte and queue it for transmission. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
MNR	Bit 3	I2C master mode NACK received interrupt flag. This flag is set in master transmitter mode after ACK/NACK bit is sent if the slave sends a NACK. It is not set if the slave sends an ACK. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
MXC	Bit 2	I2C master transfer complete interrupt flag. In master transmitter mode, this flag is set after this master has sent the data byte and the slave has sent an ACK/NACK. In master receiver mode, this flag is set after the slave has sent the data byte and before this master sends an ACK/NACK. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
STR	Bit 1	I2C start condition received interrupt flag. This flag is set whenever a start condition or repeated start condition is detected on the bus, regardless of if this master or another sent it. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt
SPR	Bit 0	I2C stop condition received interrupt flag. This flag is set whenever a stop condition condition is detected on the bus, regardless of which device sent it. Write a 1 to this bit to clear it. 0 No pending interrupt 1 Pending interrupt

18.1.4 I2CMTX Register

Register memory slot: 3, *offset:* 12 bytes, *size:* 1 byte
I2C master transmit register. Write the desired slave address and read/write bit to this register after sending a start condition to begin a transmission with a slave. Note that the desired slave address must occupy the upper seven bits and the read/write bit must occupy the least significant bit. If the read bit is 0, the master enters master transmitter mode. If the read bit is 1, the master enters master receiver mode. Write a byte of data to this register after sending the address frame or a data frame to send that byte of data to the slave. If this master is busy with a transmission when this register is written, it will send the byte after it finishes the transmission.



18.1.5 I2CMRX Register

Register memory slot: 4, *offset:* 16 bytes, *size:* 1 byte
I2C master receive register. When in master receiver mode, read this register after the master transfer complete interrupt flag (I2CMXC) is set to get the received data byte.



18.1.6 I2CSTX Register

Register memory slot: 5, *offset:* 20 bytes, *size:* 1 byte
I2C slave transmit register. When in slave transmitter mode, write to this register after the slave addressed flag (I2CSA) or the slave transaction complete flag (I2CSXC) has been set to queue the next byte to send to the master.



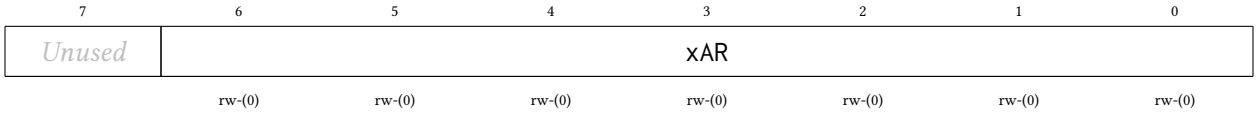
18.1.7 I2CSRX Register

Register memory slot: 6, *offset:* 24 bytes, *size:* 1 byte
I2C slave receive register. When in slave receiver mode, read this register after the slave transaction complete flag (I2CSXC) has been set to get the data byte sent from the master. Note that if this slave fails to clear the status register before another byte is received, the slave receive overflow flag (I2CSOVF) will be set.



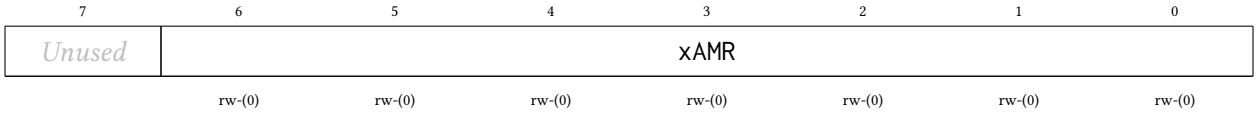
18.1.8 `I2CAR` Register

Register memory slot: 7, *offset:* 28 bytes, *size:* 1 byte
I2C this slave address register. When in slave mode, any master that sends an address frame containing this address will activate this slave.



18.1.9 `I2CAMR` Register

Register memory slot: 8, *offset:* 32 bytes, *size:* 1 byte
I2C this slave address mask register. Any bit set to 1 in this register indicates that the corresponding bit in the slave address register is a wildcard. Only the slave address register bits that correspond to 0s in this register will be compared to the received slave address.



19 Register and Peripheral Memory Map

19.1 `GPIO0` Peripheral

Base address: `0x4000`, peripheral memory slot: 0

Register	Address	Slot	Offset (bytes)	Size (bytes)
P0IN	0x4000	0	0	1
P0OUT	0x4004	1	4	1
P0OUTS	0x4008	2	8	1
P0OUTC	0x400C	3	12	1
P0OUTT	0x4010	4	16	4
P0DIR	0x4014	5	20	1
P0IF	0x4018	6	24	4
P0IES	0x401C	7	28	1
P0IE	0x4020	8	32	1
P0SEL	0x4024	9	36	1
P0REN	0x4028	10	40	1

19.2 `GPIO1` Peripheral

Base address: `0x4100`, peripheral memory slot: 1

Register	Address	Slot	Offset (bytes)	Size (bytes)
P1IN	0x4100	0	0	1
P1OUT	0x4104	1	4	1
P1OUTS	0x4108	2	8	1
P1OUTC	0x410C	3	12	1
P1OUTT	0x4110	4	16	4
P1DIR	0x4114	5	20	1
P1IF	0x4118	6	24	4
P1IES	0x411C	7	28	1
P1IE	0x4120	8	32	1
P1SEL	0x4124	9	36	1
P1REN	0x4128	10	40	1

19.3 `SPI0` Peripheral

Base address: `0x4200`, peripheral memory slot: 2

Register	Address	Slot	Offset (bytes)	Size (bytes)
SPI0CR	0x4200	0	0	4
SPI0SR	0x4204	1	4	1
SPI0TX	0x4208	2	8	4
SPI0RX	0x420C	3	12	4
SPI0FOS	0x4210	4	16	4

19.4 `SPI1` Peripheral

Base address: `0x4300`, peripheral memory slot: 3

Register	Address	Slot	Offset (bytes)	Size (bytes)
SPI1CR	0x4300	0	0	4
SPI1SR	0x4304	1	4	1
SPI1TX	0x4308	2	8	4
SPI1RX	0x430C	3	12	4
SPI1FOS	0x4310	4	16	4

19.5 UART0 Peripheral

Base address: 0x4400, peripheral memory slot: 4

Register	Address	Slot	Offset (bytes)	Size (bytes)
UART0CR	0x4400	0	0	1
UART0SR	0x4404	1	4	1
UART0BR	0x4408	2	8	2
UART0RX	0x440C	3	12	1
UART0TX	0x4410	4	16	1

19.6 UART1 Peripheral

Base address: 0x4500, peripheral memory slot: 5

Register	Address	Slot	Offset (bytes)	Size (bytes)
UART1CR	0x4500	0	0	1
UART1SR	0x4504	1	4	1
UART1BR	0x4508	2	8	2
UART1RX	0x450C	3	12	1
UART1TX	0x4510	4	16	1

19.7 TIMER0 Peripheral

Base address: 0x4600, peripheral memory slot: 6

Register	Address	Slot	Offset (bytes)	Size (bytes)
TIM0CR	0x4600	0	0	4
TIM0SR	0x4604	1	4	1
TIM0VAL	0x4608	2	8	4
TIM0CMP0	0x460C	3	12	4
TIM0CMP1	0x4610	4	16	4
TIM0CMP2	0x4614	5	20	4
TIM0CAP0	0x4618	6	24	4
TIM0CAP1	0x461C	7	28	4

19.8 TIMER1 Peripheral

Base address: 0x4700, peripheral memory slot: 7

Register	Address	Slot	Offset (bytes)	Size (bytes)

TIM1CR	0x4700	0	0	4
TIM1SR	0x4704	1	4	1
TIM1VAL	0x4708	2	8	4
TIM1CMP0	0x470C	3	12	4
TIM1CMP1	0x4710	4	16	4
TIM1CMP2	0x4714	5	20	4
TIM1CAP0	0x4718	6	24	4
TIM1CAP1	0x471C	7	28	4

19.9 GPIO2 Peripheral

Base address: 0x4800, peripheral memory slot: 8

Register	Address	Slot	Offset (bytes)	Size (bytes)
P2IN	0x4800	0	0	1
P2OUT	0x4804	1	4	1
P2OUTS	0x4808	2	8	1
P2OUTC	0x480C	3	12	1
P2OUTT	0x4810	4	16	4
P2DIR	0x4814	5	20	1
P2IF	0x4818	6	24	4
P2IES	0x481C	7	28	1
P2IE	0x4820	8	32	1
P2SEL	0x4824	9	36	1
P2REN	0x4828	10	40	1

19.10 SYSTEM Peripheral

Base address: 0x4900, peripheral memory slot: 9

Register	Address	Slot	Offset (bytes)	Size (bytes)
SYSCLKCR	0x4900	0	0	2
CLKDIVCR	0x4904	1	4	1
BLOCKPWR	0x4908	2	8	1
CRCDATA	0x490C	3	12	1
CRCSTATE	0x4910	4	16	2
IRQENL	0x4914	5	20	4
IRQENM	0x4918	6	24	4
IRQENU	0x491C	7	28	4
IRQPRIL	0x4920	8	32	4
IRQPRIM	0x4924	9	36	4
IRQPRIU	0x4928	10	40	4
IRQCR	0x492C	11	44	1
WDTCR	0x4930	12	48	1
WDTSR	0x4934	13	52	1
WDTPASS	0x4938	14	56	4
WDTVAL	0x493C	15	60	4
DCO0BIAS	0x4940	16	64	2

DCO1BIAS	0x4944	17	68	2

19.11 NPU Peripheral

Base address: 0x4A00, peripheral memory slot: 10

Register	Address	Slot	Offset (bytes)	Size (bytes)
NPUCR	0x4A00	0	0	4
NPUIVSAR	0x4A04	1	4	4
NPUWVSAR	0x4A08	2	8	4
NPUOVSAR	0x4A0C	3	12	4

19.12 SARADC Peripheral

Base address: 0x4B00, peripheral memory slot: 11

Register	Address	Slot	Offset (bytes)	Size (bytes)
SARADC_CR	0x4B00	0	0	2
SARADC_CDIV	0x4B04	1	4	1
SARADC_SR	0x4B08	2	8	1
SARADC_DATA	0x4B0C	3	12	2
SARADC_TPR	0x4B10	4	16	1

19.13 AFE Peripheral

Base address: 0x4C00, peripheral memory slot: 12

Register	Address	Slot	Offset (bytes)	Size (bytes)
AFE_CR	0x4C00	0	0	4
AFE_TPR	0x4C04	1	4	4
AFE_SR	0x4C08	2	8	1
AFE_ADC_VAL	0x4C0C	3	12	2
BIAS_CR	0x4C10	4	16	1
BIAS_ADJ	0x4C14	5	20	1
BIAS_DBP	0x4C18	6	24	2
BIAS_DBPC	0x4C1C	7	28	2
BIAS_DBNC	0x4C20	8	32	2
BIAS_DBN	0x4C24	9	36	2
BIAS_TC_POT	0x4C28	10	40	1
BIAS_LC_POT	0x4C2C	11	44	1
BIAS_TIA_G_POT	0x4C30	12	48	4
BIAS_DSADC_VCM	0x4C34	13	52	2
BIAS_REV_POT	0x4C38	14	56	2
BIAS_TC_DSADC	0x4C3C	15	60	1
BIAS_LC_DSADC	0x4C40	16	64	1
BIAS_RIN_DSADC	0x4C44	17	68	1
BIAS_RFB_DSADC	0x4C48	18	72	1

19.14 GPIO3 Peripheral

Base address: 0x4D00, peripheral memory slot: 13

Register	Address	Slot	Offset (bytes)	Size (bytes)
P3IN	0x4D00	0	0	1
P3OUT	0x4D04	1	4	1
P3OUTS	0x4D08	2	8	1
P3OUTC	0x4D0C	3	12	1
P3OUTT	0x4D10	4	16	4
P3DIR	0x4D14	5	20	1
P3IF	0x4D18	6	24	4
P3IES	0x4D1C	7	28	1
P3IE	0x4D20	8	32	1
P3SEL	0x4D24	9	36	1
P3REN	0x4D28	10	40	1

19.15 I2C0 Peripheral

Base address: 0x4E00, peripheral memory slot: 14

Register	Address	Slot	Offset (bytes)	Size (bytes)
I2C0CR	0x4E00	0	0	4
I2C0FCR	0x4E04	1	4	1
I2C0SR	0x4E08	2	8	2
I2C0MTX	0x4E0C	3	12	1
I2C0MRX	0x4E10	4	16	1
I2C0STX	0x4E14	5	20	1
I2C0SRX	0x4E18	6	24	1
I2C0AR	0x4E1C	7	28	1
I2C0AMR	0x4E20	8	32	1

19.16 I2C1 Peripheral

Base address: 0x4F00, peripheral memory slot: 15

Register	Address	Slot	Offset (bytes)	Size (bytes)
I2C1CR	0x4F00	0	0	4
I2C1FCR	0x4F04	1	4	1
I2C1SR	0x4F08	2	8	2
I2C1MTX	0x4F0C	3	12	1
I2C1MRX	0x4F10	4	16	1
I2C1STX	0x4F14	5	20	1
I2C1SRX	0x4F18	6	24	1
I2C1AR	0x4F1C	7	28	1
I2C1AMR	0x4F20	8	32	1

20 Errata

Listed are the known issues with the Myshkin MCU. The last ten are digital defects found after tape-out. All but the last were reproduced by simulating the tape-out netlist with sign-off timing, and the last was found in the tape-out RTL. None has been tested on silicon. All are corrected in the RTL for subsequent chips.

- **`SARADC` converts at one quarter of the intended rate.** The ADC macro completes one conversion every four trigger periods instead of one. With `SAMPLESTEP` = 7 the trigger period is 23 peripheral-clock cycles, 1.15 µs at 20 MHz, so a conversion takes 4.60 µs — 217 kSa/s in place of the intended 870 kSa/s.

  The trigger waveform is produced by registering the gated expression `clear or sample or (conversion and clk)` on the rising edge of `clk`. At a rising edge `clk` is already 1, so the gating term is a constant and the flip-flop captures a level rather than passing a gated clock. The conversion phase emerges as a single flat 500 ns high instead of ten 25 ns pulses, leaving three falling edges per trigger period instead of twelve. The converter's successive-approximation register is clocked on the negative edge, so it accumulates its ten bit decisions over four trigger periods. Simulation places the ready strobe at 8.56, 13.16 and 17.76 µs, exactly 4 × 1.15 µs apart.

  Conversion results are correct: the converter returns the same code it would with the intended waveform and only takes four times as long, so codes read correctly on the bench. Firmware must allow 4.60 µs per conversion. `DATAVALID` and its interrupt remain the correct completion signal, because the result is latched on the rising edge of the macro's ready output, inside the approximately 100 ns window during which the data bus holds the result before returning to its idle all-ones state; code that instead assumes completion after a fixed delay of one trigger period reads the idle value, not a result.

  There is no workaround on this silicon — the trigger waveform is fixed in the gate netlist. Continuous-measurement mode (`CONTMEAS`) still streams conversions back to back, so the only loss is sample rate. Corrected in the RTL for subsequent chips.

- **The bootloader cannot boot from SPI flash.** Chip select is never deasserted between the flash wake-up command and the read command that follows it. The flash sees one uninterrupted transaction, never decodes the read opcode, and returns no program data, so a program stored in the SPI flash cannot be booted.

  The bootloader drives the flash on `SPI0` and controls chip select by hand on `P0.0`, held in primary/GPIO mode, through the `GPIO0` output set and clear registers. It clears the CS bit through `P0OUTC`, sends the flash wake-up command `0xAB`, waits for the transfer to complete, then writes `P0OUTS` to release CS. The value written is wrong. The CS mask `0x01` is held in `a4`, but the store uses `a2`, which was reloaded with the RAM base address during setup: the ROM image writes `0x00008000` (`lui a2,0x8` at `0x34`, `sw a2,8(a1)` at `0x78`), which sets no bit of `P0OUTS` and leaves CS asserted. The high-frequency read command `0x0B`, the 24-bit start address and the dummy byte are then clocked into the transaction that began with `0xAB`.

  The bootloader resides in mask ROM, so there is no workaround on this silicon. Boot in Forth mode instead: hold `BOOT` LOW, load the program into RAM over `UART0` with the ROM Forth interpreter (Section 7), and jump to its entry point. The flash upload procedure of Section 9 is unaffected, as it runs from Forth mode; only booting from the flash fails.

- **The potentiostat control loop oscillates at the counter electrode.** The counter electrode voltage of the `AFE` oscillates. Compensation between the control amplifier A_1 and the current-sensing

branch is insufficient, and the loop is not unconditionally stable.

The sensing mirror sits inside the control loop. A_1 delivers the cell current to CE through the regulated cascode current mirror Q_1–Q_6 (Figure 15), which duplicates that current for the transimpedance amplifier A_2, so the mirror pole lies in the feedback path that holds $V_{RE} = v_{pattern}$ and is not compensated. The mirror is not a passive branch: driving the cell impedance it contributes voltage gain, so the loop closes around two gain stages while only A_1 is compensated. The phase margin above 60° quoted in Section 17 is the control amplifier alone; it does not describe the loop that closes on silicon.

Measurements taken while the loop oscillates are not valid: RE is not held at the programmed potential and the mirrored current does not track the cell current, so both the applied potential and the digitised current are wrong. Chronoamperometry and cyclic voltammetry data acquired in this condition must be discarded. No register trims the control-amplifier compensation — `BIAS_TC_POT` and `BIAS_LC_POT` act on A_2 — so there is no workaround on this silicon.

- **`div`, `divu`, `rem` and `remu` fail when `rd` equals a source register.** With `rd` equal to `rs1` the result is 0. With `rd` equal to `rs2` the result is that of a division by zero. GCC emits these forms routinely. To avoid it, compile with `-mno-div`, which uses the software routines in libgcc, and keep `rd` distinct in hand-written assembly.

- **Compressed stack adjustments and calls are wrong.** `c.addi16sp` with a negative immediate adds 15 too much, so `sp` is wrong after a normal function prologue. `c.jal` and `c.jalr` save `pc+4` instead of `pc+2`, so the return lands 2 bytes too far. To avoid it, do not build with the C extension: use `-march=rv32im_zba_zbb_zbc_zbs`. The code in the mask ROM contains no compressed instructions.

- **`xori` with immediate `0x80` executes as `zext.h`.** The decoder misreads this one encoding, so `rd` silently receives the low 16 bits of `rs1` instead of `rs1` with bit 7 inverted. GCC emits this instruction for `x ^ 0x80` and for byte sign extension, so compiled code can be affected. The mask ROM is not. To avoid it, hold the constant in a register and use `xor`, and search finished images for instruction words where `(word & 0xFFF0707F) == 0x08004013`.

- **Misaligned loads and stores use the aligned location.** The core has no misaligned-access trap. `lw` and `sw` ignore address bits 1:0, and `lh`, `lhu` and `sh` ignore bit 0, so the wrong data is read or written with no error. Correctly typed C is not affected; the usual cause is a byte pointer cast to a wider type. Avoid it with good software practice: keep accesses naturally aligned, and read unaligned fields with byte loads or `memcpy`.

- **`c.lui` with destination `x0` stops the core.** This encoding is a HINT that must execute as a no-op, but the decoder treats it as illegal and the core halts until reset. Compilers do not emit it, and neither the mask ROM nor the toolchain libraries contain it. Do not hand-assemble it.

- **A CSR read-modify-write executed from flash extended memory returns the value it wrote.** The flash fetch of the next instruction freezes the CPU clock during the CSR instruction, so `rd` receives the new value rather than the old one. Only the `mcycle` and `minstret` counters are writable, so the impact is small. To avoid it, read with `csrr` and write with a separate `csrw`.

- **`jalr` does not clear bit 0 of its target.** A jump to an odd address leaves the program counter odd until the next jump to an even address. Execution continues correctly, but `auipc` returns a value one too high in that window. Compiled code does not jump to odd addresses. Software that tags code pointers must clear bit 0 before the jump.

- **The watchdog never fires.** The counter runs, but a defective edge detector means the interrupt flag, the interrupt and the watchdog reset never occur. Use a timer interrupt where a timeout is needed.
- **The UART overflow flag is unreliable.** The receiver checks `TCIF` instead of its own receive flag, so `OVF` stays clear on a real overrun unless `TCIF` is set, and sets falsely on a good byte received while `TCIF` is set. Do not rely on `OVF`; read each received byte before the next frame ends.
- **`NPUTHINK` always reads 0.** The bit is missing from the `NPUCR` readback. This is harmless, because the CPU is stopped while the `NPU` runs, so polling still returns only after it finishes.